TUGAS BESAR I

IF3070 Dasar Intelegensi Artifisial

Optimasi Penempatan Kendaraan Pada Kapal Menggunakan *Local Search*

Oleh,

Keisha Daffa Aryani	18224009
Muhammad Faris Daffa	18224025
Nafhan Shafy Aulia	18224027



SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

2026

DAFTAR ISI

BAB I
DESKRIPSI PERSOALAN..... 2
1.1 Deskripsi Permasalahan..... 2
1.2 Entity & Daftar Variabel..... 2
1.3 Input & Output Program..... 5
1.4 Representasi State..... 6
1.5 Initital State & Neighbor..... 6
1.6 Constraints..... 7
1.7 Final State & Solution Criteria..... 7
BAB II
PERANCANGAN & IMPLEMENTASI SISTEM..... 8
2.1 Gambaran Umum Solusi..... 8
2.2 Fungsi Objektif & Evaluasi State..... 9
2.4 Implementasi Simulated Annealing..... 44
2.5 Implementasi Genetic Algorithm..... 59
2.6 Rancangan Eksperimen..... 71
BAB III
HASIL EKSPERIMEN & ANALISIS..... 81
3.1 Hasil Eksperimen Hill-Climbing..... 81
3.2 Hasil Eksperimen Simulated Annealing..... 111
3.3 Hasil Eksperimen Genetic Algorithm..... 120
3.4 Perbandingan Hasil Antar Algoritma..... 171
BAB IV
KESIMPULAN & SARAN..... 176
4.1 Hasil Eksperimen Hill-Climbing..... 176
4.2 Saran..... 177
BAB V
PEMBAGIAN TUGAS..... 178
REFERENSI..... 180
LAMPIRAN..... 182

BAB I
DESKRIPSI PERSOALAN

1.1 Deskripsi Permasalahan

PT Intim Line perlu menentukan kendaraan yang akan diangkut sekaligus mengatur posisi dan orientasinya pada geladak kapal. Kendaraan tersebut memiliki ukuran, berat, dan *shipping fee* yang berbeda. Dikarenakan luas geladak serta kapasitas angkutnya terbatas, tidak semua kendaraan dapat dimuat bersamaan. Penataan yang memenuhi batasan belum tentu menghasilkan pendapatan pengiriman terbesar. Persoalan ini dapat dimodelkan sebagai optimasi kombinatorial pada bidang dua dimensi, dengan keputusan yang dicari meliputi pemilihan kendaraan, koordinat penempatan, dan orientasi setiap kendaraan yang dimuat. Tujuannya adalah untuk memaksimalkan jumlah *shipping fee* tanpa menimbulkan tumpang tindih dan tetap sesuai pada constraints yang ada (tidak melampaui batas geladak atau melebihi kapasitas berat kapal).

Dalam penyelesaiannya, digunakan pendekatan *local search* untuk mengeksplorasi perubahan penataan secara bertahap yaitu Hill-Climbing, Simulated Annealing, dan Genetic Algorithm. Ketiganya menggunakan model persoalan serta fungsi objektif yang sama agar perbedaan hasil dapat dikaji dari strategi pencariannya.

1.2 Entity & Daftar Variabel

Berikut variabel yang digunakan dan menjadi fokus pada permasalahannya. Ship dan Vehicle menyimpan spesifikasi persoalan, sedangkan Placement menyimpan *decision* yang berubah selama *searching*.

Tabel Daftar Spesifikasi Variabel

Entitas	Atribut	Contoh	Keterangan
Ship	width, length, max_capacity	10, 20, 500	Ukuran geladak dan batas total berat.
Vehicle	id, width, length, shipping_fee, weight, eta	V01, 1, 4, 19, 48, 5	Spesifikasi kendaraan. ETA belum dipakai dalam evaluasi.

Placement	loaded, x, y, orientation	True, 2, 3, "H"	Keputusan penempatan. UNLOADED = False, None, H.

Adapun pemetaan dari daftar variabel dengan implementasi pada kode adalah sebagai berikut:

Tabel 1.2 Pemetaan Implementasi Kode

Kode	**Deskripsi**
Vehicle, Ship, Problem src/core.py	Mendefinisikan data kendaraan, dimensi dan kapasitas kapal, serta persoalan yang akan diselesaikan. Input diperiksa saat objek Problem dibentuk.
Placement, UNLOADED src/core.py	Merepresentasikan status muat, posisi, dan orientasi kendaraan. Satu state berisi tuple Placement sesuai urutan kendaraan, sedangkan UNLOADED menandai kendaraan yang tidak dimuat.
RunResult, snapshot() src/result.py	Mencatat hasil run, mulai dari konfigurasi, seed, solusi awal dan akhir, hingga history dan durasi. snapshot() menyimpan kondisi penempatan beserta nilai objektif, berat, jumlah kendaraan, dan status kelayakannya.
Algoritma Local Search	
hc_stochastic() src/algorithms/hc_stochastic.py	Mengambil satu neighbor acak pada setiap iterasi dan berpindah jika nilai objektifnya lebih tinggi.
hc_sideways(), hc_random_restart(), hc_stochastic_aima() src/algorithms/hc_sideways.py src/algorithms/hc_random_restart.py src/algorithms/hc_stochastic_aima.py	Menjalankan varian HC dengan sideways move, random restart, dan pemilihan acak dari kumpulan neighbor yang lebih baik.
simulated_annealing() src/algorithms/simulated_annealing.py	Menjalankan SA dengan temperatur yang menurun secara bertahap. Kandidat yang lebih buruk dapat diterima berdasarkan probabilitasnya, sementara solusi terbaik tetap dicatat terpisah dari solusi akhir.

GAConfig, genetic_algorithm() src/algorithms/genetic_algorithm.py	Mendefinisikan parameter GA dan menjalankan proses optimasi berbasis populasi, dari pembentukan populasi awal hingga generasi terakhir.
Helper Function (Fungsi Pembantu)	
footprint(), rect(), inside_deck(), overlaps() src/core.py	Menghitung ukuran dan area kendaraan sesuai orientasinya, lalu memeriksa apakah penempatannya berada di dalam geladak dan bertumpang tindih dengan kendaraan lain.
validate_state(), capacity_ok(), constraint_violations(), is_feasible() src/core.py	Memeriksa struktur state, batas geladak, tumpang tindih, dan kapasitas berat untuk menentukan apakah solusi valid.
objective(), total_weight(), loaded_count() src/core.py	Menghitung total shipping fee, berat muatan, dan jumlah kendaraan yang dimuat.
load_problem(), problem_from_dict(), problem_to_dict(), state_to_dict(), state_from_dict() src/core.py	Membaca data input dan mengubah data persoalan maupun state ke format yang digunakan saat menyimpan atau memuat hasil.
random_feasible_state() src/search_space.py	Membentuk penempatan awal secara acak dengan tetap memenuhi seluruh hard constraints.
swap(), relocate(), rotate() src/search_space.py	Mengubah penempatan melalui pertukaran dua kendaraan, pemindahan posisi atau perubahan status muat, serta rotasi kendaraan.
random_feasible_neighbor(), generate_feasible_neighbors(), describe_move() src/search_space.py	Mengambil satu feasible neighbor acak, menelusuri seluruh feasible neighbor satu langkah, dan mencatat perubahan antara dua state.
temperature_at(), acceptance_probability() src/algorithms/simulated_annealing.py	Menghitung temperatur pada iterasi berjalan dan peluang kandidat diterima oleh SA.
is_local_optimum(), check_stuck_history(), make_log()	Memeriksa kondisi local optimum, menghitung episode stuck dan escape, serta mencatat hasil setiap

src/algorithms/simulated_annealing.py	iterasi.
roulette_index(), one_point_crossover(), mutate(), repair(), next_generation() src/algorithms/genetic_algorithm.py	Memilih induk, melakukan crossover dan mutasi, memperbaiki individu yang tidak valid, serta membentuk generasi berikutnya.
Eksperimen dan Visualisasi	
run_repeated(), run_experiments() src/experiments.py	Menjalankan pengujian sesuai konfigurasi dan daftar seed, lalu menyimpan hasil tiap run, tabel rekap, dan statistik eksperimen.
draw_deck(), plot_run(), plot_sa_run() src/experiments.py	Menggambar penempatan kendaraan dan grafik hasil searching. Grafik SA mencakup perkembangan nilai objektif, temperatur, dan probabilitas penerimaan.
check_result(), main() src/main.py	Membaca instruksi dari terminal untuk menjalankan algoritma, memulai eksperimen, atau memeriksa hasil yang sudah disimpan.
create_app(), parse_run_request() src/server.py	Menghubungkan UI dengan program Python, memeriksa parameter yang dimasukkan, dan mengirim hasil solver ke browser.
frontend/index.html, app.js, deck.js, charts.js, runmodel.js	Menampilkan pengaturan algoritma, ringkasan hasil, penempatan kendaraan, grafik, dan replay proses searching pada aplikasi.

1.3 Input & Output Program

Program menerima data persoalan dalam format JSON yang memuat spesifikasi kapal dan daftar kendaraan yang terdiri atas width, length, dan max_capacity, sedangkan setiap kendaraan memiliki id, width, length, shipping_fee, weight, dan eta. Konfigurasi eksperimennya memuat algoritma yang digunakan, parameter searching, daftar seed, lokasi data, dan folder keluaran. Untuk konfigurasi, standarnya kapal berukuran 10 × 20 dengan kapasitas berat 500 dan terdapat 30 kendaraan, dari V01 hingga V30. Satuan ukuran, berat, dan shipping fee mengikuti satuan model yang digunakan. Setiap algoritma menghasilkan RunResult yang berisi seed, konfigurasi, initial state, final state, nilai objektif awal dan akhir, durasi, riwayat searching, serta metrik tambahan.

1.4 Representasi State

Satu state merepresentasikan satu susunan kendaraan pada geladak kapal. Implementasinya menggunakan tuple Placement dengan panjang yang sama dengan jumlah kendaraan. Elemen ke-i selalu mewakili kendaraan ke-i pada problem.vehicles. Setiap Placement menyimpan status loaded, koordinat x dan y, serta orientation. Kendaraan yang tidak dimuat menggunakan UNLOADED, yaitu Placement(False, None, None, "H"). Sebagai contoh:

```
(Placement(True, 2, 3, "H"), UNLOADED, Placement(True, 5, 1, "V"), ...)
```

Pada contoh tersebut, *vehicle* pertama dimuat pada koordinat (2, 3) dengan orientasi horizontal, *vehicle* kedua tidak dimuat, dan *vehicle* ketiga dimuat secara vertikal pada (5, 1). Titik (0, 0) berada di pojok kiri atas geladak. Nilai x bertambah ke kanan dan y bertambah ke bawah. H menggunakan ukuran asli kendaraan, sedangkan V menukar width dan length. Data seperti berat, *shipping fee*, dan ETA tetap disimpan pada objek Vehicle, bukan di dalam Placement.

1.5 Initital State & Neighbor

Initial state dibentuk secara acak melalui random_feasible_state(problem, rng). Program memulai dari seluruh kendaraan dalam kondisi UNLOADED, mengacak urutan kendaraan, lalu mencoba posisi dan orientasi acak paling banyak 20 kali untuk setiap kendaraan. Penempatan hanya diterima jika berada di dalam geladak, tidak bertumpang tindih, dan tidak melampaui kapasitas berat. Kendaraan yang tidak dapat ditempatkan tetap berada di luar kapal. Seed mengatur proses pengacakan agar hasil dapat direproduksi pada lingkungan dan versi program yang sama. Program kami membentuk neighbor melalui tiga operator:

- swap, menukar penempatan dua kendaraan
- relocate, memindahkan kendaraan, memasukkan kendaraan dari luar kapal, atau mengeluarkannya dari kapal
- rotate, mengubah orientasi kendaraan yang sedang dimuat.

Setiap kandidat diperiksa kembali sebelum digunakan. Kandidat yang melanggar batas geladak, bertumpang tindih, atau melebihi kapasitas berat akan ditolak. Selain itu ada pembangkitan satu neighbor acak maupun seluruh feasible neighbor satu langkah sesuai kebutuhan algoritma.

1.6 Constraints

Setiap *state* harus memenuhi tiga *hard constraints*:

1. Kendaraan yang dimuat harus berada sepenuhnya di dalam geladak. Untuk kapal berukuran lebar W dan panjang L, kendaraan pada posisi (x, y) dengan ukuran efektif $w \times l$ harus memenuhi: $0 \leq x, 0 \leq y, x + w \leq W,\ y + l \leq L$
2. Dua kendaraan yang dimuat tidak boleh saling bertumpang tindih. Kendaraan boleh bersentuhan pada sisi atau sudut karena tidak ada area yang saling menutupi.
3. Total berat kendaraan yang dimuat tidak boleh melebihi kapasitas kapal:

$$\sum_{i=1}^{n} z_i \times weight_i \leq max_capacity$$

dengan $z_i = 1$ apabila kendaraan ke-i dimuat dan $z_i = 0$ apabila kendaraan tersebut tidak dimuat. ETA disimpan sebagai atribut kendaraan, tetapi belum digunakan dalam *constraint* maupun fungsi objektif. Dengan demikian, kelayakan suatu *state* ditentukan oleh posisi kendaraan, kemungkinan tumpang tindih, dan total berat muatan.

1.7 Final State & Solution Criteria

Solusi dinyatakan valid apabila seluruh kendaraan yang dimuat berada di dalam geladak, tidak bertumpang tindih, dan total beratnya tidak melebihi kapasitas kapal. Kualitas solusi dinilai dari total *shipping fee* kendaraan yang dimuat:

$$f(s) = \sum_{i=1}^{n} z_i \times shipping_fee_i$$

dengan $z_i = 1$ apabila kendaraan ke-i dimuat dan $z_i = 0$ apabila kendaraan tersebut tidak dimuat. Semakin tinggi nilai fungsi objektif, semakin baik solusi tersebut. *Final state* adalah *state* saat algoritma berhenti. Kondisi berhenti bergantung pada algoritma dan parameternya, seperti batas iterasi, temperatur minimum, atau jumlah generasi. Pada Simulated Annealing, *final state* dapat berbeda dari *best-found state* karena algoritma dapat menerima kandidat bernilai lebih rendah selama *searching*.

BAB II
PERANCANGAN & IMPLEMENTASI SISTEM

2.1 Gambaran Umum Solusi

Program dirancang untuk menyelesaikan permasalahan pemilihan dan penataan kendaraan pada geladak kapal. Setiap kendaraan memiliki dimensi, berat, *shipping fee*, dan ETA yang berbeda. Program harus menentukan kendaraan yang akan dimuat, posisi penempatannya, serta orientasinya dengan tetap memenuhi batas dimensi geladak, larangan tumpang tindih, dan kapasitas berat kapal. Proses searching diawali dengan membentuk *initial state* yang valid. Seluruh kendaraan mula-mula berada di luar kapal, kemudian program mencoba menempatkannya satu per satu pada posisi dan orientasi yang dipilih secara acak. Setiap penempatan diperiksa terlebih dahulu agar tidak melanggar ketiga *hard constraints*. Hasil dari proses ini menjadi solusi awal bagi algoritma pencarian.

Optimasi dilakukan menggunakan Hill-Climbing, Simulated Annealing, dan Genetic Algorithm. Ketiga algoritma menggunakan representasi *state*, fungsi objektif, dan aturan validasi yang sama agar hasilnya dapat dibandingkan secara konsisten. Perbedaan ketiganya terletak pada cara menghasilkan kandidat solusi, memilih kandidat yang diterima, serta menentukan kondisi berhenti pencarian. Hill-Climbing bergerak menuju *neighbor* dengan nilai objektif yang lebih baik. Simulated Annealing dapat menerima solusi yang lebih buruk dengan probabilitas tertentu agar pencarian tidak mudah terhenti pada *local optimum*. Genetic Algorithm mengembangkan sekumpulan solusi melalui proses seleksi, *crossover*, mutasi, dan pemilihan generasi berikutnya.

Hasil setiap pencarian memuat *initial state*, solusi akhir, nilai fungsi objektif, jumlah iterasi, dan durasi eksekusi. Posisi kendaraan juga divisualisasikan dalam bentuk grid dua dimensi agar perubahan antara kondisi awal dan hasil akhir dapat diamati secara langsung. Secara umum, alur penyelesaian yang digunakan adalah sebagai berikut:

1. Membaca data kapal, kendaraan, dan parameter algoritma.
2. Membentuk representasi Problem berdasarkan data masukan.
3. Menghasilkan *initial state* yang memenuhi seluruh *constraints*.

4. Menjalankan algoritma pencarian yang dipilih.
5. Memeriksa validitas solusi yang dihasilkan.
6. Menghitung total *shipping fee* dan metrik eksperimen.
7. Menampilkan serta menyimpan hasil pencarian.

2.2 Fungsi Objektif & Evaluasi State

2.2.1 Objective Function Utama

Fungsi objektif digunakan untuk mengukur kualitas suatu *state*. Pada permasalahan ini, kualitas solusi ditentukan berdasarkan total *shipping fee* dari seluruh kendaraan yang berhasil dimuat ke dalam kapal. Semakin besar total *shipping fee*, semakin baik solusi tersebut. Misalkan terdapat n kendaraan. Variabel z_i bernilai 1 apabila kendaraan ke-i dimuat dan bernilai 0 apabila kendaraan tersebut tidak dimuat. Jika c_i merupakan *shipping fee* kendaraan ke-i, fungsi objektif dirumuskan sebagai berikut:

$$\max\ f(s) = \sum_{i=1}^{n} c_i z_i$$

Maksimisasi tersebut dilakukan terhadap *state* yang memenuhi seluruh *hard constraints*. Posisi dan orientasi kendaraan tidak memberikan nilai secara langsung pada fungsi objektif, tetapi menentukan apakah kendaraan dapat ditempatkan secara valid. Oleh karena itu, dua susunan kendaraan dapat memiliki nilai objektif yang sama apabila keduanya memuat kumpulan kendaraan yang sama. Komponen yang digunakan dalam evaluasi *state* dapat dirangkum sebagai berikut.

Komponen	Perlakuan dalam Program	Alasan
Total shipping fee	Dijumlahkan sebagai nilai fungsi objektif	Menjadi ukuran utama kualitas solusi yang dimaksimalkan
Batas dimensi geladak	Diperiksa sebagai hard constraint	Seluruh bagian kendaraan harus berada di dalam kapal

Tumpang tindih kendaraan	Diperiksa sebagai hard constraint	Dua kendaraan tidak boleh menempati area yang sama
Kapasitas berat kapal	Diperiksa sebagai hard constraint	Total berat kendaraan tidak boleh melebihi kapasitas kapal
Posisi dan orientasi	Menentukan validitas penempatan	Mempengaruhi ruang yang ditempati oleh kendaraan
ETA	Disimpan sebagai atribut kendaraan	Belum digunakan dalam fungsi objektif maupun constraints pada implementasi saat ini

2.2.2 Perhitungan Total Shipping Fee

Perhitungan total *shipping fee* diimplementasikan dalam fungsi objective() pada src/core.py. Fungsi tersebut memasangkan setiap kendaraan dengan Placement yang bersesuaian, kemudian menjumlahkan *shipping fee* kendaraan yang memiliki status loaded=True.

```
def objective(state, problem):
    return sum(
        vehicle.shipping_fee
        for vehicle, placement in zip(problem.vehicles, state)
        if placement.loaded
    )
```

Tahapan perhitungannya adalah sebagai berikut:

1. Fungsi menerima state yang akan dinilai dan problem yang menyimpan seluruh data kendaraan.
2. Fungsi zip() memasangkan kendaraan ke-i dengan Placement ke-i.
3. Status loaded pada setiap Placement diperiksa.
4. Jika loaded=True, nilai shipping_fee kendaraan ditambahkan ke total.
5. Jika loaded=False, kendaraan tidak memberikan nilai pada fungsi objektif.
6. Setelah seluruh kendaraan diperiksa, fungsi mengembalikan jumlah *shipping fee* sebagai nilai objektif dari *state* tersebut.

Sebagai contoh, apabila suatu *state* memuat tiga kendaraan dengan *shipping fee* masing-masing sebesar 80, 120, dan 150, sedangkan kendaraan lainnya berada di luar kapal, nilai objektifnya adalah:

$$f(s) = 80 + 120 + 150 = 350$$

Nilai tersebut tidak bergantung pada posisi masing-masing kendaraan selama seluruh penempatan tetap valid. Posisi dan orientasi baru memengaruhi nilai secara tidak langsung ketika keterbatasan ruang membuat suatu kendaraan dapat atau tidak dapat dimuat.

2.2.3 Penanganan State yang Tidak Valid

Pelanggaran terhadap *constraints* tidak diubah menjadi penalti numerik dalam fungsi objektif. Setiap kandidat solusi diperiksa terlebih dahulu dan hanya kandidat yang memenuhi seluruh batasan yang dapat digunakan dalam proses pencarian. Pemeriksaan validitas dilakukan terhadap tiga kondisi utama:

1. Seluruh kendaraan yang dimuat berada di dalam batas geladak.
2. Tidak terdapat dua kendaraan yang menempati area yang sama.
3. Total berat kendaraan yang dimuat tidak melebihi kapasitas kapal.

Jika salah satu kondisi tersebut tidak terpenuhi, kandidat dinyatakan tidak valid dan tidak diterima sebagai *state* berikutnya. Dengan demikian, fungsi objektif dapat difokuskan untuk membandingkan nilai ekonomi antar solusi yang sudah valid.

2.2.4 Alasan Pemilihan Fungsi Objektif

Total *shipping fee* dipilih sebagai fungsi objektif karena merepresentasikan nilai ekonomi dari kendaraan yang berhasil diangkut. Kendaraan memiliki ukuran, berat, dan *shipping fee* yang berbeda sehingga memaksimalkan jumlah kendaraan belum tentu menghasilkan pendapatan terbesar. Sebagai contoh, beberapa kendaraan berukuran kecil dapat menghasilkan jumlah kendaraan yang lebih banyak, tetapi total *shipping fee*-nya dapat lebih rendah dibandingkan kombinasi kendaraan lain dengan jumlah lebih sedikit. Hal yang sama berlaku untuk luas geladak dan kapasitas berat. Penggunaan ruang atau

kapasitas yang lebih besar tidak selalu menunjukkan bahwa solusi tersebut memiliki nilai ekonomi yang lebih tinggi.

2.3 Implementasi Hill-Climbing

Hill-Climbing diimplementasikan dalam empat varian, masing-masing pada berkas Python tersendiri di folder src/algorithms/. Keempat varian tersebut adalah stochastic hill-climbing yang mengambil satu neighbor acak pada tiap iterasi, hill-climbing with sideways move, random restart hill-climbing, dan stochastic hill-climbing menurut definisi pada buku teks Artificial Intelligence: A Modern Approach (4th Edition) (Russell & Norvig, 2021). Seluruh konsep hill-climbing yang dipakai mengacu pada buku teks tersebut. Seluruh varian memakai model persoalan, fungsi objektif, dan pembangkit neighbor yang sama sehingga perbedaan hasil dapat dikaitkan dengan strategi pencariannya. Ringkasan varian dapat dilihat pada Tabel 2.4.

Tabel Varian Hill-Climbing yang Diimplementasikan

Varian	Sumber	Berkas	Deskripsi
Stochastic Hill-Climbing	Russell & Norvig (2021), Sec. 4.1 (first-choice, dengan penyesuaian)	hc_stochastic.py	Mengambil satu neighbor acak pada tiap iterasi dan berpindah hanya jika neighbor tersebut lebih baik. Berhenti setelah max_iterations.
Hill-Climbing with Sideways Move	Russell & Norvig (2021), Sec. 4.1	hc_sideways.py	Memeriksa seluruh neighbor, berpindah ke neighbor terbaik, dan mengizinkan perpindahan ke nilai yang sama (sideways) hingga batas max_sideways beruntun.

Varian	Sumber	Berkas	Deskripsi
Random Restart Hill-Climbing	Russell & Norvig (2021), Sec. 4.1	hc_random_restart.py	Mengulang steepest ascent dari state awal acak yang baru sebanyak max_restarts kali dan menyimpan hasil terbaik.
Stochastic Hill-Climbing (Versi Buku Teks)	Russell & Norvig (2021), Sec. 4.1	hc_stochastic_aima.py	Mengumpulkan seluruh uphill neighbor lalu memilih satu secara acak, dengan peluang sebanding dengan kecuramannya.

2.3.1 Konsep, Alur Algoritma dan Parameter yang Digunakan

Hill-Climbing adalah metode local search yang menjaga satu state saat ini dan bergerak ke neighbor yang memiliki nilai objektif lebih baik. Karena persoalan ini memaksimalkan total shipping fee, neighbor dikatakan lebih baik apabila nilai f(s) lebih besar. Pencarian tidak dapat turun ke nilai yang lebih rendah sehingga dapat terhenti pada local optimum atau plateau. Keempat varian memakai pembangkit neighbor yang sama pada search_space.py (swap, relocate, dan rotate; lihat subbab 1.5), tetapi berbeda dalam cara memeriksa neighbor. Stochastic hill-climbing pada hc_stochastic.py hanya mengambil satu neighbor acak per iterasi melalui random_feasible_neighbor(). Ketiga varian lainnya memeriksa seluruh neighbor feasible satu langkah melalui generate_feasible_neighbors(), sehingga biaya per iterasi jauh lebih besar tetapi keputusan tidak bergantung pada satu sampel acak.

2.3.1.1 Stochastic Hill-Climbing

Pada varian ini, setiap iterasi membangkitkan satu neighbor secara acak (tidak seluruh neighbor), lalu berpindah ke neighbor tersebut apabila nilainya lebih baik daripada state saat ini. Neighbor yang bernilai sama atau lebih buruk ditolak. Varian ini berkerabat dengan first-choice hill-climbing pada buku teks, yaitu membangkitkan

successor secara acak dan hanya menerima yang lebih baik (Russell & Norvig, 2021, Sec. 4.1). Perbedaannya, first-choice hill-climbing terus membangkitkan successor sampai menemukan yang lebih baik, sedangkan implementasi ini mengambil satu neighbor per iterasi sehingga iterasi dapat terpakai tanpa perpindahan. Karena satu neighbor acak yang ditolak tidak membuktikan bahwa state sudah berada pada peak, pencarian dibatasi oleh max_iterations. Varian ini umumnya membutuhkan lebih banyak langkah dibandingkan steepest ascent. Buku teks memakai istilah stochastic hill-climbing untuk definisi lain, yaitu memilih acak di antara uphill move, dan versi itu diimplementasikan terpisah pada subbab 2.3.1.4.

```
current_state <- state awal acak yang feasible
ulangi sebanyak max_iterations kali:
    neighbor_state <- SATU neighbor acak dari current_state
    jika nilai(neighbor_state) > nilai(current_state):
        current_state <- neighbor_state
```

Alur kerja varian ini adalah sebagai berikut.

1. Membaca dan memeriksa konfigurasi (max_iterations dan record_snapshots).
2. Membuat satu generator acak dari seed dan menyiapkan penghitung serta iteration_log.
3. Membentuk initial state acak yang feasible dan mencatatnya sebagai iterasi ke-0.
4. Pada tiap iterasi, mengambil satu neighbor acak yang feasible. Jika tidak ditemukan, iterasi dicatat tanpa perubahan state.
5. Menghitung nilai objektif neighbor. Neighbor yang lebih baik diterima, sedangkan yang bernilai sama atau lebih buruk ditolak.
6. Mengubah iteration_log menjadi history, menghitung metrics, lalu mengemas seluruhnya ke dalam RunResult.

Tabel Parameter Stochastic Hill-Climbing

Parameter	Default	Eksperimen	Keterangan
max_iterations	wajib diisi	2000	Jumlah iterasi pencarian. Integer ≥ 0.

Parameter	Default	Eksperimen	Keterangan
record_snapshots	true	true	Menyimpan snapshot state pada iterasi 0 dan setiap kali state berubah.
seed	-	3001, 3002, 3003	Satu seed untuk setiap run agar hasil dapat direproduksi.
tries_per_vehicle	20	20	Percobaan penempatan acak per kendaraan saat membentuk initial state (tertanam di search_space.py).
max_tries	200	200	Percobaan maksimum mencari satu neighbor feasible per iterasi (tertanam di search_space.py).

2.3.1.2 Hill-Climbing with Sideways Move

Pada persoalan ini nilai objektif hanya berubah ketika himpunan kendaraan yang dimuat berubah, sedangkan perubahan posisi atau orientasi tidak mengubah nilai selama seluruh kendaraan tetap termuat. Akibatnya terdapat banyak plateau. Varian sideways move mengizinkan perpindahan ke neighbor yang bernilai sama agar pencarian dapat bergerak melintasi plateau dan menemukan jalan keluar yang membaik (Russell & Norvig, 2021, Sec. 4.1). Agar pencarian tidak berputar tanpa akhir pada flat local maximum, jumlah sideways move dibatasai oleh max_sideways. Pada tiap iterasi, seluruh neighbor feasible diperiksa dan neighbor terbaik dipilih. Apabila terdapat beberapa neighbor yang sama baiknya, satu di antaranya dipilih secara acak. Batas max_sideways dihitung sebagai jumlah sideways move yang terjadi berturut-turut dan kembali ke 0 setelah ada langkah yang membaik. Penafsiran ini mengikuti buku teks Artificial Intelligence: A Modern Approach (4th Edition), yang membatasi sideways move yang terjadi berturut-turut. Spesifikasi tugas menuliskan "banyak sideways move yang dilakukan sudah mencapai maksimum", yang dapat dibaca sebagai jumlah total, sehingga jumlah total tetap dicatat pada metrics (sideways_moves_total).

```
current_state <- state awal acak yang feasible
consecutive_sideways <- 0
ulangi sampai berhenti:
    best_neighbor_value <- nilai neighbor terbaik dari current_state
    jika best_neighbor_value > nilai(current_state):
        pindah ke salah satu neighbor terbaik (acak)
        consecutive_sideways <- 0
    jika best_neighbor_value == nilai(current_state):
        jika consecutive_sideways >= max_sideways: berhenti
        selain itu: pindah ke salah satu neighbor seri (acak)
                    consecutive_sideways <- consecutive_sideways + 1
    jika best_neighbor_value < nilai(current_state) atau tidak ada neighbor:
        berhenti (local optimum)
    jika jumlah iterasi mencapai max_iterations: berhenti
```

Alur kerja varian ini adalah sebagai berikut.

1. Membaca dan memeriksa konfigurasi (max_sideways, max_iterations, record_snapshots).
2. Menyiapkan generator acak, penghitung sideways, dan iteration_log.
3. Membentuk initial state acak yang feasible dan mencatatnya sebagai iterasi ke-0.
4. Pada tiap iterasi, memeriksa seluruh neighbor feasible dan menentukan nilai terbaik beserta semua neighbor yang seri.
5. Jika nilai terbaik lebih besar, berpindah dan mengatur ulang penghitung sideways beruntun. Jika sama, berpindah sebagai sideways move selama batas belum tercapai. Jika lebih kecil atau tidak ada neighbor, pencarian berhenti.
6. Mengubah catatan menjadi history, menghitung metrics, lalu mengemas hasil ke dalam RunResult.

Tabel Parameter Hill-Climbing with Sideways Move

Parameter	Default	Eksperimen	Keterangan
max_sideways	100	100	Batas sideways move beruntun. Integer ≥ 0. Nilai 100 mengikuti contoh pada buku teks (persoalan 8-queens).

Parameter	Default	Eksperimen	Keterangan
max_iterations	1000	1000	Batas pengaman jumlah iterasi. Integer ≥ 0.
record_snapshots	true	true	Menyimpan snapshot state saat state berubah.
seed	-	3201, 3202, 3203	Satu seed untuk setiap run.

2.3.1.3 Random Restart Hill-Climbing

Random restart hill-climbing mengatasi ketergantungan hasil pada satu initial state dengan menjalankan serangkaian pencarian hill-climbing dari initial state acak yang berbeda. Setiap restart memakai steepest ascent murni (climb_to_peak), yaitu memeriksa seluruh neighbor pada tiap langkah dan berpindah ke neighbor terbaik hingga mencapai peak atau batas langkah per restart. Hasil terbaik dari seluruh restart menjadi solusi akhir. Hasil sebuah restart hanya menggantikan solusi terbaik apabila nilainya benar-benar lebih besar, sehingga jika terjadi seri, hasil yang lebih awal dipertahankan.

```
best_state <- kosong
ulangi max_restarts kali:
    initial_state <- state awal acak yang baru
    peak_state <- climb_to_peak(initial_state, iterations_per_restart)
    jika nilai(peak_state) > nilai(best_state):
        best_state <- peak_state
kembalikan best_state
```

Alur kerja varian ini adalah sebagai berikut.

1. Membaca dan memeriksa konfigurasi (max_restarts, iterations_per_restart, record_snapshots).
2. Menyiapkan generator acak, daftar hasil per restart, dan iteration_log.

3. Untuk setiap restart, membentuk initial state acak yang baru dan mencatatnya dengan alasan restart_initial.
4. Menjalankan climb_to_peak dari state tersebut sampai mencapai peak atau batas langkah per restart.
5. Memindahkan catatan langkah ke iteration_log dengan nomor iterasi global yang berlanjut lintas restart, dan memperbarui nilai terbaik sejauh ini.
6. Setelah seluruh restart selesai, mengubah catatan menjadi history, menghitung metrics, dan mengemas hasil dengan state terbaik sebagai final_state.

Tabel Parameter Random Restart Hill-Climbing

Parameter	Default	Eksperimen	Keterangan
max_restarts	10	100	Jumlah restart. Integer ≥ 1. Pencarian berhenti setelah jumlah ini tercapai.
iterations_per_restart	1000	1000	Batas langkah pada tiap restart. Integer ≥ 0.
record_snapshots	true	true	Menyimpan snapshot state saat state berubah.
seed	-	3301, 3302, 3303	Satu seed untuk setiap run. Seluruh restart memakai satu generator acak yang dilanjutkan.

2.3.1.4 Stochastic Hill-Climbing

Buku teks artificial intelligence: a modern approach (4th edition) mendefinisikan stochastic hill-climbing sebagai pemilihan acak di antara uphill move, dengan peluang pemilihan yang dapat bergantung pada kecuramannya (Russell & Norvig, 2021, Sec. 4.1). Definisi ini berbeda dari varian pada subbab 2.3.1.1. Varian tersebut mengambil satu neighbor acak lalu menerimanya hanya jika lebih baik, sedangkan versi buku teks mengumpulkan seluruh uphill neighbor terlebih dahulu lalu memilih satu di antaranya.

Kedua varian diimplementasikan karena keduanya merupakan bentuk stochastic hill-climbing yang bekerja dengan cara berbeda, sehingga hasilnya dapat dibandingkan. Terdapat dua mode pemilihan. Pada mode proportional, bobot setiap uphill neighbor adalah selisih nilainya terhadap state saat ini, sehingga neighbor yang lebih curam lebih besar peluangnya. Bobot selalu positif karena hanya neighbor dengan nilai lebih besar yang dikumpulkan. Pada mode uniform, seluruh uphill neighbor berpeluang sama.

```
current_state <- state awal acak yang feasible
ulangi sampai berhenti:
    uphill_neighbors <- semua neighbor feasible dengan nilai >
nilai(current_state)
    jika uphill_neighbors kosong: berhenti (local optimum)
    pilih satu neighbor dari uphill_neighbors sesuai mode selection, lalu
pindah
    jika jumlah iterasi mencapai max_iterations: berhenti
```

Alur kerja varian ini adalah sebagai berikut.

1. Membaca dan memeriksa konfigurasi (max_iterations, selection, record_snapshots).
2. Menyiapkan generator acak, penghitung, dan iteration_log.
3. Membentuk initial state acak yang feasible dan mencatatnya sebagai iterasi ke-0.
4. Pada tiap iterasi, mengumpulkan seluruh uphill neighbor. Jika tidak ada, pencarian berhenti pada local optimum.
5. Memilih satu uphill neighbor sesuai mode selection dan berpindah ke neighbor tersebut.
6. Mengubah catatan menjadi history, menghitung metrics, lalu mengemas hasil ke dalam RunResult.

Tabel Parameter Stochastic Hill-Climbing Versi Buku Teks

Parameter	Default	Eksperimen	Keterangan
max_iterations	1000	1000	Batas pengaman jumlah iterasi. Integer ≥ 0.

Parameter	Default	Eksperimen	Keterangan
selection	"proportional"	"proportional"	Mode pemilihan: "proportional" (peluang sebanding dengan selisih nilai) atau "uniform".
record_snapshots	true	true	Menyimpan snapshot state saat state berubah.
seed	-	3401, 3402, 3403	Satu seed untuk setiap run.

2.3.2 Kode dan Penjelasan Implementasi

Setiap fungsi varian ditulis dengan enam bagian yang berurutan dan sama pada keempat berkas, yaitu (1) membaca dan memeriksa config, (2) mendefinisikan variabel awal, (3) logika algoritma, (4) mengubah catatan iterasi menjadi history, (5) meringkas metrics, dan (6) mengemas hasil menjadi RunResult. Penjelasan dimulai dari komponen bersama, kemudian tiap varian secara berurutan. Pada varian setelah subbab 2.4.2.2, bagian yang identik dengan subbab tersebut tidak diulang.

2.3.2.1 Komponen Bersama (hc_common.py)

Berikut tiga fungsi pada hc_common.py. Docstring pada kode asli tidak ditampilkan agar ringkas. Fungsi check_integer_config ditampilkan berikut.

```
def check_integer_config(name, value, minimum):
    # True/False di Python dianggap angka juga (bool turunan int), jadi
harus ditolak manual
    is_valid_number = isinstance(value, int) and not isinstance(value, bool)
    if not is_valid_number or value < minimum:
        raise ValueError(f"{name} harus integer >= {minimum}")
    return value
```

1. Pada Python, True dan False termasuk bilangan bulat karena bool merupakan turunan int. Oleh karena itu bool ditolak secara eksplisit.

2. Jika nilai bukan integer atau lebih kecil dari batas minimum, fungsi melempar ValueError dengan pesan yang memuat nama parameter dan batasnya.
3. Jika valid, nilai dikembalikan sehingga fungsi dapat dipakai langsung pada penugasan variabel.

Fungsi find_best_neighbors ditampilkan berikut.

```
def find_best_neighbors(state, problem):
    best_neighbor_value = None
    tied_best_neighbors = []
    neighbors_evaluated = 0

    for move, neighbor_state in generate_feasible_neighbors(state, problem):
        neighbor_value = objective(neighbor_state, problem)
        neighbors_evaluated += 1

        # Kasus 1: lebih baik dari yang terbaik sejauh ini, daftar seri
dimulai ulang
        if best_neighbor_value is None or neighbor_value >
best_neighbor_value:
            best_neighbor_value = neighbor_value
            tied_best_neighbors = [(move, neighbor_state)]

        # Kasus 2: sama dengan yang terbaik, masuk daftar seri
        elif neighbor_value == best_neighbor_value:
            tied_best_neighbors.append((move, neighbor_state))

    return best_neighbor_value, tied_best_neighbors, neighbors_evaluated
```

1. Fungsi menyiapkan best_neighbor_value, tied_best_neighbors, dan neighbors_evaluated.
2. generate_feasible_neighbors menghasilkan seluruh neighbor feasible satu langkah (swap, rotate, relocate) secara deterministik dan tanpa duplikat.
3. Setiap neighbor dinilai dengan objective() dan neighbors_evaluated bertambah satu.
4. Kasus 1: apabila neighbor lebih baik daripada yang terbaik sejauh ini, nilai terbaik diperbarui dan daftar seri dimulai ulang. Kasus 2: apabila nilainya sama dengan yang terbaik, neighbor dimasukkan ke daftar seri.
5. Fungsi mengembalikan nilai terbaik, daftar neighbor yang seri, dan jumlah neighbor yang dinilai. Jika tidak ada neighbor feasible, hasilnya (None, [], 0).

6. Pemilihan satu neighbor di antara yang seri dilakukan oleh pemanggil dengan random_generator (pemilihan acak di antara neighbor terbaik yang seri) sehingga seluruh run memakai satu generator acak.

Fungsi climb_to_peak ditampilkan berikut.

```
def climb_to_peak(start_state, problem, random_generator, max_steps):
    current_state = start_state
    current_value = objective(current_state, problem)
    step_log = []
    neighbors_evaluated = 0

    for step in range(1, max_steps + 1):
        best_neighbor_value, tied_best_neighbors, step_evaluated =
find_best_neighbors(current_state, problem)
        neighbors_evaluated += step_evaluated

        # Kasus 1: tidak ada neighbor atau tidak ada yang lebih baik, sudah
di peak
        if best_neighbor_value is None or best_neighbor_value <=
current_value:
            step_log.append({
                "iteration": step,
                "state": current_state,
                "value": current_value,
                "neighbor_value": best_neighbor_value,
                "accepted": False,
                "move": None,
                "reason": "local_optimum",
            })
            return current_state, step_log, "local_optimum",
neighbors_evaluated

        # Kasus 2: ada neighbor lebih baik, pilih acak di antara yang seri
terbaik
        move, neighbor_state = random_generator.choice(tied_best_neighbors)
        current_state = neighbor_state
        current_value = best_neighbor_value
        step_log.append({
            "iteration": step,
            "state": current_state,
            "value": current_value,
            "neighbor_value": best_neighbor_value,
            "accepted": True,
            "move": move,
```

```
            "reason": "improved",
        })

    return current_state, step_log, "max_iterations", neighbors_evaluated
```

1. Fungsi menerima state awal dan menghitung nilainya. step_log menampung catatan tiap langkah dengan format yang sama seperti iteration_log pada varian lain.
2. Pada tiap langkah, find_best_neighbors memeriksa seluruh neighbor. Inilah ciri steepest ascent.
3. Kasus 1: apabila tidak ada neighbor atau neighbor terbaik tidak lebih baik daripada state saat ini, state berada pada peak. Langkah dicatat dengan alasan local_optimum dan fungsi berhenti. Neighbor bernilai sama tidak dipakai untuk berpindah (Russell & Norvig, 2021, Sec. 4.1: pencarian berhenti ketika tidak ada neighbor yang lebih baik, sehingga plateau datar dianggap peak).
4. Kasus 2: apabila ada neighbor yang lebih baik, satu di antara neighbor terbaik yang seri dipilih acak, state berpindah, dan langkah dicatat dengan alasan improved.
5. Jika jumlah langkah mencapai max_steps, fungsi berhenti dengan alasan max_iterations.
6. Nomor iterasi pada step_log dimulai dari 1 karena entri state awal dicatat oleh pemanggil.

2.3.2.2 Stochastic Hill-Climbing (hc_stochastic.py)

Bagian 1 dan 2. Membaca config dan mendefinisikan variabel awal.

```
  # 1. Membaca dan memeriksa config
    # check_integer_config menolak bool, negatif, dan tipe selain integer
    max_iterations = check_integer_config("max_iterations",
config["max_iterations"], 0)

    # Kalau tidak diisi, snapshot disimpan (default True)
    save_snapshots = config.get("record_snapshots", True)

    # 2. Mendefinisikan variabel awal yang akan dipakai
    # Satu generator acak untuk seluruh run supaya hasil bisa diulang dengan
seed yang sama
    random_generator = random.Random(seed)
```

```
    # Penghitung: berapa kali tiap hasil keputusan terjadi
    count_improved = 0  # neighbor lebih baik dan diterima
    count_equal = 0  # neighbor bernilai sama dan ditolak
    count_worse = 0  # neighbor lebih buruk dan ditolak
    count_no_neighbor = 0  # iterasi yang tidak menemukan neighbor feasible
    last_improvement_iteration = 0  # iterasi terakhir saat current_state
membaik

    # Catatan tiap iterasi, nanti diubah menjadi history pada bagian 4
    iteration_log = []
```

1. check_integer_config memastikan max_iterations berupa integer tidak negatif dan menolak bool.
2. record_snapshots bersifat opsional dan bernilai True apabila tidak diisi.
3. Satu objek random.Random(seed) dipakai untuk seluruh proses, baik pembentukan initial state maupun pemilihan neighbor, sehingga seed yang sama menghasilkan jalannya pencarian yang sama.
4. Empat penghitung (count_improved, count_equal, count_worse, count_no_neighbor) dan last_improvement_iteration disiapkan untuk menyusun metrics. iteration_log menampung catatan setiap iterasi sebelum diubah menjadi history pada bagian 4.

Bagian 3a. Pembentukan initial state.

```
# 3. Logika algoritma (inti pencarian), hanya bagian ini yang dihitung
waktunya
    start_time = time.perf_counter()

    # State awal dipilih acak dan dijamin feasible
    current_state = random_feasible_state(problem, random_generator)
    current_value = objective(current_state, problem)

    # Disimpan untuk dilaporkan di hasil akhir
    initial_state = current_state
    initial_value = current_value

    # Iterasi 0 mencatat state awal, belum ada neighbor yang dinilai
    iteration_log.append({
        "iteration": 0,
        "state": current_state,
        "value": current_value,
```

```
        "neighbor_value": None,
        "accepted": False,
        "move": None,
        "reason": "initial",
    })
```

1. Pencatatan waktu dimulai sebelum initial state dibentuk. Pembentukan initial state dan loop pencarian termasuk dalam durasi, sedangkan konversi history dan perhitungan metrics tidak.
2. random_feasible_state() menghasilkan state awal yang selalu feasible (subbab 1.5). Nilai objektifnya dihitung dengan objective().
3. State dan nilai awal disimpan terpisah pada initial_state dan initial_value karena current_state akan berubah selama pencarian.
4. Iterasi ke-0 dicatat dengan alasan initial sebagai titik awal kurva objektif.

Bagian 3b. Loop pencarian.

```
   for iteration in range(1, max_iterations + 1):
        # Ambil SATU neighbor acak, hasilnya None kalau tidak ketemu yang
feasible
        neighbor_state = random_feasible_neighbor(current_state, problem,
random_generator)

        # Kasus 1: tidak ada neighbor feasible, state tidak berubah
        if neighbor_state is None:
            count_no_neighbor += 1
            iteration_log.append({
                "iteration": iteration,
                "state": current_state,
                "value": current_value,
                "neighbor_value": None,
                "accepted": False,
                "move": None,
                "reason": "no_feasible_candidate",
            })
            continue  # lanjut ke iterasi berikutnya

        neighbor_value = objective(neighbor_state, problem)
        move = describe_move(current_state, neighbor_state, problem)

        # Kasus 2: neighbor lebih baik, pindah ke neighbor
```

```
        if neighbor_value > current_value:
            current_state = neighbor_state
            current_value = neighbor_value
            count_improved += 1
            last_improvement_iteration = iteration
            accepted = True
            reason = "improved"

        # Kasus 3: nilainya sama, tidak pindah
        elif neighbor_value == current_value:
            count_equal += 1
            accepted = False
            reason = "equal"

        # Kasus 4: neighbor lebih buruk, tidak pindah
        else:
            count_worse += 1
            accepted = False
            reason = "worse"

        # Yang dicatat adalah state SETELAH keputusan diambil
        iteration_log.append({
            "iteration": iteration,
            "state": current_state,
            "value": current_value,
            "neighbor_value": neighbor_value,
            "accepted": accepted,
            "move": move,
            "reason": reason,
        })

    runtime_seconds = time.perf_counter() - start_time
```

Pada setiap iterasi terdapat empat kemungkinan hasil.

1. Tidak ada neighbor feasible. Apabila random_feasible_neighbor() mengembalikan None, iterasi dicatat dengan alasan no_feasible_candidate, state tidak berubah, dan pencarian lanjut ke iterasi berikutnya.
2. Neighbor lebih baik (neighbor_value > current_value). Algoritma berpindah ke neighbor tersebut, menambah count_improved, dan memperbarui last_improvement_iteration. Ini satu-satunya kondisi yang mengubah current_state.

3. Neighbor bernilai sama. Neighbor ditolak. Varian ini tidak menerima sideways move sehingga tidak dapat melintasi plateau. Situasi ini sering terjadi karena nilai objektif hanya berubah ketika himpunan kendaraan yang dimuat berubah.
4. Neighbor lebih buruk. Neighbor ditolak dan count_worse bertambah. Tidak ada peluang menerima solusi yang lebih buruk, berbeda dengan Simulated Annealing.
5. describe_move() menurunkan jenis perubahan (swap, relocate, rotate) dari selisih dua state. Catatan dibuat setelah keputusan diambil sehingga state dan value pada catatan selalu menunjukkan kondisi saat ini. Durasi dihitung segera setelah loop selesai.

Bagian 4. Mengubah catatan iterasi menjadi history.

```
 # 4. Mengubah catatan iterasi (iteration_log) menjadi history
    history = []
    for entry in iteration_log:
        # Snapshot ukurannya besar, jadi hanya disimpan di iterasi 0 dan
saat state berubah
        state_changed = entry["iteration"] == 0 or entry["accepted"]
        if save_snapshots and state_changed:
            state_snapshot = snapshot(entry["state"], problem)
        else:
            state_snapshot = None  # None artinya "sama dengan entri
sebelumnya"

        history.append({
            "iteration": entry["iteration"],
            "objective": entry["value"],
            "candidate_objective": entry["neighbor_value"],
            "accepted": entry["accepted"],
            "improved": entry["accepted"],  # Di hill climbing, diterima
berarti membaik
            "move": list(entry["move"]) if entry["move"] else None,
            "reason": entry["reason"],
            "current_state": state_snapshot,
        })
```

1. Konversi dilakukan setelah waktu pencarian dihentikan sehingga pembuatan snapshot tidak ikut dihitung sebagai durasi.
2. Snapshot state berukuran besar, sehingga hanya disimpan pada iterasi ke-0 dan pada iterasi yang menerima neighbor. Nilai None berarti state sama dengan entri sebelumnya.

3. Pada varian ini, neighbor yang diterima selalu merupakan perbaikan sehingga improved bernilai sama dengan accepted.

Bagian 5 dan 6. Meringkas metrics dan mengemas hasil.

```
 # 5. Meringkas metrics hasil iterasi dan algoritma
    # objective() dipanggil 1 kali untuk state awal, lalu 1 kali per
neighbor yang dinilai
    # (iterasi tanpa neighbor tidak memanggil objective)
    objective_evaluations = 1 + max_iterations - count_no_neighbor

    metrics = {
        "improvements": count_improved,
        "rejected_equal": count_equal,
        "rejected_worse": count_worse,
        "no_candidate": count_no_neighbor,
        "iterations": max_iterations,
        "last_improvement_iteration": last_improvement_iteration,
        "objective_evaluations": objective_evaluations,
    }

    # 6. Mengemas semua hasil akhir menjadi RunResult
    return RunResult(
        algorithm="hc",
        seed=seed,
        config={"max_iterations": max_iterations, "record_snapshots":
save_snapshots},
        problem=problem_to_dict(problem),
        initial_state=snapshot(initial_state, problem),
        final_state=snapshot(current_state, problem),
        initial_objective=initial_value,
        final_objective=current_value,
        runtime_seconds=runtime_seconds,
        history=history,
        metrics=metrics,
    )
```

1. objective_evaluations dihitung sebagai satu evaluasi untuk initial state ditambah satu evaluasi untuk setiap neighbor yang berhasil dibangkitkan. Iterasi tanpa neighbor feasible tidak memanggil objective() sehingga dikurangkan.
2. Metrics merangkum jumlah perbaikan, penolakan karena nilai sama, penolakan karena lebih buruk, iterasi tanpa kandidat, dan iterasi terakhir yang menghasilkan

perbaikan. Nilai terakhir berguna untuk menilai apakah pencarian sudah stagnan sebelum iterasi habis.

3. RunResult memuat nama algoritma (hc), seed, konfigurasi, data persoalan, initial state, final state, nilai objektif awal dan akhir, durasi, history, dan metrics, sehingga formatnya sama dengan algoritma lain dan dapat divalidasi dengan perintah validate pada CLI.

2.3.2.3 Hill-Climbing with Sideways Move (hc_sideways.py)

Bagian 1 dan 2 serupa dengan subbab 2.3.2.2. Perbedaannya ada pada dua parameter (max_sideways dan max_iterations, masing-masing dengan nilai default) dan variabel penghitung sideways berikut.

```
 # 1. Membaca dan memeriksa config
    max_sideways = check_integer_config("max_sideways",
config.get("max_sideways", 100), 0)
    max_iterations = check_integer_config("max_iterations",
config.get("max_iterations", 1000), 0)

    # Kalau tidak diisi, snapshot disimpan (default True)
    save_snapshots = config.get("record_snapshots", True)

    # 2. Mendefinisikan variabel awal yang akan dipakai
    # Satu generator acak untuk seluruh run supaya hasil bisa diulang dengan
seed yang sama
    random_generator = random.Random(seed)

    # Penghitung: berapa kali tiap hasil keputusan terjadi
    count_improved = 0  # langkah yang membuat nilai naik
    total_sideways = 0  # seluruh sideways move selama run
    consecutive_sideways = 0  # sideways move beruntun sejak langkah
terakhir yang membaik
    max_consecutive_sideways = 0  # rekor consecutive_sideways tertinggi
selama run
    neighbors_evaluated = 0  # jumlah neighbor yang dinilai objective-nya
    last_improvement_iteration = 0  # iterasi terakhir saat nilai current
naik
    termination_reason = "max_iterations"  # alasan berhenti, diganti kalau
loop di-break

    # Catatan tiap iterasi, nanti diubah menjadi history pada bagian 4
```

```
    iteration_log = []
```

1. consecutive_sideways menghitung sideways move beruntun sejak langkah terakhir yang membaik dan menjadi penentu batas max_sideways.
2. total_sideways dan max_consecutive_sideways hanya dicatat untuk metrics.
3. termination_reason berisi max_iterations secara awal dan diganti apabila loop dihentikan oleh kondisi lain.

Bagian 3. Loop pencarian. Pembentukan initial state sama dengan subbab 2.3.2.2, dengan tambahan key sideways pada catatan.

```
   for iteration in range(1, max_iterations + 1):
        # Periksa SEMUA neighbor, lalu ambil nilai terbaik dan semua
neighbor yang seri
        best_neighbor_value, tied_best_neighbors, evaluated_now =
find_best_neighbors(current_state, problem)
        neighbors_evaluated += evaluated_now

        # Kasus 1: tidak ada neighbor feasible atau semua neighbor lebih
buruk, sudah di peak
        if best_neighbor_value is None or best_neighbor_value <
current_value:
            iteration_log.append({
                "iteration": iteration,
                "state": current_state,
                "value": current_value,
                "neighbor_value": best_neighbor_value,
                "accepted": False,
                "sideways": False,
                "move": None,
                "reason": "local_optimum",
            })
            termination_reason = "local_optimum"
            break

        # Kasus 2: neighbor terbaik lebih baik, pindah dan counter sideways
direset
        if best_neighbor_value > current_value:
            move, neighbor_state =
random_generator.choice(tied_best_neighbors)
            current_state = neighbor_state
            current_value = best_neighbor_value
```

```
            consecutive_sideways = 0
            count_improved += 1
            last_improvement_iteration = iteration
            is_sideways = False
            reason = "improved"

        # Kasus 3: neighbor terbaik bernilai sama (plateau), boleh sideways
selama belum mencapai batas
        else:
            if consecutive_sideways >= max_sideways:
                iteration_log.append({
                    "iteration": iteration,
                    "state": current_state,
                    "value": current_value,
                    "neighbor_value": best_neighbor_value,
                    "accepted": False,
                    "sideways": False,
                    "move": None,
                    "reason": "max_sideways",
                })
                termination_reason = "max_sideways"
                break

            move, neighbor_state =
random_generator.choice(tied_best_neighbors)
            current_state = neighbor_state
            consecutive_sideways += 1
            total_sideways += 1
            if consecutive_sideways > max_consecutive_sideways:
                max_consecutive_sideways = consecutive_sideways
            is_sideways = True
            reason = "sideways"

        # Yang dicatat adalah state SETELAH keputusan diambil
        iteration_log.append({
            "iteration": iteration,
            "state": current_state,
            "value": current_value,
            "neighbor_value": best_neighbor_value,
            "accepted": True,
            "sideways": is_sideways,
            "move": move,
            "reason": reason,
        })

    runtime_seconds = time.perf_counter() - start_time
```

1. find_best_neighbors memeriksa seluruh neighbor feasible pada tiap iterasi.
2. Kasus 1: tidak ada neighbor feasible atau nilai terbaik lebih kecil daripada nilai saat ini, berarti state berada pada local optimum. Iterasi dicatat dengan alasan local_optimum, termination_reason diperbarui, dan loop dihentikan.
3. Kasus 2: nilai terbaik lebih besar. Satu neighbor terbaik dipilih acak, state berpindah, consecutive_sideways diatur ulang menjadi 0, dan count_improved bertambah.
4. Kasus 3: nilai terbaik sama dengan nilai saat ini (plateau). Apabila consecutive_sideways sudah mencapai max_sideways, iterasi dicatat dengan alasan max_sideways dan loop dihentikan tanpa mengubah state. Apabila belum, satu neighbor seri dipilih acak, state berpindah sebagai sideways move, consecutive_sideways dan total_sideways bertambah, dan rekor max_consecutive_sideways diperbarui. current_value tidak diubah karena nilai neighbor sama.
5. Catatan dibuat setelah keputusan diambil. Langkah improved maupun sideways berstatus accepted, dan key sideways membedakan keduanya.
6. Jika loop selesai tanpa break, pencarian berhenti karena max_iterations tercapai, sesuai nilai awal termination_reason.

Bagian 4. History. Pada history, improved bernilai benar hanya untuk langkah yang diterima dan bukan sideways, karena sideways diterima tetapi tidak membaik. Key tambahan sideways ikut disimpan.

```
    # 4. Mengubah catatan iterasi (iteration_log) menjadi history
    history = []
    for entry in iteration_log:
        # Snapshot ukurannya besar, jadi hanya disimpan di iterasi 0 dan
saat state berubah
        state_changed = entry["iteration"] == 0 or entry["accepted"]
        if save_snapshots and state_changed:
            state_snapshot = snapshot(entry["state"], problem)
        else:
            state_snapshot = None  # None artinya "sama dengan entri
sebelumnya"

        history.append({
            "iteration": entry["iteration"],
```

```
            "objective": entry["value"],
            "candidate_objective": entry["neighbor_value"],
            "accepted": entry["accepted"],
            "improved": entry["accepted"] and not entry["sideways"],  #
sideways diterima tetapi tidak membaik
            "move": list(entry["move"]) if entry["move"] else None,
            "reason": entry["reason"],
            "current_state": state_snapshot,
            "sideways": entry["sideways"],
        })
```

Bagian 5. Metrics.

```
   # 5. Meringkas metrics hasil iterasi dan algoritma
    # objective() dipanggil 1 kali untuk state awal, lalu 1
kali per neighbor yang dinilai
    objective_evaluations = 1 + neighbors_evaluated

    metrics = {
        "iterations": iteration_log[-1]["iteration"],  #
iterasi terakhir yang dijalankan
        "improvements": count_improved,
        "sideways_moves_total": total_sideways,
        "max_consecutive_sideways": max_consecutive_sideways,
        "termination_reason": termination_reason,
        "neighbors_evaluated": neighbors_evaluated,
        "objective_evaluations": objective_evaluations,
        "last_improvement_iteration":
last_improvement_iteration,
    }
```

1. iterations diambil dari nomor iterasi terakhir pada iteration_log. Jika pencarian berhenti karena local optimum atau max_sideways, iterasi terakhir adalah iterasi tempat kondisi berhenti terdeteksi dan tidak mengubah state.
2. objective_evaluations adalah satu evaluasi untuk initial state ditambah seluruh neighbor yang dinilai (neighbors_evaluated).

3. Metrics khusus varian ini adalah sideways_moves_total, max_consecutive_sideways, dan termination_reason. Bagian 6 sama dengan subbab 2.4.2.2 dengan algorithm="hc_sideways".

2.3.2.4 Random Restart Hill-Climbing (hc_random_restart.py)

Bagian 1 dan 2. Membaca config dan mendefinisikan variabel awal.

```
  # 1. Membaca dan memeriksa config
    max_restarts = check_integer_config("max_restarts",
config.get("max_restarts", 10), 1)
    iterations_per_restart_limit = check_integer_config(
        "iterations_per_restart", config.get("iterations_per_restart",
1000), 0)

    # Kalau tidak diisi, snapshot disimpan (default True)
    save_snapshots = config.get("record_snapshots", True)

    # 2. Mendefinisikan variabel awal yang akan dipakai
    # Satu generator acak untuk seluruh run supaya hasil bisa diulang dengan
seed yang sama
    random_generator = random.Random(seed)

    iterations_per_restart = []  # jumlah langkah yang benar-benar
dijalankan tiap restart
    restart_initial_objectives = []  # nilai state awal tiap restart
    restart_final_objectives = []  # nilai peak yang dicapai tiap restart
    restart_termination_reasons = []  # alasan berhenti tiap restart
    neighbors_evaluated = 0  # jumlah neighbor yang dinilai objective-nya,
dari semua restart

    best_state = None  # state terbaik di antara semua restart
    best_value = None
    best_restart_index = 0
    initial_state = None  # state awal restart pertama, dilaporkan sebagai
initial_state

    # Catatan tiap iterasi, nanti diubah menjadi history pada bagian 4
    iteration_log = []
    global_iteration = 0  # nomor iterasi yang berlanjut terus lintas
restart
```

1. Dua parameter diperiksa dengan check_integer_config: max_restarts (minimum 1) dan iterations_per_restart (minimum 0).

2. Daftar iterations_per_restart, restart_initial_objectives, restart_final_objectives, dan restart_termination_reasons menyimpan ringkasan setiap restart untuk metrics.
3. best_state dan best_value menyimpan hasil terbaik di antara semua restart, dan initial_state menyimpan state awal restart pertama untuk dilaporkan.
4. global_iteration adalah nomor iterasi yang berlanjut terus lintas restart sehingga seluruh pencarian dapat digambar dalam satu kurva.

Bagian 3. Loop restart.

```
  # 3. Logika algoritma (inti pencarian), hanya bagian ini yang dihitung
waktunya
    start_time = time.perf_counter()

    for restart_index in range(max_restarts):
        # State awal baru untuk restart ini, dijamin feasible
        restart_state = random_feasible_state(problem, random_generator)
        restart_value = objective(restart_state, problem)
        if restart_index == 0:
            initial_state = restart_state

        # Nilai terbaik yang diketahui SETELAH state awal restart ini ikut
dipertimbangkan
        # (best_so_far dicatat di tiap entri supaya kurvanya bisa digambar)
        if best_value is None or restart_value > best_value:
            best_so_far = restart_value
        else:
            best_so_far = best_value

        iteration_log.append({
            "iteration": global_iteration,
            "restart": restart_index,
            "state": restart_state,
            "value": restart_value,
            "neighbor_value": None,
            "accepted": False,
            "move": None,
            "reason": "restart_initial",
            "best_so_far": best_so_far,
        })

        # Naik sampai peak atau sampai batas langkah per restart
        peak_state, step_log, termination_reason, evaluated_now =
climb_to_peak(
```

```
            restart_state, problem, random_generator,
iterations_per_restart_limit)
        neighbors_evaluated += evaluated_now
        peak_value = objective(peak_state, problem)

        # Pindahkan catatan langkah ke iteration_log dengan nomor iterasi
global
        for step_entry in step_log:
            global_iteration += 1
            if step_entry["value"] > best_so_far:
                best_so_far = step_entry["value"]
            iteration_log.append({
                "iteration": global_iteration,
                "restart": restart_index,
                "state": step_entry["state"],
                "value": step_entry["value"],
                "neighbor_value": step_entry["neighbor_value"],
                "accepted": step_entry["accepted"],
                "move": step_entry["move"],
                "reason": step_entry["reason"],
                "best_so_far": best_so_far,
            })
        # Entri restart_initial berikutnya memakai nomor global yang baru
        global_iteration += 1

        iterations_per_restart.append(len(step_log))
        restart_initial_objectives.append(restart_value)
        restart_final_objectives.append(peak_value)
        restart_termination_reasons.append(termination_reason)

        # Hasil terbaik hanya diganti kalau STRICTLY lebih besar (seri
mempertahankan yang awal)
        if best_value is None or peak_value > best_value:
            best_state = peak_state
            best_value = peak_value
            best_restart_index = restart_index

    runtime_seconds = time.perf_counter() - start_time
```

1. Loop luar berjalan sebanyak max_restarts kali. Seluruh restart memakai satu random_generator yang dilanjutkan sehingga hasil tetap dapat direproduksi.
2. Setiap restart dimulai dari state awal acak yang baru dan feasible. State awal restart pertama disimpan sebagai initial_state.

3. best_so_far adalah nilai terbaik yang diketahui setelah state awal restart ikut dipertimbangkan. Nilai ini dicatat pada tiap entri agar kurva best-so-far dapat digambar. Awal restart dicatat dengan alasan restart_initial.
4. climb_to_peak menjalankan steepest ascent dari state awal restart sampai peak atau sampai batas iterations_per_restart.
5. Catatan langkah dipindahkan ke iteration_log dengan nomor iterasi global, dan best_so_far diperbarui apabila ada nilai yang lebih besar.
6. global_iteration ditambah satu setelah tiap restart sehingga entri restart_initial berikutnya mendapat nomor baru dan tidak ada nomor iterasi yang ganda.
7. Hasil tiap restart disimpan ke daftar ringkasan. iterations_per_restart memakai len(step_log) sehingga termasuk langkah terakhir yang mendeteksi peak.
8. best_state hanya diganti apabila nilai peak lebih besar daripada best_value. Jika seri, hasil restart yang lebih awal dipertahankan.

Bagian 4. History.

```
 # 4. Mengubah catatan iterasi (iteration_log) menjadi history
    history = []
    for entry in iteration_log:
        # Snapshot ukurannya besar, jadi hanya disimpan saat state berubah:
        # awal restart (state baru) dan langkah yang diterima
        state_changed = entry["reason"] == "restart_initial" or
entry["accepted"]
        if save_snapshots and state_changed:
            state_snapshot = snapshot(entry["state"], problem)
        else:
            state_snapshot = None  # None artinya "sama dengan entri
sebelumnya"

        history.append({
            "iteration": entry["iteration"],
            "objective": entry["value"],
            "candidate_objective": entry["neighbor_value"],
            "accepted": entry["accepted"],
            "improved": entry["accepted"],  # Di steepest ascent, diterima
berarti membaik
            "move": list(entry["move"]) if entry["move"] else None,
            "reason": entry["reason"],
            "current_state": state_snapshot,
```

```
            "restart": entry["restart"],
            "best_so_far_objective": entry["best_so_far"],
        })
```

1. Snapshot disimpan pada awal tiap restart (state baru) dan pada langkah yang diterima.
2. Key tambahan restart menunjukkan indeks restart (mulai dari 0) dan best_so_far_objective menunjukkan nilai terbaik sampai entri tersebut.

Bagian 5. Metrics.

```
    # 5. Meringkas metrics hasil iterasi dan algoritma
    # objective() dipanggil 1 kali per state awal restart, lalu 1 kali per
neighbor yang dinilai
    objective_evaluations = max_restarts + neighbors_evaluated

    metrics = {
        "restarts": max_restarts,
        "max_restarts": max_restarts,
        "iterations_per_restart_limit": iterations_per_restart_limit,
        "iterations_per_restart": iterations_per_restart,
        "total_iterations": sum(iterations_per_restart),
        "restart_initial_objectives": restart_initial_objectives,
        "restart_final_objectives": restart_final_objectives,
        "restart_termination_reasons": restart_termination_reasons,
        "best_restart_index": best_restart_index,
        "best_found_objective": best_value,
        "neighbors_evaluated": neighbors_evaluated,
        "objective_evaluations": objective_evaluations,
    }
```

1. objective_evaluations adalah satu evaluasi per state awal restart ditambah seluruh neighbor yang dinilai pada semua restart.
2. Metrics memuat banyak restart, batas dan jumlah iterasi aktual per restart, nilai awal dan akhir tiap restart, alasan berhenti tiap restart, dan indeks restart terbaik.
3. Pada bagian 6, initial_objective diambil dari restart pertama dan final_state serta final_objective diambil dari hasil terbaik di antara semua restart.

2.3.2.5 Stochastic Hill-Climbing (hc_stochastic_aima.py)

Bagian 1. Membaca config.

```
    # 1. Membaca dan memeriksa config
    max_iterations = check_integer_config("max_iterations",
config.get("max_iterations", 1000), 0)

    selection = config.get("selection", "proportional")
    if selection not in SELECTION_MODES:
        raise ValueError("selection harus 'proportional' atau 'uniform'")

    # Kalau tidak diisi, snapshot disimpan (default True)
    save_snapshots = config.get("record_snapshots", True)
```

1. max_iterations diperiksa dengan check_integer_config dan memiliki nilai default 1000.
2. selection hanya boleh bernilai proportional atau uniform. Nilai lain menyebabkan ValueError.

Bagian 2. Loop pencarian. Pembentukan initial state sama dengan subbab 2.4.2.2, dengan tambahan key uphill_count pada catatan.

```
for iteration in range(1, max_iterations + 1):
        # Kumpulkan SEMUA uphill neighbor (nilai lebih besar dari current)
        uphill_moves = []
        uphill_states = []
        uphill_values = []
        for move, neighbor_state in
generate_feasible_neighbors(current_state, problem):
            neighbor_value = objective(neighbor_state, problem)
            neighbors_evaluated += 1
            if neighbor_value > current_value:
                uphill_moves.append(move)
                uphill_states.append(neighbor_state)
                uphill_values.append(neighbor_value)

        # Kasus 1: tidak ada uphill move, sudah di local optimum
        if len(uphill_states) == 0:
            iteration_log.append({
                "iteration": iteration,
                "state": current_state,
                "value": current_value,
                "neighbor_value": None,
                "accepted": False,
                "uphill_count": 0,
                "move": None,
```

```
                "reason": "local_optimum",
            })
            termination_reason = "local_optimum"
            break

        # Kasus 2: ada uphill move, pilih satu sesuai mode selection
        if selection == "proportional":
            # Bobot = kecuraman (selisih nilai), makin curam makin besar
peluangnya
            weights = []
            for uphill_value in uphill_values:
                weights.append(uphill_value - current_value)
            chosen_index =
random_generator.choices(range(len(uphill_states)), weights=weights)[0]
        else:
            chosen_index = random_generator.randrange(len(uphill_states))

        move = uphill_moves[chosen_index]
        current_state = uphill_states[chosen_index]
        current_value = uphill_values[chosen_index]
        count_improved += 1
        last_improvement_iteration = iteration

        # Yang dicatat adalah state SETELAH keputusan diambil
        iteration_log.append({
            "iteration": iteration,
            "state": current_state,
            "value": current_value,
            "neighbor_value": current_value,
            "accepted": True,
            "uphill_count": len(uphill_states),
            "move": move,
            "reason": "improved",
        })

    runtime_seconds = time.perf_counter() - start_time
```

1. Pada tiap iterasi, seluruh neighbor feasible dibangkitkan dan dinilai. Hanya neighbor dengan nilai lebih besar daripada state saat ini yang disimpan pada tiga daftar paralel, yaitu uphill_moves, uphill_states, dan uphill_values.
2. Kasus 1: apabila daftar uphill kosong, state berada pada local optimum. Iterasi dicatat dengan alasan local_optimum dan loop dihentikan.
3. Kasus 2: apabila ada uphill neighbor, satu dipilih sesuai selection. Pada mode proportional, bobot tiap neighbor adalah selisih nilainya terhadap current_value dan

pemilihan memakai random_generator.choices. Pada mode uniform, pemilihan memakai random_generator.randrange.

4. State berpindah ke neighbor terpilih dan catatan menyimpan uphill_count, yaitu jumlah uphill neighbor yang tersedia pada iterasi tersebut.
5. Karena setiap langkah yang diambil adalah uphill, nilai objektif tidak pernah turun.

Bagian 5. Metrics.

```
# 5. Meringkas metrics hasil iterasi dan algoritma
    # objective() dipanggil 1 kali untuk state awal, lalu 1 kali per
neighbor yang dinilai
    objective_evaluations = 1 + neighbors_evaluated

    metrics = {
        "iterations": iteration_log[-1]["iteration"],  # iterasi terakhir
yang dijalankan
        "improvements": count_improved,
        "termination_reason": termination_reason,
        "selection": selection,
        "neighbors_evaluated": neighbors_evaluated,
        "objective_evaluations": objective_evaluations,
        "last_improvement_iteration": last_improvement_iteration,
    }
```

1. Metrics memuat selection yang dipakai, termination_reason, neighbors_evaluated, dan last_improvement_iteration.
2. Bagian 4 sama dengan subbab 2.4.2.2 dengan tambahan key uphill_count, dan bagian 6 sama dengan algorithm="hc_stochastic_aima".

2.3.3 Kondisi Berhenti Pencarian

Kondisi berhenti berbeda untuk tiap varian dan dirangkum pada tabel berikut

Tabel Kondisi Berhenti Tiap Varian

Varian	Kondisi berhenti	Nilai pada hasil
Stochastic	Jumlah iterasi mencapai max_iterations. Tidak ada penghentian lebih awal.	iterations selalu sama dengan max_iterations.
Sideways Move	(a) tidak ada neighbor yang lebih baik atau sama (local optimum); (b) sideways move beruntun mencapai max_sideways; (c) jumlah iterasi mencapai max_iterations.	termination_reason: local_optimum, max_sideways, atau max_iterations.
Random Restart	Tiap restart berhenti pada peak (local optimum) atau saat langkah mencapai iterations_per_restart. Seluruh pencarian berhenti setelah max_restarts restart.	restart_termination_reasons: local_optimum atau max_iterations untuk tiap restart.
Stochastic (versi buku teks)	(a) tidak ada uphill neighbor (local optimum); (b) jumlah iterasi mencapai max_iterations.	termination_reason: local_optimum atau max_iterations.

Untuk Stochastic hill-climbing, satu-satunya kondisi berhenti adalah jumlah iterasi mencapai max_iterations, karena satu neighbor acak yang ditolak tidak dapat membuktikan bahwa state sudah berada pada peak. Loop for iteration in range(1, max_iterations + 1) selalu dijalankan sampai selesai, dan apabila max_iterations bernilai 0, final state sama dengan initial state. Varian ini tidak memeriksa apakah state saat ini sudah merupakan local optimum, karena pemeriksaan itu membutuhkan pembangkitan seluruh neighbor sedangkan varian ini hanya mengevaluasi satu neighbor acak per iterasi.

Iterasi tanpa neighbor feasible (no_feasible_candidate) juga bukan kondisi berhenti, karena kegagalan menemukan neighbor dalam 200 percobaan acak belum membuktikan bahwa neighbor feasible tidak ada. Akibatnya sebagian iterasi dapat terpakai tanpa menghasilkan perbaikan. Pada konfigurasi eksperimen (max_iterations = 2000) dengan seed 3001, 3002, dan 3003, perbaikan terakhir terjadi pada iterasi 434, 317, dan 402. Sisa iterasi hanya menghasilkan neighbor bernilai sama atau lebih buruk.

Untuk sideways move, searching berhenti pada local optimum apabila neighbor terbaik lebih buruk daripada state saat ini atau tidak ada neighbor. Pada plateau, pencarian boleh bergerak ke neighbor bernilai sama sampai sideways move beruntun mencapai max_sideways, lalu berhenti dengan alasan max_sideways. Counter beruntun kembali ke 0 setiap kali ditemukan langkah yang membaik. max_iterations berfungsi sebagai batas pengaman terakhir.

Untuk random restart, di dalam tiap restart, steepest ascent berhenti pada peak, termasuk peak yang datar karena neighbor bernilai sama tidak dipakai untuk berpindah (Russell & Norvig, 2021, Sec. 4.1), atau ketika langkah mencapai iterations_per_restart. Pencarian keseluruhan berhenti setelah max_restarts restart dijalankan dan hasil terbaik di antara semua restart dilaporkan sebagai final state.

While untuk Stochastic versi buku teks, searching berhenti ketika tidak ada uphill neighbor, yaitu ketika state berada pada local optimum menurut definisi hill-climbing pada buku teks, atau ketika max_iterations tercapai. Dalam perhitungan iterasi, pada varian yang berhenti karena kondisi local optimum atau max_sideways, iterasi terakhir yang dicatat adalah iterasi tempat kondisi berhenti terdeteksi, dan iterasi itu tidak mengubah state. Dengan demikian nilai iterations pada metrics sama dengan jumlah langkah yang mengubah state ditambah satu iterasi pendeteksi. Hal yang sama berlaku pada iterations_per_restart di random restart. Seluruh varian merupakan metode pencarian heuristik. Sebagaimana dinyatakan pada subbab 1.7, final state yang dihasilkan merupakan solusi terbaik yang ditemukan dalam batas iterasi dan proses pencarian yang digunakan, tanpa jaminan sebagai optimum global.

2.4 Implementasi Simulated Annealing

Dalam implementasi Simulated Annealing digunakan representasi state, fungsi objektif, dan operator neighbor yang sama dengan algoritma lainnya. Perbedaannya ada pada cara menerima kandidat. Kandidat yang lebih baik selalu diterima, sedangkan kandidat yang lebih buruk masih dapat diterima dengan probabilitas tertentu. Pada penataan vehicle, langkah ini memungkinkan perubahan kombinasi muatan meskipun total shipping fee sempat turun misalnya mengeluarkan satu kendaraan akan mengurangi nilai objektif, tetapi juga menyediakan ruang dan kapasitas berat untuk kendaraan lain. Jika kendaraan penggantinya memiliki shipping fee lebih tinggi, nilai solusi dapat meningkat pada langkah berikutnya.

2.4.1 Konsep, Alur Algoritma dan Parameter yang Digunakan

Simulated Annealing merupakan metode *local search* yang menggunakan temperatur untuk mengatur peluang penerimaan kandidat. Saat temperatur masih tinggi, kandidat yang lebih buruk lebih mudah diterima sehingga *searching* dapat menjangkau susunan kendaraan yang berbeda. Ketika temperatur turun, algoritma semakin selektif terhadap penurunan nilai objektif. Karena tujuan optimasinya adalah memaksimalkan total *shipping fee*, perubahan nilai objektif dihitung dari selisih nilai kandidat dan *current state*:

$$\Delta = f(s') - f(s)$$

dengan:

- s merupakan *current state*.
- s' merupakan kandidat *neighbor*.
- f(s) merupakan total *shipping fee* kendaraan yang dimuat.
- Δ merupakan perubahan nilai objektif apabila kandidat diterima.

Probabilitas penerimaan kandidat dirumuskan sebagai:

$$P(\text{diterima}) = 1, \text{ jika } \Delta \geq 0$$

$$P(\text{diterima}) = \exp(\Delta/T), \text{ jika } \Delta < 0$$

Kandidat yang lebih baik atau memiliki nilai sama selalu diterima. Untuk kandidat yang lebih buruk, program mengambil bilangan acak r pada rentang [0,1). Kandidat diterima apabila:

$$r < P(\text{diterima})$$

Kandidat yang lebih buruk memiliki Δ negatif, sehingga exp(Δ/T) menghasilkan peluang antara 0 dan 1. Semakin besar penurunan nilainya, semakin kecil peluang kandidat diterima pada temperatur yang sama. Sebagai contoh, kandidat yang menurunkan nilai objektif sebesar 10 memiliki peluang diterima sekitar 0,8187 pada temperatur 50. Saat temperatur turun menjadi 5, peluangnya menjadi sekitar 0,1353. Penurunan nilai yang sama karena itu lebih mudah diterima pada awal *searching*.

Temperatur diturunkan menggunakan *geometric cooling*:

$$T_i = T_0 \times \alpha^{i-1}$$

dengan T_0 sebagai temperatur awal, α sebagai *cooling rate*, dan i sebagai nomor iterasi yang dimulai dari 1. Nilai α harus berada pada interval $0 < \alpha < 1$ agar temperatur terus menurun. Geometric cooling ini diambil berdasarkan referensi Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). "Optimization by Simulated Annealing". *Science*, 220(4598), 671-680 yang menyatakan proses penurunan temperatur dalam algoritma simulated annealing umumnya dilakukan secara bertahap menggunakan skema *geometric cooling* dimana temperatur pada setiap iterasi ke-i dihitung berdasarkan perkalian antara suhu awal T0 dengan faktor cooling α yang dipangkatkan. Agar nilai temperatur konsisten menurun sepanjang iterasi, nilai α ditetapkan sebagai konstanta pengurang pada rentang antara 0 dan 1

Tabel Parameter *Simulated Annealing*

Parameter	**Keterangan**
max_iterations	Batas maksimum iterasi *searching*.
initial_temperature	Temperatur yang digunakan pada iterasi pertama.
cooling_rate	Faktor pengali temperatur pada setiap iterasi.

min_temperature	Batas temperatur untuk melanjutkan *searching*.
detect_stuck	Mengaktifkan pemeriksaan *local optimum* setelah *searching* selesai.
record_snapshots	Menyimpan *snapshot* pada kondisi awal dan setiap perpindahan yang diterima.
seed	Mengendalikan pengacakan untuk tiga kali pengujian.

Selama *searching*, current_state menyimpan solusi yang sedang digunakan, sedangkan best_state menyimpan solusi dengan nilai objektif tertinggi yang pernah ditemukan. Keduanya disimpan terpisah karena current_state dapat turun setelah kandidat yang lebih buruk diterima. Alur searchingnya adalah sebagai berikut:

1. Membaca serta memeriksa parameter algoritma.
2. Membentuk satu *initial state* acak yang memenuhi seluruh *hard constraints*.
3. Menetapkan solusi awal sebagai current_state dan best_state.
4. Menghitung temperatur pada iterasi berjalan.
5. Mengambil satu *feasible neighbor* melalui operator swap, relocate, atau rotate.
6. Menghitung nilai objektif kandidat dan selisihnya terhadap current_state.
7. Menerima kandidat berdasarkan aturan probabilitas *Simulated Annealing*.
8. Memperbarui current_state apabila kandidat diterima dan memperbarui best_state apabila ditemukan nilai tertinggi baru.
9. Mencatat hasil iterasi dan melanjutkan *searching* sampai kondisi berhenti terpenuhi.
10. Memeriksa kejadian *stuck*, menyusun *history*, dan mengemas hasil *searching* ke dalam RunResult.

2.4.2 Kode dan Penjelasan Implementasi

Implementasi terdiri atas fungsi utama simulated_annealing() dan beberapa *helper function*. Fungsi utama menjalankan *searching*, sedangkan *helper function* menghitung temperatur dan probabilitas, mencatat iterasi, serta memeriksa kondisi *local optimum*. Penjelasan berikut mengikuti urutan fungsi dan tahapan pengerjaannya.

Fungsi	**Peran**
temperature_at()	Menghitung temperatur berdasarkan nomor iterasi.
acceptance_probability()	Menghitung probabilitas penerimaan kandidat.
is_local_optimum()	Memeriksa keberadaan *feasible neighbor* yang memiliki nilai lebih tinggi.
make_log()	Membentuk catatan iterasi dengan struktur yang konsisten.
check_stuck_history()	Menghitung kejadian masuk dan keluar dari *local optimum*.
simulated_annealing()	Membentuk solusi awal, menjalankan *searching*, dan menghasilkan RunResult.

a) Modul yang digunakan

```
import math
import random
import time

from src.core import objective, problem_to_dict
from src.result import RunResult, snapshot
from src.search_space import (
    describe_move,
    generate_feasible_neighbors,
    random_feasible_neighbor,
    random_feasible_state,
)
```

Modul math digunakan untuk menghitung fungsi eksponensial dan memeriksa apakah parameter numerik bernilai valid. Modul random mengatur pengacakan berdasarkan *seed*, sedangkan time mengukur durasi *searching*. Nilai solusi dihitung melalui objective(). *Initial state* dibuat dengan random_feasible_state(), sedangkan kandidat diambil melalui random_feasible_neighbor(). Fungsi *snapshot*() digunakan untuk menyimpan informasi penempatan beserta nilai objektif, berat, jumlah kendaraan, dan validitasnya.

b) Perhitungan temperatur dan probabilitas penerimaan

```
def temperature_at(initial_temperature, cooling_rate, iteration):
    return initial_temperature * cooling_rate ** (iteration - 1)
def acceptance_probability(delta, temperature):

    if delta >= 0:
        return 1.0
    return math.exp(delta / temperature)
```

Fungsi temperature_at() menghitung temperatur menggunakan *geometric cooling*. Pangkat iteration - 1 membuat iterasi pertama menggunakan initial_temperature, kemudian temperatur berkurang sesuai cooling_rate. Fungsi acceptance_probability() mengembalikan 1 ketika delta ≥ 0. Artinya, kandidat yang lebih baik atau sama selalu diterima. Jika delta < 0, peluangnya dihitung dengan exp(Δ/T). Delta sudah negatif pada kondisi ini, sehingga tidak perlu ditambahkan tanda negatif pada rumus.

c) Pemeriksaan *local optimum*

```
def is_local_optimum(state, problem):
    current_value = objective(state, problem)
    for _, neighbor in generate_feasible_neighbors(state, problem):
        if objective(neighbor, problem) > current_value:
            return False
    return True
```

Fungsi is_local_optimum() memeriksa *feasible neighbor* satu langkah. Jika ada satu *neighbor* dengan nilai lebih tinggi, fungsi langsung mengembalikan False. Jika seluruh *neighbor* selesai di cek tanpa menemukan perbaikan, fungsi mengembalikan True. Hasil True menunjukkan bahwa *state* tidak memiliki perbaikan langsung melalui satu *move*. Solusi masih mungkin diperbaiki melalui beberapa *move*, termasuk langkah

bernilai sama atau lebih rendah, sehingga kondisi ini belum membuktikan *global optimum*.

d) Iteration Log

```
def make_log(iteration, state, value, best_value, candidate_value=None,
             delta=None, temperature=None, probability=None,
random_draw=None,
             accepted=False, move=None, reason=""):
    return {
        "iteration": iteration, "state": state, "value": value,
        "best_value": best_value, "candidate_value": candidate_value,
        "delta": delta, "temperature": temperature,
        "probability": probability, "random_draw": random_draw,
        "accepted": accepted, "move": move, "reason": reason,
    }
```

Fungsi make_log() digunakan agar format log tetap sama pada kondisi awal, iterasi dengan kandidat, dan iterasi tanpa kandidat. Setiap log menyimpan *state* setelah keputusan penerimaan, nilai current dan best, nilai kandidat, temperatur, probabilitas, *random draw*, *move*, dan alasan penerimaan atau penolakan. Pada iterasi ke-0, nilai kandidat dan probabilitas diisi None karena belum ada kandidat yang dinilai. Pada iterasi selanjutnya, value menunjukkan nilai current_state setelah keputusan penerimaan dibuat.

e) Pemeriksaan kejadian *stuck*

```
def check_stuck_history(iteration_log, problem, detect_stuck):
    # pengecekan stuck dilakukan setelah timer searching berhenti
    if not detect_stuck:
        for entry in iteration_log:
            entry["stuck"] = None
            entry["stuck_event"] = False
        return None, None, None, 0, 0, 0.0

    cache = {}
    stuck_events = 0
    escape_count = 0
    check_count = 0
    cache_hits = 0
    check_seconds = 0.0
    previous_stuck = False

    for entry in iteration_log:
```

```
        state_changed = entry["iteration"] == 0 or entry["accepted"]

        if state_changed:
            state = entry["state"]

            if state in cache:
                current_stuck = cache[state]
                cache_hits += 1
            else:
                check_start = time.perf_counter()
                current_stuck = is_local_optimum(state, problem)
                check_seconds += time.perf_counter() - check_start
                cache[state] = current_stuck
                check_count += 1
        else:
            current_stuck = previous_stuck

        stuck_event = current_stuck and not previous_stuck

        if stuck_event:
            stuck_events += 1
        elif previous_stuck and not current_stuck:
            escape_count += 1

        entry["stuck"] = current_stuck
        entry["stuck_event"] = stuck_event
        previous_stuck = current_stuck

    initial_stuck = iteration_log[0]["stuck"]

    return (
        stuck_events, escape_count, initial_stuck,
        check_count, cache_hits, check_seconds,
    )
```

Pemeriksaan *stuck* dilakukan setelah *searching* selesai. Hasilnya digunakan untuk analisis eksperimen dan tidak menentukan penerimaan kandidat maupun kondisi berhenti algoritma. Tahapan pemeriksaannya adalah sebagai berikut:

1. Jika detect_stuck=False, status *stuck* diisi None untuk menunjukkan bahwa pemeriksaan tidak dilakukan.
2. Kondisi awal dan setiap *state* yang diterima diperiksa menggunakan is_local_optimum().

3. Hasil pemeriksaan disimpan dalam *cache* agar *state* yang pernah dikunjungi tidak perlu diperiksa ulang.
4. Jika kandidat ditolak, status *stuck* mengikuti kondisi sebelumnya karena current_state tidak berubah.
5. stuck_events bertambah ketika status berubah dari tidak *stuck* menjadi *stuck*.
6. escape_count bertambah ketika status berubah dari *stuck* menjadi tidak *stuck*.

Jika *initial state* sudah berada pada *local optimum*, kondisi tersebut dihitung sebagai satu kejadian *stuck*. Perpindahan antardua *state* yang sama-sama *stuck* masih dianggap satu episode. Episode baru dihitung setelah algoritma sempat keluar dari kondisi *stuck*, lalu masuk kembali. Jadi, frekuensi *stuck* menunjukkan berapa kali algoritma masuk ke *local optimum*, bukan berapa banyak iterasi yang tidak menghasilkan peningkatan.

f) Pembacaan dan validasi konfigurasi

```
def simulated_annealing(problem, config, seed):
    # 1. read and cek config
    max_iterations = config["max_iterations"]
    initial_temperature = config["initial_temperature"]
    cooling_rate = config["cooling_rate"]
    min_temperature = config["min_temperature"]
    detect_stuck = config.get("detect_stuck", True)
    save_snapshots = config.get("record_snapshots", True)

    valid_iterations = (
        isinstance(max_iterations, int)
        and not isinstance(max_iterations, bool)
    )

    if not valid_iterations or max_iterations < 0:
        raise ValueError("max_iterations harus integer >= 0")

    def valid_number(value):
        return (
            isinstance(value, (int, float))
            and not isinstance(value, bool)
            and math.isfinite(value)
        )
    if not valid_number(initial_temperature) or initial_temperature <= 0:
        raise ValueError("initial_temperature harus angka berhingga > 0")

    if not valid_number(cooling_rate) or not 0 < cooling_rate < 1:
```

```
        raise ValueError("cooling_rate harus berada di antara 0 dan 1")

    if not valid_number(min_temperature) or min_temperature <= 0:
        raise ValueError("min_temperature harus angka berhingga > 0")

    if not isinstance(detect_stuck, bool) or not isinstance(save_snapshots,
bool):
        raise ValueError("detect_stuck dan record_snapshots harus berupa
boolean")
```

Empat parameter utama dibaca dari config. Jika detect_stuck dan record_snapshots tidak diisi, keduanya menggunakan nilai *default* True. Parameter diperiksa sebelum *searching* dimulai. max_iterations harus berupa integer tidak negatif, temperatur harus positif dan berhingga, sedangkan cooling_rate harus berada di antara 0 dan 1. Boolean ditolak sebagai parameter numerik karena True dan False juga dianggap sebagai integer oleh Python.

g) Inisialisasi variabel dan *initial state*

```
    # 2. siapin variable awal
    random_generator = random.Random(seed)

    counter = {
        "accepted_better": 0,
        "accepted_equal": 0,
        "accepted_worse": 0,
        "rejected_worse": 0,
        "no_candidate": 0,
    }

    iteration_log = []
    start_time = time.perf_counter()

    current_state = random_feasible_state(problem, random_generator)
    current_value = objective(current_state, problem)

    initial_state = current_state
    initial_value = current_value

    # current menyimpan state sekarang, best menyimpan hasil terbaik
    best_state = current_state
    best_value = current_value
    best_iteration = 0

    iteration_log.append(make_log(
```

```
        0, current_state, current_value, best_value,
        reason="initial",
    ))

    termination_reason = "max_iterations"
    final_temperature = None
```

Satu *random generator* digunakan untuk membentuk *initial state*, memilih kandidat, dan mengambil *random draw*. Dengan *seed* yang sama pada implementasi dan lingkungan yang sama, urutan pengacakannya dapat diulang. Counter mencatat lima kondisi: kandidat lebih baik yang diterima, kandidat bernilai sama yang diterima, kandidat lebih buruk yang diterima, kandidat lebih buruk yang ditolak, dan iterasi tanpa kandidat.

Timer dimulai sebelum *initial state* dibuat. Durasi yang dicatat mencakup pembentukan solusi awal dan loop *searching*. *Initial state* dan nilainya disimpan pada initial_state dan initial_value untuk membandingkan kondisi awal dengan hasil akhir. Pada tahap ini, solusi yang sama juga menjadi best_state pertama dengan best_iteration = 0. Log iterasi ke-0 menjadi titik awal grafik.

h) Perulangan *searching* dan pemilihan kandidat

```
    # 3. logic simulated annealing
    for iteration in range(1, max_iterations + 1):
        temperature = temperature_at(
            initial_temperature, cooling_rate, iteration
        )
        if temperature < min_temperature:
            termination_reason = "min_temperature"
            break

        final_temperature = temperature

        candidate_state = random_feasible_neighbor(
            current_state, problem, random_generator
        )

        # none berarti percobaan random habis, bukan pasti local optimum
        if candidate_state is None:
            counter["no_candidate"] += 1
```

```
            iteration_log.append(make_log(
                iteration, current_state, current_value, best_value,
                temperature=temperature,
                reason="no_feasible_candidate",
            ))
            continue
        candidate_value = objective(candidate_state, problem)
        delta = candidate_value - current_value
        probability = acceptance_probability(delta, temperature)
        random_draw = None

        if delta > 0:
            accepted = True
            reason = "better"
            counter["accepted_better"] += 1

        elif delta == 0:
            accepted = True
            reason = "equal"
            counter["accepted_equal"] += 1

        else:
            random_draw = random_generator.random()
            accepted = random_draw < probability
            reason = "worse_accepted" if accepted else "worse_rejected"

            if accepted:
                counter["accepted_worse"] += 1
            else:
                counter["rejected_worse"] += 1

        move = describe_move(current_state, candidate_state, problem)

        # current pindah kalau candidate diterima
        if accepted:
            current_state = candidate_state
            current_value = candidate_value

            # best cuma berubah kalau nilainya lebih tinggi
            if current_value > best_value:
                best_state = current_state
                best_value = current_value
                best_iteration = iteration

        iteration_log.append(make_log(
            iteration, current_state, current_value, best_value,
            candidate_value, delta, temperature, probability,
```

```
            random_draw, accepted, move, reason,
        ))

    runtime_seconds = time.perf_counter() - start_time
```

Setiap iterasi dimulai dengan menghitung temperatur. Jika temperatur belum berada di bawah batas minimum, program mengambil satu kandidat menggunakan random_feasible_neighbor(). Kandidat yang dihasilkan sudah memenuhi batas geladak, larangan *overlap*, dan kapasitas berat kapal. Jika hasilnya None, *current state* dipertahankan dan program melanjutkan ke iterasi berikutnya. None hanya menunjukkan bahwa percobaan acak tidak berhasil memperoleh kandidat, bukan bahwa seluruh *neighborhood* tidak memiliki perbaikan.

Terdapat tiga kondisi penerimaan:

1. delta > 0. Kandidat memiliki total *shipping fee* lebih tinggi dan langsung diterima.
2. delta == 0. Kandidat memiliki nilai objektif yang sama dan tetap diterima. Perpindahan ini memungkinkan perubahan susunan kendaraan tanpa menurunkan nilai objektif.
3. delta < 0. Kandidat diterima apabila bilangan acak lebih kecil daripada probabilitas penerimaannya.

Random draw hanya diambil ketika kandidat lebih buruk. Kandidat yang lebih baik atau sama langsung diterima tanpa pengundian. Ketika kandidat diterima, current_state dan current_value diperbarui. *Best state* hanya berubah jika nilainya lebih tinggi daripada best_value. Dengan aturan ini, best_iteration mencatat iterasi saat nilai terbaik yang tersimpan pertama kali ditemukan. Fungsi describe_move() menentukan jenis perubahan dari selisih *current state* dan kandidat. Log kemudian dicatat setelah keputusan dibuat, sehingga *current objective* selalu sesuai dengan *state* yang digunakan pada akhir iterasi.

i) Pemeriksaan stuck dan pembentukan history

```
    # 4. cek stuck di luar runtime searching
    stuck_result = check_stuck_history(
        iteration_log, problem, detect_stuck )
```

```
(
    stuck_events,
    escape_count,
    initial_stuck,
    check_count,
    cache_hits,
    check_seconds,
) = stuck_result

# 5. convert ke history
history = []
for entry in iteration_log:
    state_changed = (
        entry["iteration"] == 0
        or entry["accepted"]
    )
    if save_snapshots and state_changed:
        state_snapshot = snapshot(entry["state"], problem)
    else:
        state_snapshot = None

    history.append({
        "iteration": entry["iteration"],
        "current_objective": entry["value"],
        "best_so_far_objective": entry["best_value"],
        "candidate_objective": entry["candidate_value"],
        "delta_e": entry["delta"],
        "temperature": entry["temperature"],
        "acceptance_probability": entry["probability"],
        "random_draw": entry["random_draw"],
        "accepted": entry["accepted"],
        "move": list(entry["move"]) if entry["move"] else None,
        "reason": entry["reason"],
        "stuck": entry["stuck"],
        "stuck_event": entry["stuck_event"],
        "current_state": state_snapshot,
    })
```

Setelah timer *searching* dihentikan, status *stuck* diperiksa dari iteration_log. Waktu pemeriksaannya dicatat terpisah agar durasi *searching* tidak didominasi pemeriksaan seluruh *neighborhood*. *Iteration log* kemudian diubah menjadi *history* untuk disimpan dan divisualisasikan. Nilai objektif diambil dari log yang sudah ada, sehingga tidak perlu dihitung ulang.

Jika record_snapshots aktif, *snapshot* disimpan pada iterasi ke-0 dan setiap iterasi yang menerima kandidat. Pada iterasi tanpa perpindahan, current_state di dalam *history* diisi None karena *state*-nya sama dengan sebelumnya. Jika opsi ini dimatikan, *snapshot history* tidak disimpan, tetapi *initial state*, *final state*, dan *best-found state* tetap tersedia pada hasil utama.

j) Penyusunan *metrics* dan RunResult

```
    # 6. summary
    iterations_run = len(iteration_log) - 1
    metrics = {
        **counter,
        "iterations_run": iterations_run,
        "objective_evaluations": (
            1 + iterations_run - counter["no_candidate"]
        ),
        "termination_reason": termination_reason,
        "final_temperature": final_temperature,
        "best_found_objective": best_value,
        "best_found_iteration": best_iteration,
        "best_found_state": snapshot(best_state, problem),
        "stuck_event_count": stuck_events,
        "escape_count": escape_count,
        "initial_stuck": initial_stuck,
        "local_optimum_checks": check_count,
        "local_optimum_cache_hits": cache_hits,
        "local_optimum_check_seconds": check_seconds,
    }
    used_config = {
        "max_iterations": max_iterations,
        "initial_temperature": initial_temperature,
        "cooling_rate": cooling_rate,
        "min_temperature": min_temperature,
        "detect_stuck": detect_stuck,
        "record_snapshots": save_snapshots,
    }

    # 7. cover semua hasil jadi RunResult
    return RunResult(
        algorithm="sa",
        seed=seed,
        config=used_config,
        problem=problem_to_dict(problem),
        initial_state=snapshot(initial_state, problem),
        final_state=snapshot(current_state, problem),
```

```
        initial_objective=initial_value,
        final_objective=current_value,
        runtime_seconds=runtime_seconds,
        history=history,
        metrics=metrics,
    )
```

iterations_run dihitung dari jumlah log setelah dikurangi entri *initial state*. objective_evaluations menghitung satu evaluasi *initial state* dan satu evaluasi untuk setiap kandidat yang diperoleh. Evaluasi tambahan untuk pemeriksaan *local optimum* dan *snapshot* tidak termasuk dalam angka ini. Hasil *run* disimpan dalam RunResult. *Final state* berisi *current state* terakhir, sedangkan *best-found state* disimpan pada *metrics* bersama nilai objektif dan iterasi penemuannya. Dengan begitu, kondisi akhir tetap dapat dibandingkan dengan solusi terbaik selama *searching*.

2.4.3 Kondisi Berhenti Pencarian

Simulated Annealing menggunakan dua kondisi berhenti, yaitu batas iterasi dan batas temperatur.

Tabel Kondisi Berhenti *Simulated Annealing*

Kondisi	**Pemeriksaan pada kode**	**Keterangan**
Batas iterasi tercapai	Perulangan selesai sampai max_iterations	*Searching* berhenti setelah seluruh iterasi yang diizinkan digunakan.
Temperatur di bawah batas minimum	temperature < min_temperature	*Searching* dihentikan sebelum kandidat pada iterasi tersebut dihasilkan.

Pemeriksaan temperatur menggunakan operator <. Jika temperatur masih sama dengan min_temperature, iterasi tetap dijalankan. Iterasi tanpa kandidat tidak menghentikan *searching*. Program mencatat no_feasible_candidate dan melanjutkan ke iterasi berikutnya. Kondisi *stuck* juga tidak menjadi alasan berhenti karena algoritma masih dapat menerima *move* bernilai sama atau lebih buruk.

Pada konfigurasi eksperimen, temperatur mengikuti:

$$T_i = 50 \times (0{,}998)^{i-1}$$

Pada iterasi ke-3451, temperatur masih sebesar 0,0500423 sehingga *searching* tetap berjalan. Saat memasuki iterasi ke-3452, temperatur turun menjadi 0,0499422 dan sudah berada di bawah batas 0,05.

Karena itu, hanya 3451 iterasi yang dijalankan meskipun max_iterations ditetapkan sebesar 5000. final_temperature mencatat temperatur terakhir yang dipakai dalam *searching*, yaitu pada iterasi ke-3451. Nilainya masih sedikit di atas min_temperature karena temperatur pada pemeriksaan yang menyebabkan break tidak disimpan sebagai temperatur akhir.

2.5 Implementasi Genetic Algorithm

2.5.1 Representasi Kromosom dan Inisialisasi Populasi

Satu kromosom pada implementasi kami adalah satu *state* lengkap persoalan, yaitu tuple berisi 30 objek Placement dengan urutan yang sama seperti problem.vehicles (V01 sampai V30). Gen ke-i menyimpan keputusan untuk kendaraan ke-i, sehingga panjang kromosom selalu 30 dan tidak berubah selama pencarian. Karena *state*, fungsi objektif, dan operator perubahan yang dipakai sama dengan *Hill-Climbing* dan *Simulated Annealing*, perbedaan hasil antaralgoritma dapat dikaitkan dengan strategi pencariannya. Setiap gen terdiri atas empat bidang, seperti ditunjukkan pada tabel berikut.

Tabel Bidang Gen pada Kromosom

Bidang gen	Nilai	Keterangan
loaded	True atau False	Menyatakan kendaraan dimuat di kapal atau tetap di luar kapal.
x	0 sampai 9 atau None	Kolom pojok kiri atas kendaraan pada geladak berukuran 10 x 20.

y	0 sampai 19 atau None	Baris pojok kiri atas kendaraan.
orientation	H atau V	Kendaraan tidak diputar (H) atau diputar 90 derajat (V).

Sebagai ilustrasi, gen Placement(True, 4, 11, "V") pada posisi ke-7 berarti kendaraan V07 dimuat dengan pojok kiri atas di kolom 4 baris 11 dan diputar vertikal. Kendaraan yang tidak dimuat ditulis Placement(False) tanpa koordinat. Setiap gen adalah keputusan utuh untuk satu kendaraan, sehingga tidak ada kendaraan yang dapat muncul dua kali atau hilang dari kromosom. Hal ini berbeda dengan representasi berbasis permutasi yang memerlukan perbaikan setelah *crossover*.

Populasi awal dibentuk dengan memanggil random_feasible_state() sebanyak population_size kali, yaitu fungsi yang sama dengan pembentuk initial *state* pada *Hill-Climbing* dan *Simulated Annealing*. Urutan kendaraan diacak, lalu tiap kendaraan dicoba pada posisi dan orientasi acak paling banyak 20 kali. Kendaraan yang tidak mendapat posisi valid dibiarkan di luar kapal. Dengan cara ini, seluruh individu awal sudah memenuhi hard constraints dan setiap individu berbeda dari yang lain. Pada 18 run eksperimen, rata-rata *fitness* populasi awal berada di rentang 591 sampai 667, sedangkan individu terbaik pada populasi awal bernilai 680 sampai 885. Nilai terbaik populasi awal inilah yang dicatat sebagai nilai objektif awal pada hasil eksperimen GA.

2.5.2 Alur Algoritma (Fitness Function, Seleksi, Crossover, Mutation, Elitism) dan Parameter yang Digunakan

Genetic Algorithm adalah metode pencarian berbasis populasi. Berbeda dengan *Hill-Climbing* dan *Simulated Annealing* yang hanya menjaga satu *state*, GA menjaga banyak *state* sekaligus dan membentuk generasi baru dari kombinasi individu terbaik pada generasi sebelumnya. Satu iterasi pada GA kami sebut satu generasi. Generasi 0 adalah populasi awal, lalu pencarian berlanjut dari generasi 1 sampai generasi ke-G.

a) *Fitness function*

Persoalan ini memaksimalkan total shipping fee, sehingga *fitness* suatu individu sama dengan nilai *objective function* yang sudah dipakai pada algoritma lain:

$$f(s) = \Sigma_{i=1}^{30} \text{ShippingFee}_i \cdot [\text{loaded}_i = \text{True}]$$

Nilai *fitness* yang lebih besar berarti individu lebih baik, dan tidak diperlukan transformasi seperti 1/(1+f) karena *roulette wheel* hanya membutuhkan *fitness* yang tidak negatif. Seluruh individu selalu *feasible*, sehingga tidak ada penalti untuk pelanggaran constraint. *Fitness* dihitung ulang dari *state* setiap generasi, jadi tidak ada nilai lama yang tersimpan.

b) Alur per generasi

Setiap generasi mengikuti urutan berikut sampai jumlah generasi yang ditetapkan tercapai.:

1. Hitung *fitness* seluruh individu pada populasi.
2. Urutkan populasi menurut *fitness*, lalu salin elite_count individu terbaik ke populasi baru tanpa perubahan (*elitism*).
3. Selama populasi baru belum penuh, pilih dua *parent* dengan *roulette wheel selection*. Pemilihan dilakukan dengan pengembalian, sehingga satu individu boleh terpilih dua kali.
4. Dengan peluang crossover_rate, lakukan *one-point crossover* dan ambil satu *child*. Jika tidak terjadi *crossover*, *child* adalah salinan *parent* pertama.
5. Dengan peluang mutation_rate, mutasi *child* dengan satu *move* acak (swap, relocate, atau rotate).
6. Jalankan *repair* agar *child* kembali *feasible*, lalu masukkan ke populasi baru.
7. Gantikan populasi lama dengan populasi baru, catat *fitness* maksimum, rata-rata, dan minimum, lalu ulangi dari langkah 1.

c) Seleksi *parent*

Roulette wheel selection memilih individu dengan peluang sebanding dengan *fitness*-nya:

$$P(i) = f(s_i) / \Sigma_{j=1}^{N} f(s_j)$$

dengan N sebagai ukuran populasi. Pada kode, *fitness* dijumlahkan secara kumulatif, lalu sebuah titik acak di antara 0 dan total *fitness* dicari letaknya dengan pencarian biner. Individu dengan *fitness* 0 tidak pernah terpilih karena intervalnya kosong. Jika total *fitness* 0, pemilihan jatuh ke distribusi uniform.

d) *Crossover*

Crossover yang dipakai adalah *one-point crossover*. Sebuah titik potong k dipilih acak dari 1 sampai 29, lalu *child* terdiri atas gen ke-1 sampai ke-k milik *parent* pertama dan gen ke-k+1 sampai ke-30 milik *parent* kedua. Satu pasangan *parent* menghasilkan satu *child*. Karena penempatan tiap kendaraan berasal dari dua susunan yang berbeda, *child* dapat mengalami tumpang tindih antarkendaraan atau melebihi kapasitas berat kapal.

e) *Mutation*

Mutasi dilakukan dengan peluang mutation_rate untuk setiap *child*, bukan untuk setiap gen. Operatornya memakai fungsi random_move() yang sama dengan pembangkit neighbor pada algoritma lain, yaitu swap (menukar penempatan dua kendaraan), relocate (memindahkan satu kendaraan ke posisi lain atau mengeluarkannya dari kapal), dan rotate (memutar kendaraan di posisi yang sama). Mutasi tidak mengecek feasibility karena perbaikan dilakukan oleh langkah *repair*.

f) *Repair*

Setelah *crossover* dan mutasi, *child* diperiksa. Jika sudah *feasible*, *child* dipakai apa adanya. Jika tidak, kendaraan yang dimuat diproses dari rasio shipping fee terhadap berat tertinggi. Tiap kendaraan dicoba pada posisi semula, lalu pada orientasi lain di posisi yang sama, lalu pada paling banyak 20 posisi acak. Kendaraan yang tetap tidak dapat ditempatkan, atau yang membuat berat melebihi kapasitas, dikeluarkan dari kapal.

Dengan urutan ini, kendaraan yang paling sedikit memberi fee per satuan berat dikorbankan lebih dulu. Jika *repair* gagal menghasilkan *state feasible*, *child* diganti *state* acak yang *feasible*. Kasus ini tidak terjadi pada seluruh eksperimen kami.

g) *Elitism* dan pembentukan generasi berikutnya

Populasi baru berukuran sama dengan populasi lama. Sebanyak elite_count individu terbaik disalin langsung tanpa *crossover*, mutasi, maupun *repair*, dan sisanya diisi *child* hasil proses di atas. Dengan satu elite, *fitness* maksimum per generasi tidak pernah turun. Penggantian ini bersifat generasional, yaitu seluruh individu non-elite digantikan *child*. Individu lama hanya dapat bertahan sebagai elite, atau sebagai salinan *child* ketika terpilih sebagai *parent* dan tidak mengalami *crossover*.

h) Parameter yang digunakan

Tabel Parameter *Genetic Algorithm*

Parameter	Nilai eksperimen	Keterangan
population_size	10, 50, 200	Jumlah individu per generasi. Divariasikan pada skenario 2.
generations	25, 100, 400	Banyak iterasi (generasi) setelah generasi 0. Divariasikan pada skenario 1.
crossover_rate	0,9	Peluang sepasang *parent* menjalani *crossover*.
mutation_rate	0,2	Peluang mutasi per *child*.
elite_count	1	Jumlah individu terbaik yang disalin tanpa perubahan.
repair_tries	20	Percobaan posisi acak per kendaraan saat *repair*.

record_snapshots	True	Menyimpan individu terbaik setiap generasi untuk visualisasi.
seed	5001 sampai 5009 dan 5101 sampai 5109	Mengendalikan pengacakan untuk 18 run.

2.5.3 Kode dan Penjelasan Implementasi

Genetic Algorithm diimplementasikan pada berkas src/algorithms/genetic_algorithm.py. Berkas ini memakai model persoalan dari core.py serta pembangkit *state* dan *move* dari search_space.py, sehingga ruang pencarian sama dengan algoritma lain. Bagian-bagian utamanya dijelaskan berikut.

a) Konfigurasi dan validasi parameter

```
@dataclass(frozen=True)
class GAConfig:
    population_size: int
    generations: int  # banyak iterasi; generasi 0 = populasi awal, lalu
1..generations
    crossover_rate: float = 0.9
    mutation_rate: float = 0.2  # peluang per child (bukan per gene)
    elite_count: int = 1
    repair_tries: int = 20  # percobaan anchor random per kendaraan saat
repair
    record_snapshots: bool = True  # simpan best individual tiap generasi
di history

    def __post_init__(self):
        if not (isinstance(self.population_size, int) and
self.population_size >= 1):
            raise ValueError("population_size harus integer >= 1")
        if not (isinstance(self.generations, int) and self.generations >=
0):
            raise ValueError("generations harus integer >= 0")
        for name in ("crossover_rate", "mutation_rate"):
            if not 0.0 <= getattr(self, name) <= 1.0:
                raise ValueError(f"{name} harus di [0, 1]")
        if not 0 <= self.elite_count <= self.population_size:
            raise ValueError("elite_count harus di [0, population_size]")
        if self.repair_tries < 0:
```

```
            raise ValueError("repair_tries harus >= 0")
```

Kelas GAConfig menampung seluruh parameter GA dan bersifat frozen, sehingga nilainya tidak berubah selama run. Validasi pada __post_init__ menolak ukuran populasi di bawah 1, jumlah generasi negatif, peluang di luar rentang 0 sampai 1, serta elite_count yang lebih besar dari ukuran populasi. Parameter yang divariasikan pada eksperimen adalah population_size dan generations.

b) Seleksi *roulette wheel*

```
def roulette_index(cumulative, rng) -> int:
    total = cumulative[-1]
    if total <= 0:
        return rng.randrange(len(cumulative))
    return bisect_right(cumulative, rng.random() * total)
```

Fungsi menerima daftar *fitness* kumulatif. Titik acak di antara 0 dan total *fitness* dicari letaknya dengan bisect_right, sehingga individu dengan *fitness* lebih besar menempati interval lebih lebar dan lebih mudah terpilih. Individu berfitness 0 memiliki interval kosong dan tidak pernah terpilih. Jika total *fitness* 0, fungsi memilih secara uniform agar tidak terjadi pembagian dengan nol.

c) *One-point crossover*

```
def one_point_crossover(parent1, parent2, rng):
    n = len(parent1)
    if n < 2:
        return tuple(parent1)
    cut = rng.randint(1, n - 1)
    return tuple(parent1[:cut]) + tuple(parent2[cut:])
```

Titik potong dipilih di bagian dalam kromosom sehingga *child* selalu mewarisi sedikitnya satu gen dari tiap *parent*. *Child* berupa tuple baru, dan karena *state* immutable, *parent* tidak ikut berubah.

d) Mutasi

```
def mutate(state, problem, rng):
    move = random_move(state, problem, rng)
    return move[1] if move else state
```

Mutasi memanggil random_move() dari search_space.py. Fungsi ini memilih swap, relocate, atau rotate secara acak dan menghasilkan *state* baru yang bukan no-op, tanpa mengecek feasibility. Jika semua percobaan menghasilkan no-op, *state* dikembalikan tanpa perubahan.

e) *Repair*

Repair diperlukan karena *crossover* dan mutasi dapat menghasilkan kendaraan yang saling tumpang tindih atau total berat yang melebihi kapasitas. Fungsi mengembalikan pasangan *state* dan status, yaitu ok bila *child* sudah *feasible*, repaired bila berhasil diperbaiki, dan fallback bila terpaksa diganti *state* acak. Kendaraan yang dimuat disusun ulang dari rasio shipping fee per berat tertinggi, lalu dibangun kembali satu per satu pada kapal kosong. Setiap kendaraan dicoba pada posisi semula, orientasi lain di posisi yang sama, dan sampai tries posisi acak. Kendaraan yang tidak mendapat tempat atau melampaui kapasitas dikeluarkan. Hasil rekonstruksi diperiksa sekali lagi dengan is_feasible, sehingga *child* yang tidak valid tidak pernah masuk populasi.

```
def repair(state, problem, rng, tries: int = 20):
    if is_feasible(state, problem):
        return state, "ok"
    ship, vehicles = problem.ship, problem.vehicles
    fixed = [UNLOADED] * len(state)
    placed = []
    weight = 0

    def ratio(i):
        v = vehicles[i]
        return v.shipping_fee / v.weight if v.weight > 0 else
float("inf")
    for i in sorted((i for i, p in enumerate(state) if p.loaded),
key=lambda i: (-ratio(i), i)):
        v, p = vehicles[i], state[i]
        if not capacity_ok(weight + v.weight, ship):
            continue
        candidates = [p]
        if v.width != v.length:
            candidates.append(Placement(True, p.x, p.y, "V" if
p.orientation == "H" else "H"))
        for _ in range(tries):
            o = "H" if v.width == v.length else rng.choice(("H", "V"))
            w, l = footprint(v, Placement(True, 0, 0, o))
            if w <= ship.width and l <= ship.length:
                candidates.append(Placement(True, rng.randint(0,
ship.width - w), rng.randint(0, ship.length - l), o))
        for cand in candidates:
            r = rect(v, cand)
            if inside_deck(r, ship) and not any(overlaps(r, other) for
other in placed):
                fixed[i] = cand
                placed.append(r)
                weight += v.weight
                break
    fixed = tuple(fixed)
    if is_feasible(fixed, problem):
        return fixed, "repaired"
    return random_feasible_state(problem, rng), "fallback"
```

f) Pembentukan generasi berikutnya

```
def next_generation(population, fitnesses, problem, cfg: GAConfig, rng,
counters: Counter):
    n = len(population)
    ranked = sorted(range(n), key=lambda i: -fitnesses[i])
    new = [population[i] for i in ranked[: cfg.elite_count]]
    cumulative = list(accumulate(fitnesses))
    while len(new) < n:
        x = population[roulette_index(cumulative, rng)]
        y = population[roulette_index(cumulative, rng)]
        if rng.random() < cfg.crossover_rate:
            child = one_point_crossover(x, y, rng)
            counters["crossovers"] += 1
        else:
            child = x
        if rng.random() < cfg.mutation_rate:
            child = mutate(child, problem, rng)
            counters["mutations"] += 1
        child, status = repair(child, problem, rng, cfg.repair_tries)
        counters[status] += 1
        new.append(child)
    return new
```

Fungsi ini merangkai seluruh operator. Populasi diurutkan menurut *fitness*, lalu elite_count individu terbaik masuk lebih dulu ke populasi baru. Sisanya diisi dengan siklus seleksi dua *parent*, *crossover* dengan peluang crossover_rate, mutasi dengan peluang mutation_rate, dan *repair*. Objek counters mencatat berapa kali *crossover*, mutasi, dan masing-masing status *repair* terjadi, dan angka ini dipakai pada analisis hasil.

g) Loop utama

```
def genetic_algorithm(problem, config, seed: int) -> RunResult:
    cfg = config if isinstance(config, GAConfig) else GAConfig(**config)
    rng = random.Random(seed)
    counters = Counter()
    raw_history = []
    best_so_far, best_so_far_fit = None, None

    start = time.perf_counter()
    population = [random_feasible_state(problem, rng) for _ in
range(cfg.population_size)]
    fitnesses = [objective(s, problem) for s in population]
    for generation in range(cfg.generations + 1):
        if generation > 0:
            population = next_generation(population, fitnesses, problem,
cfg, rng, counters)
            fitnesses = [objective(s, problem) for s in population]
        b = max(range(len(population)), key=fitnesses.__getitem__)
        if best_so_far_fit is None or fitnesses[b] > best_so_far_fit:
            best_so_far, best_so_far_fit = population[b], fitnesses[b]
        raw_history.append((generation, fitnesses[b], mean(fitnesses),
min(fitnesses), best_so_far_fit,
                            len(set(population)), population[b]))
    runtime = time.perf_counter() - start
```

Fungsi genetic_algorithm() membuat generator acak dengan *seed* tertentu, membentuk populasi awal, lalu menjalankan generasi 0 sampai generasi ke-G. Pada setiap generasi dicatat *fitness* maksimum, rata-rata, dan minimum, *fitness* terbaik sepanjang pencarian, serta jumlah individu unik sebagai ukuran keragaman populasi. Timer hanya mencakup proses *searching* dari pembentukan populasi awal sampai generasi terakhir. Penyusunan *history*, *snapshot*, dan penyimpanan hasil dilakukan setelah timer berhenti. Output akhirnya berupa RunResult yang berisi *state* awal, *state* akhir (individu terbaik sepanjang pencarian), nilai objektif, durasi, riwayat per generasi, dan metrik jumlah evaluasi *fitness*.

2.5.4 Kondisi Berhenti Pencarian

Genetic Algorithm berhenti pada satu kondisi saja, yaitu ketika jumlah generasi mencapai parameter generations. Tidak ada penghentian dini berdasarkan konvergensi atau ketiadaan perbaikan. Dengan aturan ini, banyak iterasi menjadi parameter yang dapat dikendalikan langsung pada eksperimen, dan perbandingan antarkonfigurasi pada skenario 1 tidak bercampur dengan pengaruh kriteria berhenti yang berbeda.

Tabel Kondisi Berhenti *Genetic Algorithm*

Aspek	Nilai pada hasil
Kondisi berhenti	Generasi ke-G tercapai, dengan G sama dengan generations.
Panjang *history*	generations + 1 entri, yaitu generasi 0 (populasi awal) sampai generasi G.
Jumlah evaluasi *fitness*	population_size x (generations + 1). Contoh: 1.010 untuk populasi 10 dengan 100 generasi, dan 20.200 untuk populasi 200 dengan 100 generasi.
Hasil akhir	Individu dengan *fitness* tertinggi sepanjang pencarian (*best-so-far*), disimpan sebagai final_state dan final_objective.
Cakupan timer	Pembentukan populasi awal sampai generasi terakhir, tanpa penyusunan *history*, *snapshot*, dan penyimpanan hasil.

Hasil akhir diambil dari *best-so-far*, bukan dari individu terbaik pada generasi terakhir. Dengan elite_count sama dengan 1 seperti pada eksperimen kami, keduanya bernilai sama karena *fitness* maksimum tidak pernah turun. Bila elite_count diatur 0, individu terbaik dapat hilang pada generasi berikutnya, dan *best-so-far* tetap menyimpannya. Kondisi ini berbeda dari *Simulated Annealing* yang berhenti ketika temperatur turun di bawah min_temperature sehingga jumlah iterasinya dapat lebih kecil daripada max_iterations.

2.6 Rancangan Eksperimen

2.6.1 Konfigurasi Data Uji dan Metrik Evaluasi

Kami menggunakan dataset sintetis synthetic-2026 yang terdiri atas satu kapal dan 30 kendaraan dengan ID V01 sampai V30. Dataset yang sama digunakan pada setiap algoritma agar hasilnya dapat dibandingkan berdasarkan strategi *searching* yang diterapkan. Dimensi, berat, dan *shipping fee* dinyatakan dalam satuan model karena dataset tidak menetapkan satuan fisik atau mata uang tertentu.

Tabel Konfigurasi Data Uji

Komponen	**Konfigurasi**
Sumber data	Dataset sintetis synthetic-2026
Berkas input	configs/problem.json
Seed pembentukan dataset	2026
Jumlah kendaraan	30 kendaraan, V01 sampai V30
Lebar geladak	10
Panjang geladak	20
Luas geladak	200 satuan luas
Kapasitas berat kapal	500
Total luas seluruh kendaraan	265 satuan luas
Total berat seluruh kendaraan	1134
Total *shipping fee* seluruh kendaraan	1510
Orientasi kendaraan	Horizontal (H) dan vertikal (V)
Hard constraints	Batas geladak, larangan tumpang tindih, dan kapasitas berat

Peran ETA	Atribut informasi yang belum digunakan dalam fungsi objektif atau constraints

Total luas seluruh kendaraan mencapai 265, sedangkan luas geladak hanya 200. Total beratnya juga mencapai 1134, melebihi kapasitas kapal sebesar 500. Karena seluruh kendaraan tidak dapat dimuat sekaligus, algoritma perlu memilih kombinasi kendaraan dan mengatur posisinya untuk memperoleh total *shipping fee* tertinggi yang dapat ditemukan. *Initial state* SA dibentuk secara acak dengan random_feasible_state(). Seluruh kendaraan mula-mula berada di luar kapal, lalu urutan kendaraan diacak. Setiap kendaraan dicoba pada posisi dan orientasi acak paling banyak 20 kali. Jika tidak ada penempatan yang valid, kendaraan tetap berada di luar kapal.

Selama *searching*, random_feasible_neighbor() mencoba menghasilkan satu kandidat melalui operator swap, relocate, atau rotate. Batasnya adalah 200 percobaan acak untuk mendapatkan satu *feasible neighbor*. Evaluasi hasil menggunakan metrik berikut.

Tabel Metrik Evaluasi Eksperimen

Metrik	**Pengertian dan penggunaan**
Nilai objektif awal	Total *shipping fee* pada *initial state*.
Nilai objektif akhir	Total *shipping fee* pada *current state* terakhir.
Best-found objective	Nilai objektif tertinggi yang ditemukan sepanjang *searching*.
Best iteration	Iterasi ketika nilai terbaik yang tersimpan pertama kali ditemukan.
Peningkatan relatif	Persentase kenaikan nilai objektif akhir terhadap nilai awal.
Jumlah iterasi	Banyaknya iterasi *searching* yang benar-benar dijalankan.

Durasi *searching*	Waktu pembentukan solusi awal dan perulangan *searching*, tanpa pemeriksaan *stuck*, pembuatan *snapshot*, penyimpanan file, dan visualisasi.
Jumlah kendaraan dimuat	Banyaknya kendaraan pada solusi akhir.
Total berat dan luas terpakai	Menggambarkan penggunaan kapasitas kapal dan geladak.
Validitas solusi	Menunjukkan apakah penempatan memenuhi seluruh *hard constraints*.
Konsistensi antar-*run*	Dinilai melalui rata-rata, simpangan baku sampel, dan rentang nilai objektif akhir.

Peningkatan relatif dihitung menggunakan:

$$\text{Peningkatan} = ((f_{akhir} - f_{awal}) / f_{awal}) \times 100\%$$

Untuk *Simulated Annealing*, metrik tambahan mencakup temperatur akhir, jumlah kandidat lebih buruk yang diterima, frekuensi masuk ke *local optimum*, jumlah keluar dari *local optimum*, dan waktu pemeriksaan *stuck*. *Final objective* dan *best-found objective* dicatat terpisah karena SA dapat menerima penurunan nilai. Selain membandingkan nilainya, kami juga memeriksa penataannya kare na dua *state* yang berbeda dapat menghasilkan total *shipping fee* yang sama.

2.6.2 Skenario Eksperimen Hill-Climbing

Hill Climbing Stochastic, First Approach (AIMA)

Hill Climbing stochastic First Approach dijalankan tiga kali menggunakan seed 3401, 3402, dan 3403. Dataset (problem.json) dan seluruh parameter pencarian dibuat tetap, yaitu max_iterations sebesar 1000 dan selection bermode proportional. Seed dibedakan untuk melihat pengaruh pengacakan terhadap initial state dan pemilihan uphill neighbor. Pengujian ini difokuskan pada perbedaan

hasil antar-run dengan satu konfigurasi. Mode pemilihan tidak divariasikan (mode uniform tidak diuji), sehingga eksperimen ini belum membandingkan konfigurasi parameter.

Hill Climbing Stochastic, Second Approach

Hill Climbing stochastic Second Approach dijalankan tiga kali menggunakan seed 3001, 3002, dan 3003. Dataset dan seluruh parameter pencarian dibuat tetap, yaitu max_iterations sebesar 2000. Seed dibedakan untuk melihat pengaruh pengacakan terhadap initial state dan pemilihan neighbor pada tiap iterasi. Pengujian ini difokuskan pada perbedaan hasil antar-run dengan satu konfigurasi. Nilai max_iterations tidak divariasikan, sehingga eksperimen ini belum membandingkan konfigurasi parameter.

Hill Climbing Sideways Move

Hill Climbing sideways move dijalankan tiga kali menggunakan seed 3201, 3202, dan 3203. Dataset dan seluruh parameter pencarian dibuat tetap, yaitu max_sideways sebesar 100 dan max_iterations sebesar 1000. Seed dibedakan untuk melihat pengaruh pengacakan terhadap initial state dan pemilihan sideways move. Pengujian ini difokuskan pada perbedaan hasil antar-run dengan satu konfigurasi. Batas sideways dan iterasi tidak divariasikan, sehingga eksperimen ini belum membandingkan konfigurasi parameter.

Hill Climbing Random Restart

Hill Climbing random restart dijalankan tiga kali menggunakan seed 3301, 3302, dan 3303. Dataset dan seluruh parameter pencarian dibuat tetap, yaitu max_restarts sebesar 100 dan iterations_per_restart sebesar 1000. Seed dibedakan untuk melihat pengaruh pengacakan terhadap initial state di tiap restart. Pengujian ini difokuskan pada perbedaan hasil antar-run dengan satu konfigurasi. Jumlah restart dan batas langkah per restart tidak divariasikan, sehingga eksperimen ini belum membandingkan konfigurasi parameter.

2.6.3 Skenario Eksperimen Simulated Annealing

Simulated Annealing dijalankan tiga kali menggunakan *seed* 4001, 4002, dan 4003. Dataset dan seluruh parameter *searching* dibuat tetap. *Seed* dibedakan untuk melihat pengaruh pengacakan terhadap *initial state*, kandidat, dan keputusan penerimaannya. Pengujian ini difokuskan pada perbedaan hasil antar-*run* dengan satu konfigurasi. Temperatur awal dan *cooling rate* tidak divariasikan, sehingga eksperimen ini belum membandingkan konfigurasi parameter.

Tabel Skenario Eksperimen *Simulated Annealing*

Komponen	***Run* 1**	***Run* 2**	***Run* 3**
Seed	4001	4002	4003
max_iterations	5000	5000	5000
initial_temperature	50,0	50,0	50,0
cooling_rate	0,998	0,998	0,998
min_temperature	0,05	0,05	0,05
detect_stuck	True	True	True
record_snapshots	True	True	True

Konfigurasi tiga *run* dituliskan sebagai berikut:

```
{
  "algorithm": "sa",
  "problem": "configs/problem.json",
  "output_dir": "output/simulated_annealing",
  "config": {
    "max_iterations": 5000,
    "initial_temperature": 50.0,
    "cooling_rate": 0.998,
    "min_temperature": 0.05,
    "detect_stuck": true,
    "record_snapshots": true
  },
```

```
  "seeds": [4001, 4002, 4003]
}
```

Ketiga *run* dijalankan melalui perintah berikut:

```
python -m src.main experiment --config configs/sa_experiment.json
```

Fungsi run_repeated() menjalankan *solver* untuk setiap *seed*, lalu menyimpan hasil dan membuat grafik. Bagian yang menjalankan setiap *run* ditunjukkan pada potongan kode berikut:

```
def run_repeated(config: dict) -> list[dict]:
    algorithm, seeds = config["algorithm"], config["seeds"]

    if len(seeds) < 3 or len(set(seeds)) != len(seeds):
        raise ValueError("butuh minimal 3 seed unik agar run independen")

    problem = load_problem(config["problem"])
    out = Path(config["output_dir"])
    solver = ALGORITHMS.get(algorithm) or HC_VARIANTS[algorithm]
    rows = []

    for seed in seeds:
        result = solver(problem, config["config"], seed)

        json_path, plot_path = (
            Path("runs") / f"{algorithm}_seed{seed}.json",
            Path("plots") / f"{algorithm}_seed{seed}.png"
        )

        result.save(out / json_path)
        plot_run(result, out / plot_path)
```

Setiap *run* menghasilkan catatan nilai objektif, berat muatan, jumlah kendaraan, iterasi, durasi, dan frekuensi *stuck*.

2.6.4 Skenario Eksperimen Genetic Algorithm

Genetic Algorithm diuji pada dataset sintetis synthetic-2026 yang sama dengan *Hill-Climbing* dan *Simulated Annealing*, yaitu satu kapal berukuran 10 x 20 dengan kapasitas berat 500 dan 30 kendaraan. Eksperimen terdiri atas dua skenario wajib. Pada tiap skenario terdapat tiga konfigurasi, dan setiap konfigurasi dijalankan tiga kali dengan *seed* berbeda, sehingga seluruhnya ada 18 run. *Seed* dibuat unik pada seluruh run agar tiap run saling independen.

Skenario 1: jumlah populasi tetap, banyak iterasi divariasikan. Jumlah populasi dibuat tetap 50 individu, sedangkan banyak iterasi (generasi) divariasikan menjadi 25, 100, dan 400. Skenario ini dipakai untuk melihat apakah tambahan generasi masih memperbaiki hasil dan sampai kapan perbaikan itu berlangsung.

Skenario 2: banyak iterasi tetap, jumlah populasi divariasikan. Banyak iterasi dibuat tetap 100 generasi, sedangkan jumlah populasi divariasikan menjadi 10, 50, dan 200. Skenario ini dipakai untuk melihat pengaruh keragaman populasi terhadap kualitas solusi dan durasi pencarian.

Tabel Skenario Eksperimen *Genetic Algorithm*

Skenario	Konfigurasi	Populasi	Generasi	*Seed*	Evaluasi *fitness* per run
1	pop50_gen25	50	25	5001, 5002, 5003	1.300
1	pop50_gen100	50	100	5004, 5005, 5006	5.050
1	pop50_gen400	50	400	5007, 5008, 5009	20.050
2	pop10_gen100	10	100	5101, 5102, 5103	1.010

2	pop50_gen100	50	100	5104, 5105, 5106	5.050
2	pop200_gen100	200	100	5107, 5108, 5109	20.200

Konfigurasi populasi 50 dengan 100 generasi muncul pada kedua skenario sebagai titik tengah. Keduanya dijalankan dengan *seed* yang berbeda, sehingga hasilnya juga menjadi gambaran variasi antarseed pada konfigurasi yang sama. Parameter lain dibuat tetap pada seluruh run, yaitu crossover_rate 0,9, mutation_rate 0,2 per *child*, elite_count 1, dan repair_tries 20. Konfigurasi eksperimen dituliskan sebagai berikut:

```
{
  "problem": "configs/problem.json",
  "output_dir": "output/genetic_algorithm",
  "ga_defaults": {"crossover_rate": 0.9, "mutation_rate": 0.2,
"elite_count": 1, "repair_tries": 20, "record_snapshots": true},
  "experiment_a": {"population_size": 50, "generations": [25, 100, 400],
                   "seeds": [[5001, 5002, 5003], [5004, 5005, 5006],
[5007, 5008, 5009]]},
  "experiment_b": {"generations": 100, "population_sizes": [10, 50, 200],
                   "seeds": [[5101, 5102, 5103], [5104, 5105, 5106],
[5107, 5108, 5109]]},
  "baseline": {"samples": 1000, "seed": 0}
}
```

Ketiga konfigurasi pada tiap skenario dijalankan melalui perintah berikut:

```
python -m src.main experiment --config configs/experiment.json
```

Fungsi build_runs() pada src/experiments.py menerjemahkan konfigurasi tersebut menjadi 18 spesifikasi run. Fungsi ini juga menolak konfigurasi yang tidak memiliki tepat tiga variasi, tiga *seed* per konfigurasi, dan *seed* yang unik:

```
def build_runs(config: dict) -> list[dict]:
    a, b = config["experiment_a"], config["experiment_b"]
    _check_group("experiment_a.generations", a["generations"],
a["seeds"])
    _check_group("experiment_b.population_sizes", b["population_sizes"],
b["seeds"])
    runs = []
    for gens, seeds in zip(a["generations"], a["seeds"]):
        runs += [dict(experiment="A",
population_size=a["population_size"], generations=gens, seed=s) for s in
seeds]
    for pop, seeds in zip(b["population_sizes"], b["seeds"]):
        runs += [dict(experiment="B", population_size=pop,
generations=b["generations"], seed=s) for s in seeds]
    if len({r["seed"] for r in runs}) != len(runs):
        raise ValueError("seed antar run harus unik agar tiap run
independen")
    return runs
```

Di dalam kode, skenario 1 diberi label A dan skenario 2 diberi label B. Setiap run menyimpan hasil lengkap ke berkas JSON dan grafik convergent ke file PNG. Evaluasi hasil memakai metrik berikut.

Tabel Metrik Evaluasi Eksperimen

Metrik	Pengertian dan penggunaan
Nilai objektif awal	*Fitness* individu terbaik pada populasi awal (generasi 0).
Nilai objektif akhir	*Fitness* terbaik sepanjang pencarian (*best-so-far*), yang sama dengan final_objective pada hasil run.
Fitness maksimum	*Fitness* individu terbaik pada suatu generasi.
Fitness rata-rata	Rata-rata *fitness* seluruh individu pada suatu generasi, sebagai ukuran kualitas populasi secara keseluruhan.

Generasi penemuan terbaik	Generasi ketika nilai terbaik akhir pertama kali muncul.
Peningkatan relatif	Persentase kenaikan nilai objektif akhir terhadap nilai objektif awal.
Durasi *searching*	Waktu pembentukan populasi awal sampai generasi terakhir, tanpa penyimpanan hasil dan pembuatan grafik.
Jumlah individu unik	Banyaknya individu berbeda pada generasi terakhir, sebagai ukuran keragaman populasi.

Peningkatan relatif dihitung menggunakan rumus yang sama dengan algoritma lain:

$$\text{Peningkatan} = (f_{akhir} - f_{awal}) / f_{awal} \times 100\%$$

Perlu diperhatikan bahwa nilai objektif awal pada GA adalah individu terbaik dari satu populasi acak, sedangkan pada *Hill-Climbing* dan *Simulated Annealing* adalah satu *state* acak. Nilai awal GA karena itu cenderung lebih tinggi, dan persentase peningkatannya tidak dapat dibandingkan langsung dengan algoritma lain. Perbandingan antaralgoritma pada subbab 3.4 memakai nilai objektif akhir.

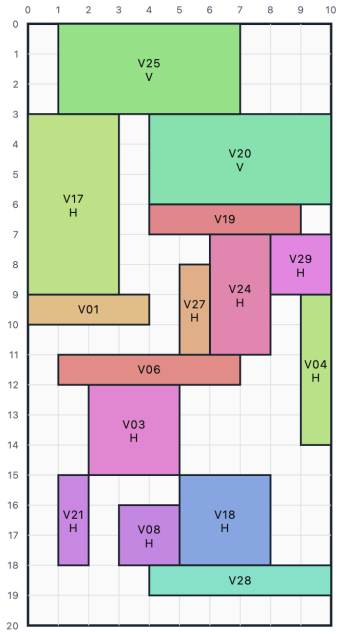
BAB III
HASIL EKSPERIMEN & ANALISIS

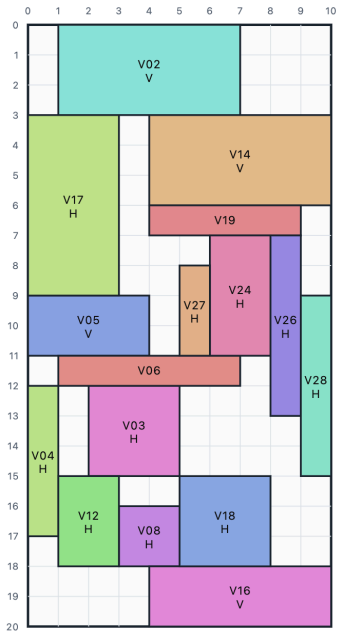
3.1 Hasil Eksperimen Hill-Climbing

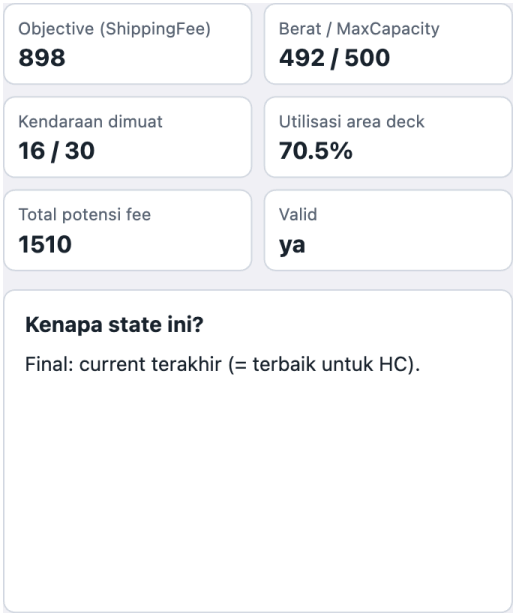
Setiap varian Hill-Climbing dijalankan tiga kali dengan seed berbeda pada instance synthetic-2026 (30 kendaraan, deck 10 x 20, MaxCapacity 500, total potensi fee 1510). Initial state dibangkitkan secara acak dan feasible. Parameter yang digunakan mengikuti Bab II. Plot pada setiap eksperimen memuat state awal, state akhir, dan grafik nilai objektif terhadap banyak iterasi.

3.1.1 Hasil Tiga Kali Run Stochastic Hill-Climbing (Second Approach)

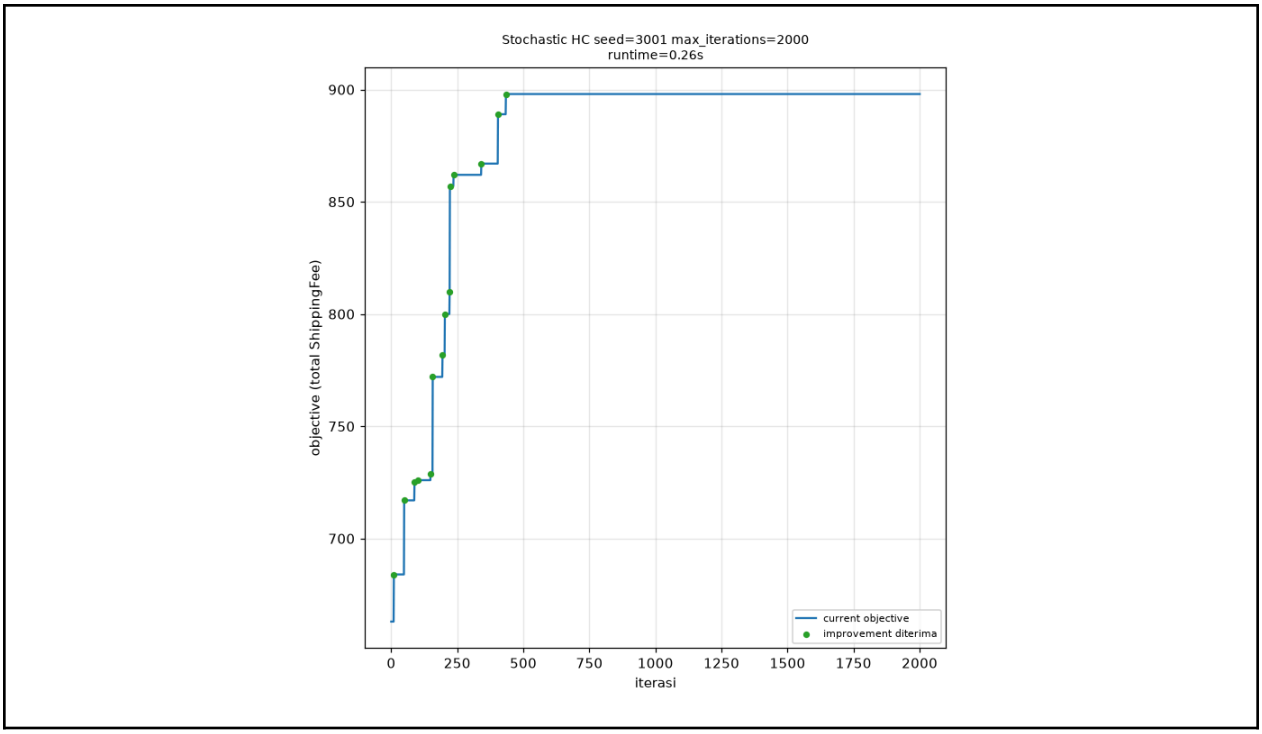
Eksperimen 1 - Stochastic Hill-Climbing (Second Approach)

***State* Awal**	
	Objective (ShippingFee) **663** Berat / MaxCapacity **496 / 500** Kendaraan dimuat **15 / 30** Utilisasi area deck **60.0%** Total potensi fee **1510** Valid **ya** **Kenapa state ini?** Initial: state awal random.
***State* Akhir**	



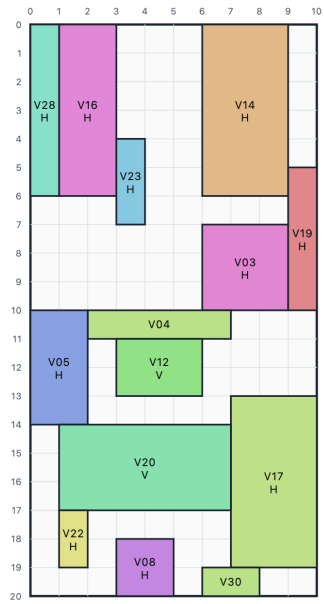
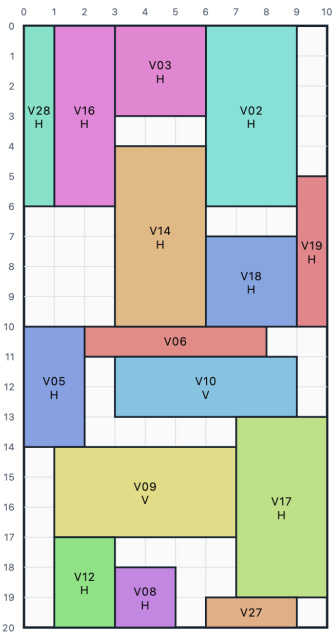


Keterangan	Hasil
Seed	3001
Objective function awal	663
Best-found objective (= objective akhir)	898
Iterasi penemuan best-found	434
Peningkatan nilai objektif	235
Persentase peningkatan	35,44%
Banyak iterasi	2000
Durasi searching	0,213 detik
Kandidat diterima (improvement)	14
Kandidat ditolak (nilai sama)	1032
Kandidat ditolak (lebih buruk)	954
Kondisi berhenti	Mencapai max_iterations (2000)
Plot	



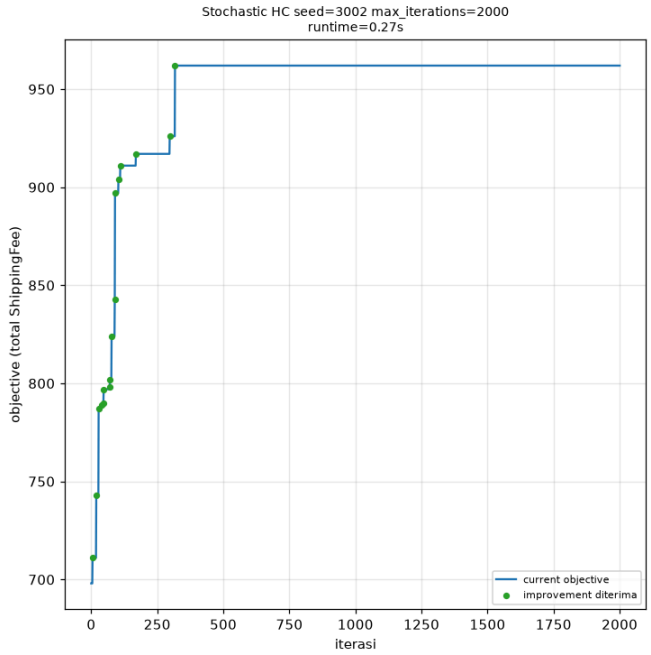
Pada eksperimen pertama, nilai objektif meningkat dari 663 menjadi 898, atau sebesar 35,44%. Dari 2000 iterasi, hanya 14 kandidat yang diterima karena menaikkan nilai objektif. Sebanyak 1032 kandidat ditolak karena bernilai sama dan 954 ditolak karena lebih buruk. Perbaikan terakhir terjadi pada iterasi ke-434, sehingga 1566 iterasi sisanya (78,3%) tidak menghasilkan perbaikan. Initial state ini adalah yang terendah di antara tiga run, tetapi hasil akhirnya berada di posisi tengah, sehingga nilai awal yang rendah tidak otomatis menghasilkan solusi yang buruk. Solusi akhir feasible dengan total bobot 492/500 dan 16 dari 30 kendaraan termuat.

Eksperimen 2 - Stochastic Hill-Climbing (Second Approach)

***State* Awal**	
	Objective (ShippingFee) **698** Berat / MaxCapacity **490 / 500** Kendaraan dimuat **14 / 30** Utilisasi area deck **58.0%** Total potensi fee **1510** Valid **ya** **Kenapa state ini?** Initial: state awal random.
***State* Akhir**	
	Objective (ShippingFee) **962** Berat / MaxCapacity **486 / 500** Kendaraan dimuat **15 / 30** Utilisasi area deck **76.0%** Total potensi fee **1510** Valid **ya** **Kenapa state ini?** Final: current terakhir (= terbaik untuk HC).

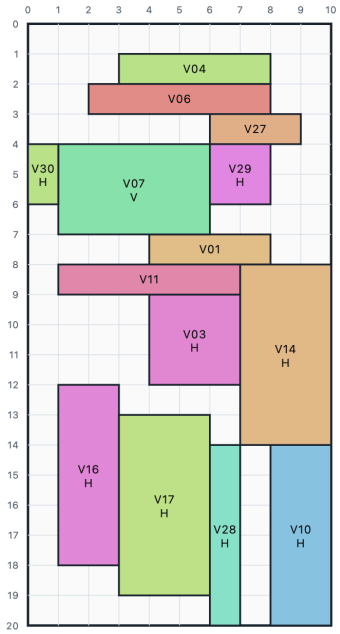
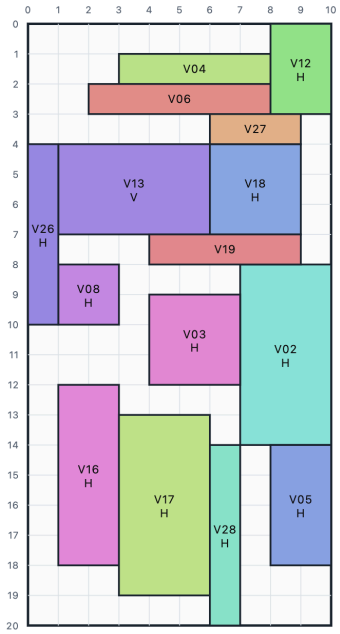
Keterangan	**Hasil**
Seed	3002
Objective function awal	698
Best-found objective (= objective akhir)	962

Iterasi penemuan best-found	317
Peningkatan nilai objektif	264
Persentase peningkatan	37,82%
Banyak iterasi	2000
Durasi searching	0,210 detik
Kandidat diterima (improvement)	16
Kandidat ditolak (nilai sama)	273
Kandidat ditolak (lebih buruk)	1711
Kondisi berhenti	Mencapai max_iterations (2000)
Plot	



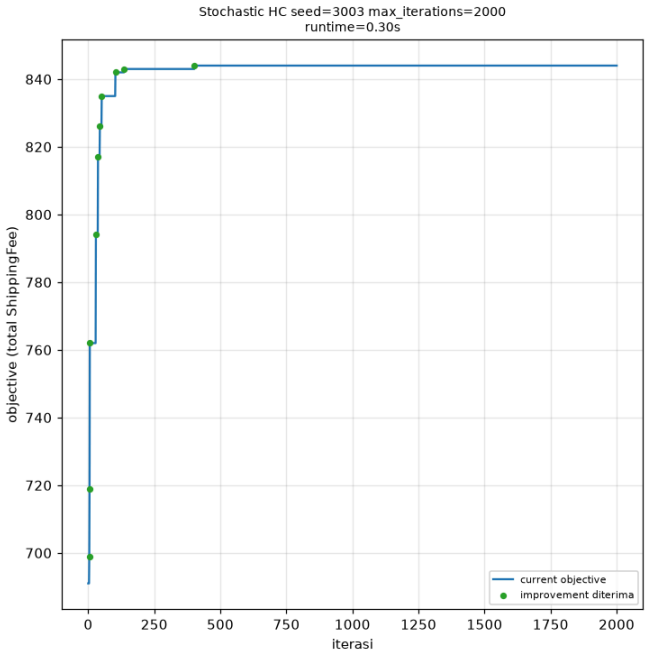
Pada eksperimen kedua, nilai objektif meningkat dari 698 menjadi 962, atau sebesar 37,82%. Dari 2000 iterasi, hanya 16 kandidat yang diterima karena menaikkan nilai objektif. Sebanyak 273 kandidat ditolak karena bernilai sama dan 1711 ditolak karena lebih buruk. Perbaikan terakhir terjadi pada iterasi ke-317, sehingga 1683 iterasi sisanya (84,2%) tidak menghasilkan perbaikan. Run ini memperoleh nilai akhir tertinggi di antara tiga run Second Approach. Initial state-nya juga yang tertinggi (698), sehingga dengan tiga sampel belum cukup untuk memisahkan pengaruh state awal dari pengaruh urutan kandidat acak. Solusi akhir feasible dengan total bobot 486/500 dan 15 dari 30 kendaraan termuat.

Eksperimen 3 - Stochastic Hill-Climbing (Second Approach)

***State* Awal**	
	Objective (ShippingFee) **691** Berat / MaxCapacity **499 / 500** Kendaraan dimuat **14 / 30** Utilisasi area deck **60.0%** Total potensi fee **1510** Valid **ya** **Kenapa state ini?** Initial: state awal random.
***State* Akhir**	
	Objective (ShippingFee) **844** Berat / MaxCapacity **485 / 500** Kendaraan dimuat **15 / 30** Utilisasi area deck **65.0%** Total potensi fee **1510** Valid **ya** **Kenapa state ini?** Final: current terakhir (= terbaik untuk HC).

Keterangan	**Hasil**
Seed	3003
Objective function awal	691
Best-found objective (= objective akhir)	844

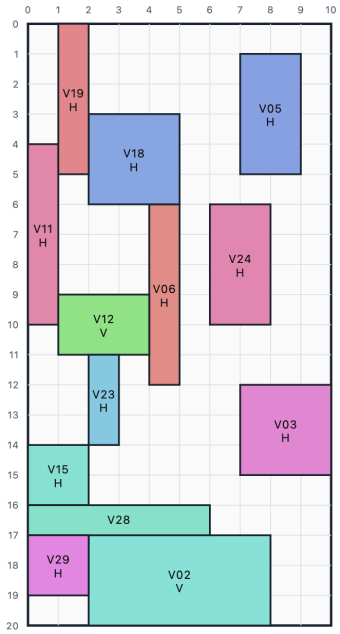
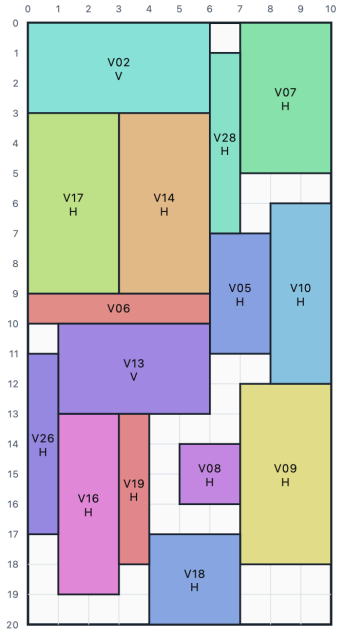
Iterasi penemuan best-found	402
Peningkatan nilai objektif	153
Persentase peningkatan	22,14%
Banyak iterasi	2000
Durasi searching	0,224 detik
Kandidat diterima (improvement)	10
Kandidat ditolak (nilai sama)	488
Kandidat ditolak (lebih buruk)	1502
Kondisi berhenti	Mencapai max_iterations (2000)
Plot	



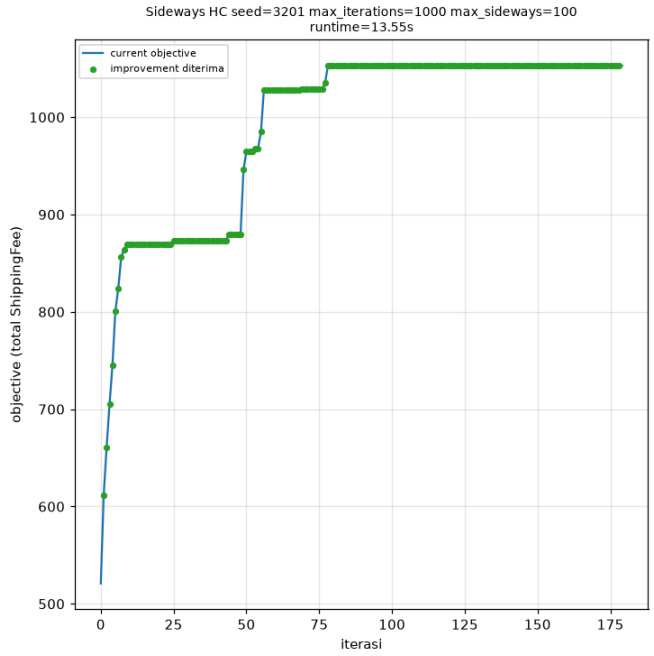
Pada eksperimen ketiga, nilai objektif meningkat dari 691 menjadi 844, atau sebesar 22,14%. Dari 2000 iterasi, hanya 10 kandidat yang diterima karena menaikkan nilai objektif. Sebanyak 488 kandidat ditolak karena bernilai sama dan 1502 ditolak karena lebih buruk. Perbaikan terakhir terjadi pada iterasi ke-402, sehingga 1598 iterasi sisanya (79,9%) tidak menghasilkan perbaikan. Run ini memperoleh nilai akhir terendah meskipun initial state-nya lebih tinggi daripada run pertama. Hanya 10 perbaikan yang diterima, paling sedikit di antara tiga run. Solusi akhir feasible dengan total bobot 485/500 dan 15 dari 30 kendaraan termuat.

3.1.1.2 Hill-Climbing dengan Sideways Move

Eksperimen 1 - Hill-Climbing dengan Sideways Move

***State* Awal**	
	Objective (ShippingFee) **521** Berat / MaxCapacity **489 / 500** Kendaraan dimuat **13 / 30** Utilisasi area deck **46.0%** Total potensi fee **1510** Valid **ya** **Kenapa state ini?** Initial: state awal random.
***State* Akhir**	
	Objective (ShippingFee) **1053** Berat / MaxCapacity **497 / 500** Kendaraan dimuat **15 / 30** Utilisasi area deck **85.0%** Total potensi fee **1510** Valid **ya** **Kenapa state ini?** Final: current saat berhenti (sideways move tidak menurunkan nilai).
Keterangan	**Hasil**
Seed	3201

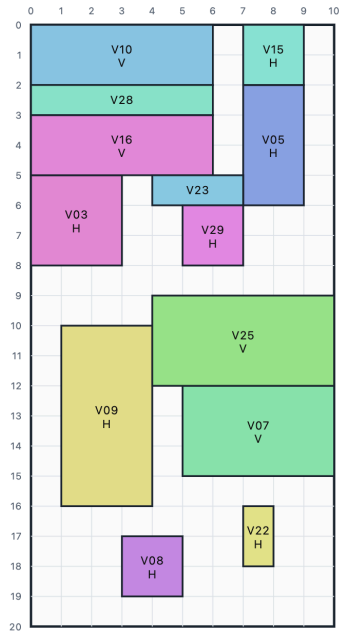
Objective function awal	521
Best-found objective (= objective akhir)	1053
Iterasi penemuan best-found	78
Peningkatan nilai objektif	532
Persentase peningkatan	102,11%
Banyak iterasi	179
Durasi searching	10,344 detik
Langkah yang menaikkan nilai (improvement)	19
Total sideways move	159
Sideways move beruntun terpanjang	100
Jumlah neighbor dievaluasi	24.925
Kondisi berhenti	Sideways beruntun mencapai max_sideways (100)
Plot	

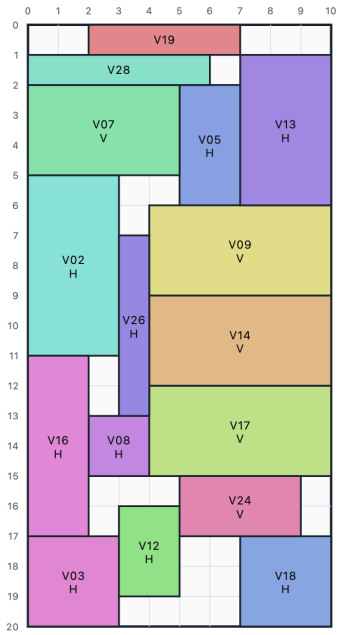


Pada eksperimen pertama, nilai objektif meningkat dari 521 menjadi 1053, atau sebesar 102,11%. Dari 179 iterasi, 19 langkah menaikkan nilai objektif dan 159 langkah merupakan sideways move, yaitu perpindahan ke state dengan nilai objektif yang sama. Perbaikan terakhir

terjadi pada iterasi ke-78. Setelah itu, algoritma menjalani 100 sideways move berturut-turut tanpa perbaikan sehingga batas max_sideways tercapai dan searching dihentikan. Kondisi ini menunjukkan plateau yang tidak berhasil dilewati dalam batas toleransi, bukan bukti bahwa state akhir adalah local optimum yang sebenarnya. Initial state ini adalah yang terendah di antara tiga run (521), tetapi peningkatan yang diperoleh paling besar, yaitu 532 poin. Solusi akhir feasible dengan total bobot 497/500 dan 15 dari 30 kendaraan termuat.

Eksperimen 2 - Hill-Climbing dengan Sideways Move

***State* Awal**	
	Objective (ShippingFee) **601** Berat / MaxCapacity **498 / 500** Kendaraan dimuat **13 / 30** Utilisasi area deck **57.5%** Total potensi fee **1510** Valid **ya** **Kenapa state ini?** Initial: state awal random.
***State* Akhir**	



Objective (ShippingFee) **1071**

Berat / MaxCapacity **498 / 500**

Kendaraan dimuat **16 / 30**

Utilisasi area deck **87.5%**

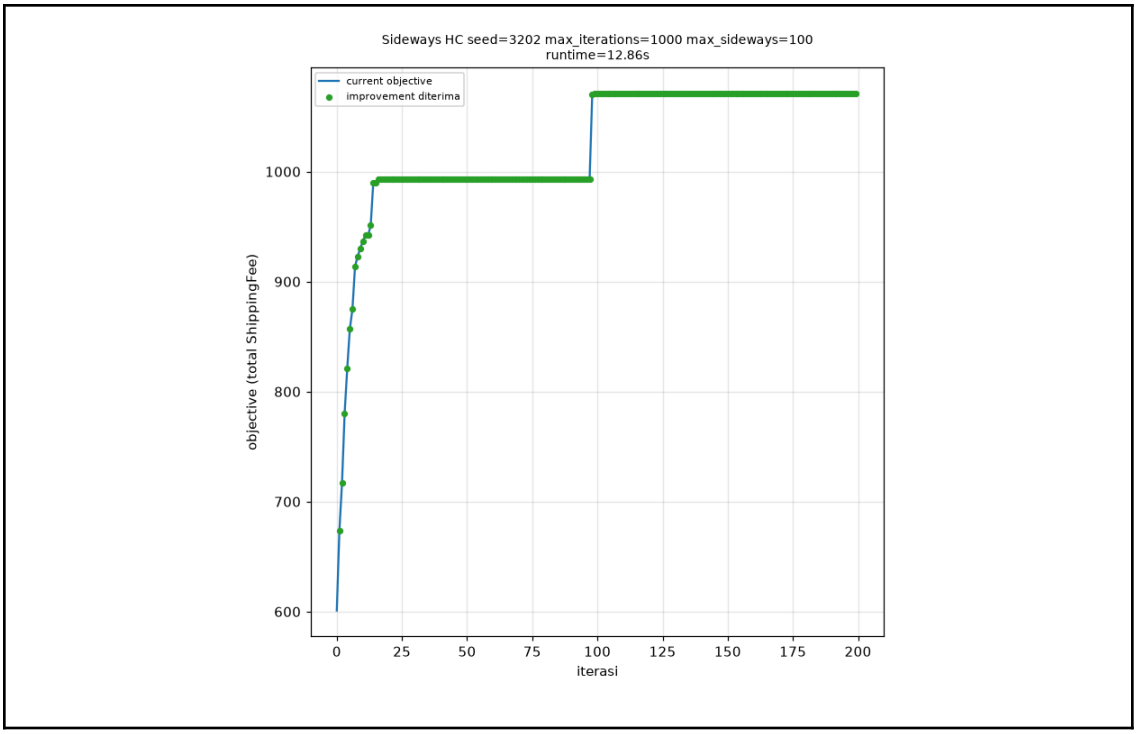
Total potensi fee **1510**

Valid **ya**

Kenapa state ini?

Final: current saat berhenti (sideways move tidak menurunkan nilai).

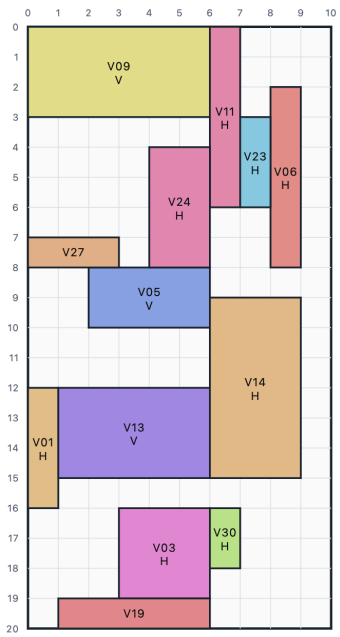
Keterangan	Hasil
Seed	3202
Objective function awal	601
Best-found objective (= objective akhir)	1071
Iterasi penemuan best-found	99
Peningkatan nilai objektif	470
Persentase peningkatan	78,20%
Banyak iterasi	200
Durasi searching	11,419 detik
Langkah yang menaikkan nilai (improvement)	16
Total sideways move	183
Sideways move beruntun terpanjang	100
Jumlah neighbor dievaluasi	19.545
Kondisi berhenti	Sideways beruntun mencapai max_sideways (100)
Plot	



Pada eksperimen kedua, nilai objektif meningkat dari 601 menjadi 1071, atau sebesar 78,20%. Dari 200 iterasi, 16 langkah menaikkan nilai objektif dan 183 langkah merupakan sideways move, yaitu perpindahan ke state dengan nilai objektif yang sama. Perbaikan terakhir terjadi pada iterasi ke-99. Setelah itu, algoritma menjalani 100 sideways move berturut-turut tanpa perbaikan sehingga batas max_sideways tercapai dan searching dihentikan. Kondisi ini menunjukkan plateau yang tidak berhasil dilewati dalam batas toleransi, bukan bukti bahwa state akhir adalah local optimum yang sebenarnya. Nilai akhir berada di posisi tengah dan jumlah neighbor yang dievaluasi paling sedikit di antara tiga run. Solusi akhir feasible dengan total bobot 498/500 dan 16 dari 30 kendaraan termuat.

Eksperimen 3 - Hill-Climbing dengan Sideways Move

***State* Awal**



Objective (ShippingFee) **642**

Berat / MaxCapacity **498 / 500**

Kendaraan dimuat **13 / 30**

Utilisasi area deck **52.5%**

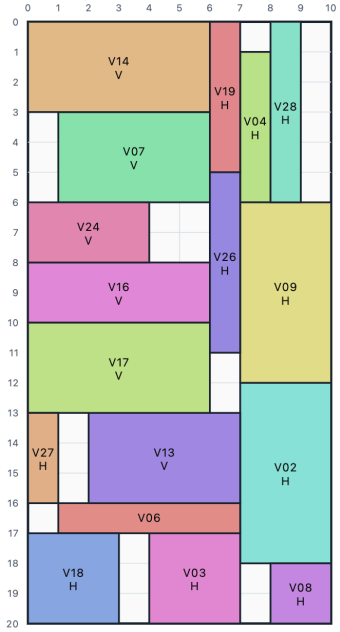
Total potensi fee **1510**

Valid **ya**

Kenapa state ini?

Initial: state awal random.

***State* Akhir**



Objective (ShippingFee) **1087**

Berat / MaxCapacity **497 / 500**

Kendaraan dimuat **17 / 30**

Utilisasi area deck **87.5%**

Total potensi fee **1510**

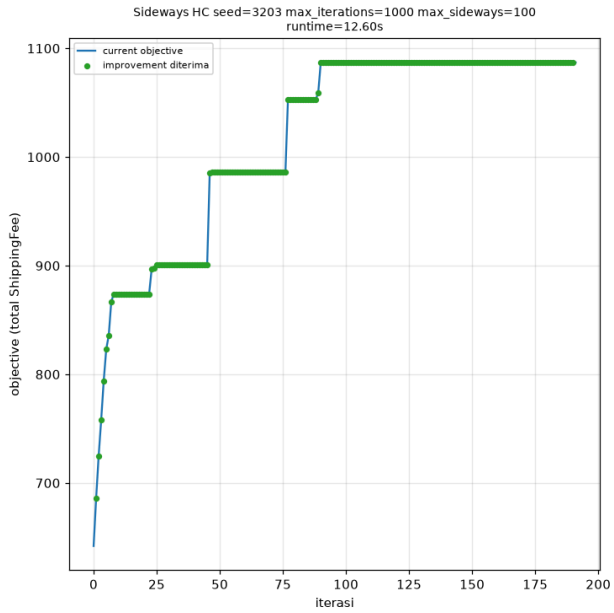
Valid **ya**

Kenapa state ini?

Final: current saat berhenti (sideways move tidak menurunkan nilai).

Keterangan	Hasil
Seed	3203
Objective function awal	642
Best-found objective (= objective akhir)	1087
Iterasi penemuan best-found	90
Peningkatan nilai objektif	445

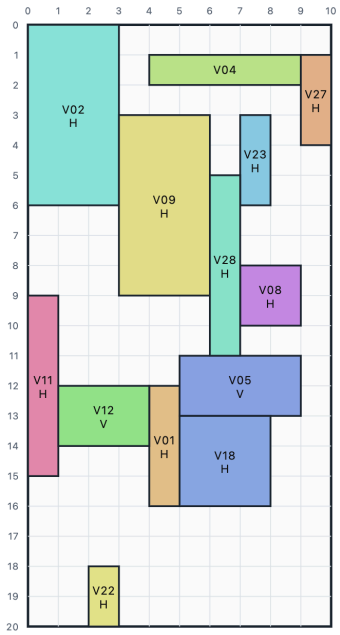
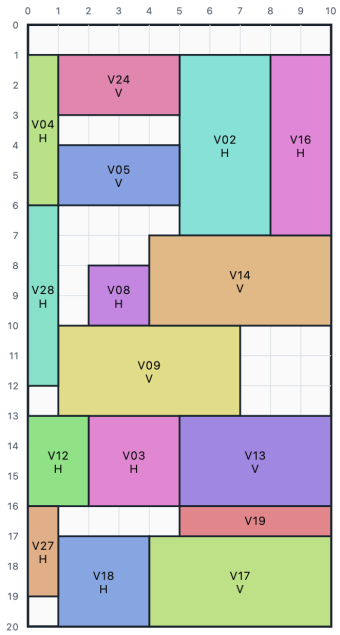
Persentase peningkatan	69,31%
Banyak iterasi	191
Durasi searching	10,862 detik
Langkah yang menaikkan nilai (improvement)	16
Total sideways move	174
Sideways move beruntun terpanjang	100
Jumlah neighbor dievaluasi	22.872
Kondisi berhenti	Sideways beruntun mencapai max_sideways (100)
Plot	



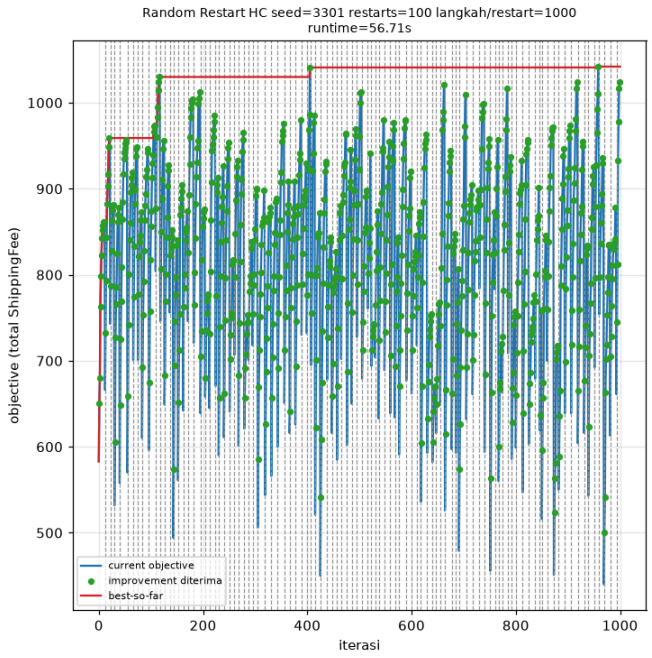
Pada eksperimen ketiga, nilai objektif meningkat dari 642 menjadi 1087, atau sebesar 69,31%. Dari 191 iterasi, 16 langkah menaikkan nilai objektif dan 174 langkah merupakan sideways move, yaitu perpindahan ke state dengan nilai objektif yang sama. Perbaikan terakhir terjadi pada iterasi ke-90. Setelah itu, algoritma menjalani 100 sideways move berturut-turut tanpa perbaikan sehingga batas max_sideways tercapai dan searching dihentikan. Kondisi ini menunjukkan plateau yang tidak berhasil dilewati dalam batas toleransi, bukan bukti bahwa state akhir adalah local optimum yang sebenarnya. Run ini memperoleh nilai akhir tertinggi di antara tiga run sideways, dengan initial state tertinggi (642). Solusi akhir feasible dengan total bobot 497/500 dan 17 dari 30 kendaraan termuat.

3.1.1.3 Random Restart Hill-Climbing

Eksperimen 1 - Random Restart Hill-Climbing

***State* Awal**	
	Objective (ShippingFee) **583** Berat / MaxCapacity **500 / 500** Kendaraan dimuat **13 / 30** Utilisasi area deck **46.0%** Total potensi fee **1510** Valid **ya** **Kenapa state ini?** Initial: state awal random.
***State* Akhir**	
	Objective (ShippingFee) **1042** Berat / MaxCapacity **498 / 500** Kendaraan dimuat **16 / 30** Utilisasi area deck **81.0%** Total potensi fee **1510** Valid **ya** **Kenapa state ini?** Final: peak terbaik di antara semua restart.
Keterangan	**Hasil**

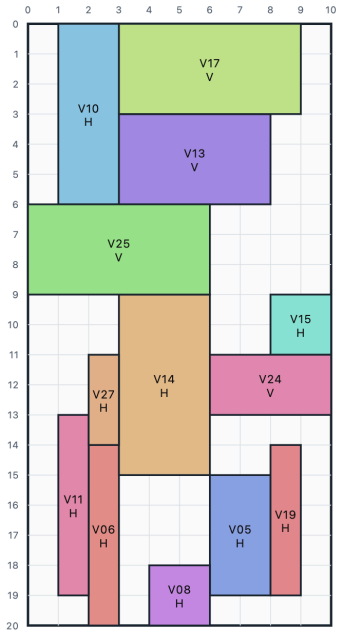
Seed	3301
Objective function awal (restart pertama)	583
Best-found objective	1042
Restart penemu best-found	ke-96 dari 100
Objective awal restart terbaik	692
Peningkatan nilai objektif (dari awal restart pertama)	459
Persentase peningkatan	78,73%
Rata-rata objective akhir per restart	907,54
Objective akhir terendah dan tertinggi antar restart	670 dan 1042
Total langkah perbaikan (seluruh restart)	901
Jumlah neighbor dievaluasi	257.788
Durasi searching	56,709 detik
Kondisi berhenti	100 restart selesai, seluruhnya berakhir di local optimum
Plot	



Pada eksperimen pertama, nilai terbaik sebesar 1042 ditemukan pada restart ke-96 dari 100 restart. Dibandingkan objective awal restart pertama (583), peningkatannya sebesar 459 atau 78,73%. Setiap restart berhenti di local optimum setelah rata-rata 9,01 langkah perbaikan. Rata-rata objective akhir per restart hanya 907,54, dengan rentang 670 sampai 1042. Artinya, nilai terbaik berasal dari pemilihan hasil terbaik di antara 100 local optimum, bukan dari kualitas tiap restart. Restart terbaik justru dimulai dari objective awal 692, di atas rata-rata objective awal seluruh restart (631,27). Total 257.788 neighbor dievaluasi dengan durasi 56,71 detik. Solusi akhir feasible dengan total bobot 498/500 dan 16 dari 30 kendaraan termuat.

Eksperimen 2 - Random Restart Hill-Climbing

***State* Awal**



Objective (ShippingFee)
739

Berat / MaxCapacity
491 / 500

Kendaraan dimuat
13 / 30

Utilisasi area deck
62.5%

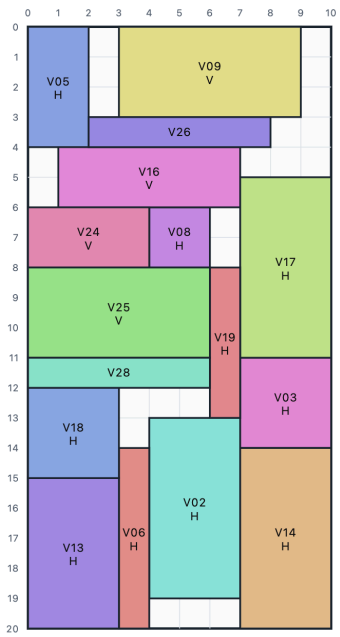
Total potensi fee
1510

Valid
ya

Kenapa state ini?

Initial: state awal random.

***State* Akhir**



Objective (ShippingFee) **1101**

Berat / MaxCapacity **491 / 500**

Kendaraan dimuat **16 / 30**

Utilisasi area deck **89.0%**

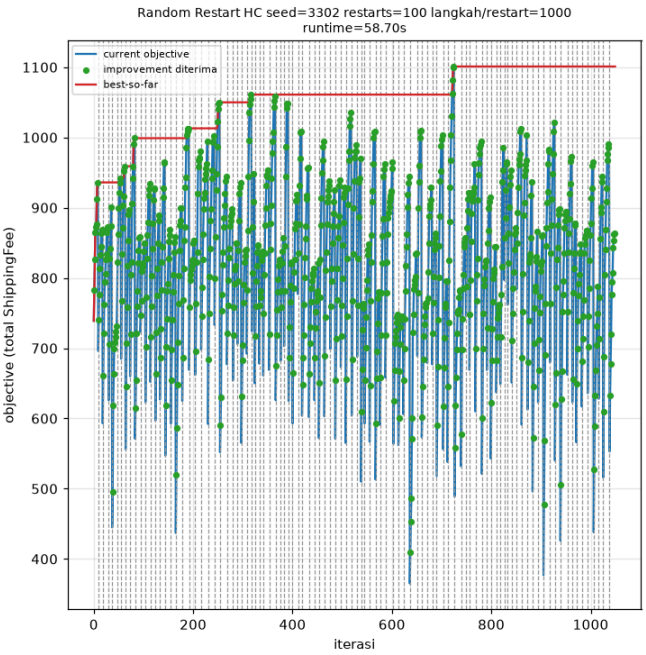
Total potensi fee **1510**

Valid **ya**

Kenapa state ini?

Final: peak terbaik di antara semua restart.

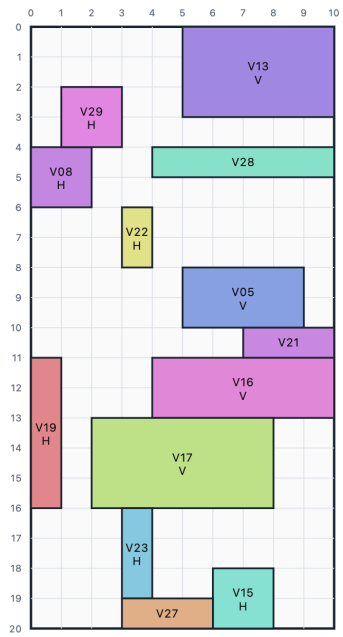
Keterangan	Hasil
Seed	3302
Objective function awal (restart pertama)	739
Best-found objective	1101
Restart penemu best-found	ke-69 dari 100
Objective awal restart terbaik	538
Peningkatan nilai objektif (dari awal restart pertama)	362
Persentase peningkatan	48,99%
Rata-rata objective akhir per restart	913,25
Objective akhir terendah dan tertinggi antar restart	731 dan 1101
Total langkah perbaikan (seluruh restart)	950
Jumlah neighbor dievaluasi	279.374
Durasi searching	58,702 detik
Kondisi berhenti	100 restart selesai, seluruhnya berakhir di local optimum
Plot	



Pada eksperimen kedua, nilai terbaik sebesar 1101 ditemukan pada restart ke-69 dari 100 restart. Dibandingkan objective awal restart pertama (739), peningkatannya sebesar 362 atau 48,99%. Setiap restart berhenti di local optimum setelah rata-rata 9,50 langkah perbaikan. Rata-rata objective akhir per restart hanya 913,25, dengan rentang 731 sampai 1101. Artinya, nilai terbaik berasal dari pemilihan hasil terbaik di antara 100 local optimum, bukan dari kualitas tiap restart. Restart terbaik dimulai dari objective awal 538, di bawah rata-rata objective awal seluruh restart (615,65). Run ini menghasilkan nilai akhir tertinggi di antara seluruh 12 eksperimen Hill-Climbing. Total 279.374 neighbor dievaluasi dengan durasi 58,70 detik. Solusi akhir feasible dengan total bobot 491/500 dan 16 dari 30 kendaraan termuat.

Eksperimen 3 - Random Restart Hill-Climbing

***State* Awal**



Objective (ShippingFee)	Berat / MaxCapacity
582	**489 / 500**
Kendaraan dimuat	Utilisasi area deck
13 / 30	**43.5%**
Total potensi fee	Valid
1510	**ya**

Kenapa state ini?

Initial: state awal random.

***State* Akhir**



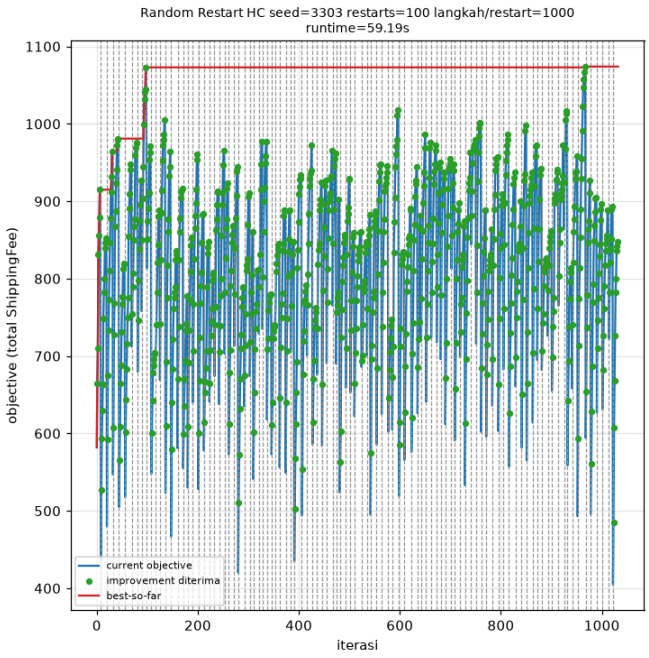
Objective (ShippingFee)	Berat / MaxCapacity
1074	**482 / 500**
Kendaraan dimuat	Utilisasi area deck
16 / 30	**86.0%**
Total potensi fee	Valid
1510	**ya**

Kenapa state ini?

Final: peak terbaik di antara semua restart.

Keterangan	Hasil
Seed	3303
Objective function awal (restart pertama)	582
Best-found objective	1074
Restart penemu best-found	ke-94 dari 100
Objective awal restart terbaik	493

Peningkatan nilai objektif (dari awal restart pertama)	492
Persentase peningkatan	84,54%
Rata-rata objective akhir per restart	908,41
Objective akhir terendah dan tertinggi antar restart	704 dan 1074
Total langkah perbaikan (seluruh restart)	933
Jumlah neighbor dievaluasi	272.354
Durasi searching	59,186 detik
Kondisi berhenti	100 restart selesai, seluruhnya berakhir di local optimum
Plot	

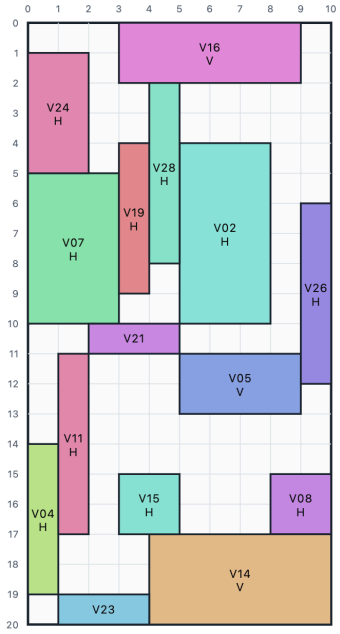


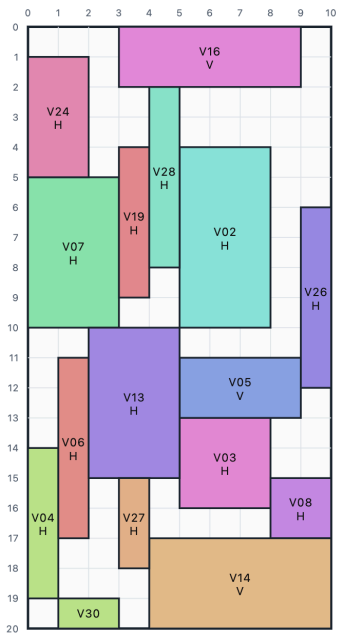
Pada eksperimen ketiga, nilai terbaik sebesar 1074 ditemukan pada restart ke-94 dari 100 restart. Dibandingkan objective awal restart pertama (582), peningkatannya sebesar 492 atau 84,54%. Setiap restart berhenti di local optimum setelah rata-rata 9,33 langkah perbaikan. Rata-rata objective akhir per restart hanya 908,41, dengan rentang 704 sampai 1074. Artinya, nilai terbaik berasal dari pemilihan hasil terbaik di antara 100 local optimum, bukan dari kualitas tiap restart. Restart terbaik dimulai dari objective awal 493, jauh di bawah rata-rata objective awal seluruh

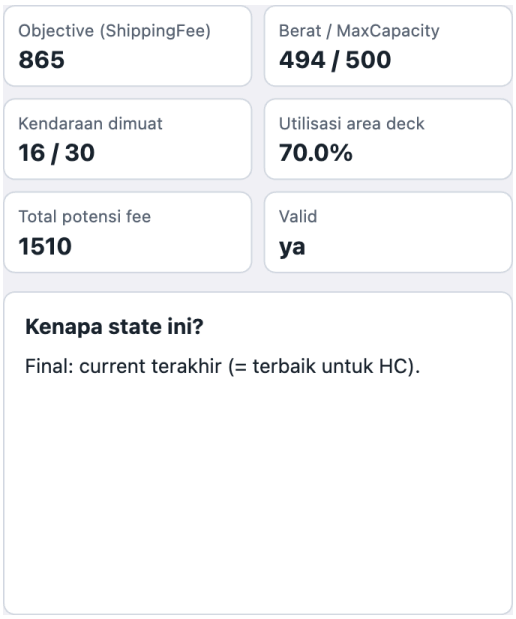
restart (624,50), sehingga kualitas state awal restart tidak menentukan kualitas local optimum yang dicapai. Total 272.354 neighbor dievaluasi dengan durasi 59,19 detik. Solusi akhir feasible dengan total bobot 482/500 dan 16 dari 30 kendaraan termuat.

3.1.1.4 Stochastic Hill-Climbing (First Approach)

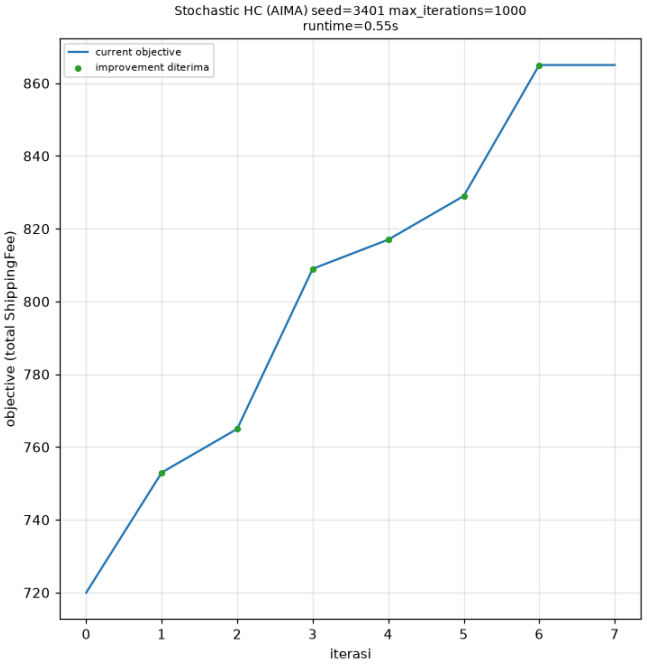
Eksperimen 1 - Stochastic Hill-Climbing (First Approach)

***State* Awal**	
	Objective (ShippingFee) **720** Berat / MaxCapacity **494 / 500** Kendaraan dimuat **15 / 30** Utilisasi area deck **60.5%** Total potensi fee **1510** Valid **ya** **Kenapa state ini?** Initial: state awal random.
***State* Akhir**	





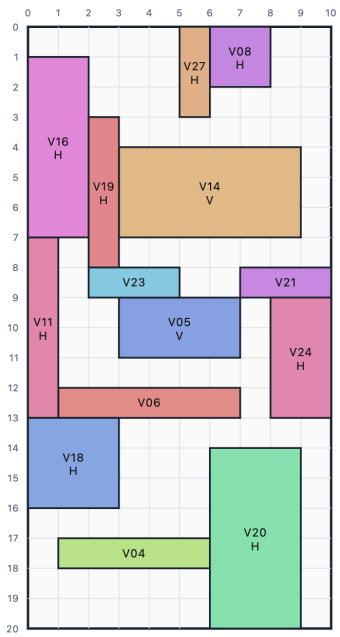
Keterangan	Hasil
Seed	3401
Objective function awal	720
Best-found objective (= objective akhir)	865
Iterasi penemuan best-found	6
Peningkatan nilai objektif	145
Persentase peningkatan	20,14%
Banyak iterasi	7
Durasi searching	0,392 detik
Langkah yang menaikkan nilai (improvement)	6
Jumlah neighbor dievaluasi	1.704
Mekanisme seleksi	Proporsional terhadap besar kenaikan
Kondisi berhenti	Tidak ada neighbor yang lebih baik (local optimum)
Plot	



Pada eksperimen pertama, nilai objektif meningkat dari 720 menjadi 865, atau sebesar 20,14%. Algoritma berhenti pada iterasi ke-7 karena tidak ada lagi neighbor yang menaikkan nilai objektif, sehingga state akhir merupakan local optimum terhadap operator swap, relocate, dan rotate. Dari 7 iterasi, 6 langkah menaikkan nilai objektif dengan pemilihan acak yang proporsional terhadap besar kenaikan. Nilai akhir berada di posisi tengah di antara tiga run. Solusi akhir feasible dengan total bobot 494/500 dan 16 dari 30 kendaraan termuat.

Eksperimen 2 - Stochastic Hill-Climbing (First Approach)

***State* Awal**



Objective (ShippingFee) **642**

Berat / MaxCapacity **492 / 500**

Kendaraan dimuat **14 / 30**

Utilisasi area deck **54.0%**

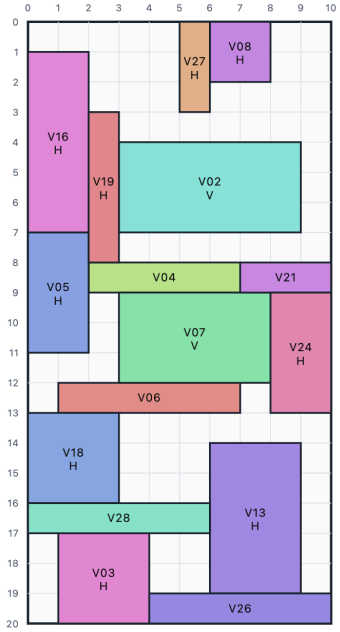
Total potensi fee **1510**

Valid **ya**

Kenapa state ini?

Initial: state awal random.

***State* Akhir**



Objective (ShippingFee) **799**

Berat / MaxCapacity **496 / 500**

Kendaraan dimuat **16 / 30**

Utilisasi area deck **66.0%**

Total potensi fee **1510**

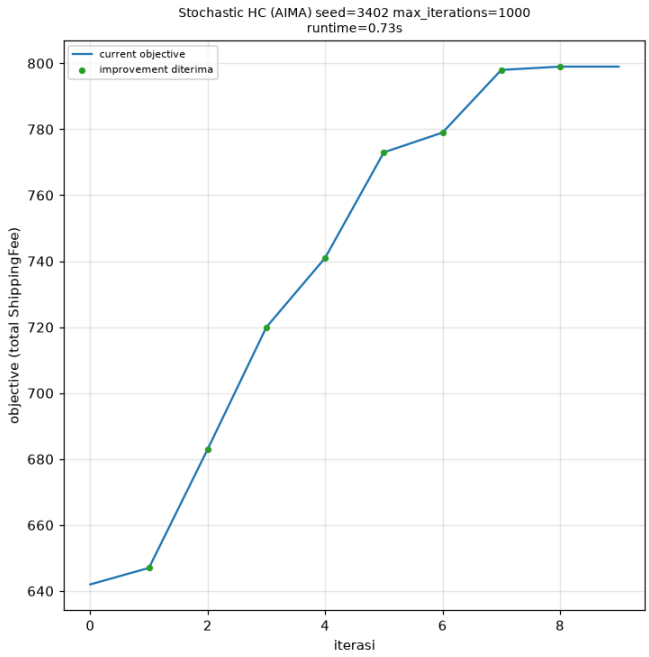
Valid **ya**

Kenapa state ini?

Final: current terakhir (= terbaik untuk HC).

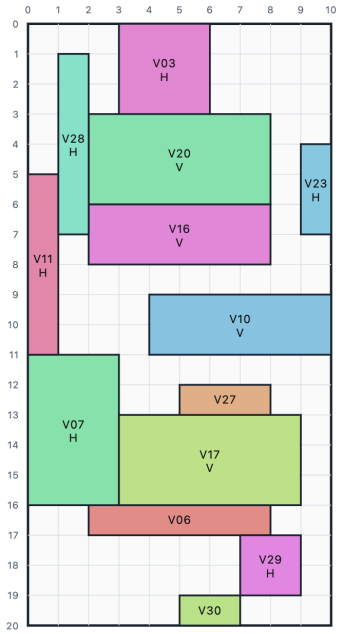
Keterangan	Hasil
Seed	3402
Objective function awal	642
Best-found objective (= objective akhir)	799
Iterasi penemuan best-found	8
Peningkatan nilai objektif	157

Persentase peningkatan	24,45%
Banyak iterasi	9
Durasi searching	0,502 detik
Langkah yang menaikkan nilai (improvement)	8
Jumlah neighbor dievaluasi	2.294
Mekanisme seleksi	Proporsional terhadap besar kenaikan
Kondisi berhenti	Tidak ada neighbor yang lebih baik (local optimum)
Plot	



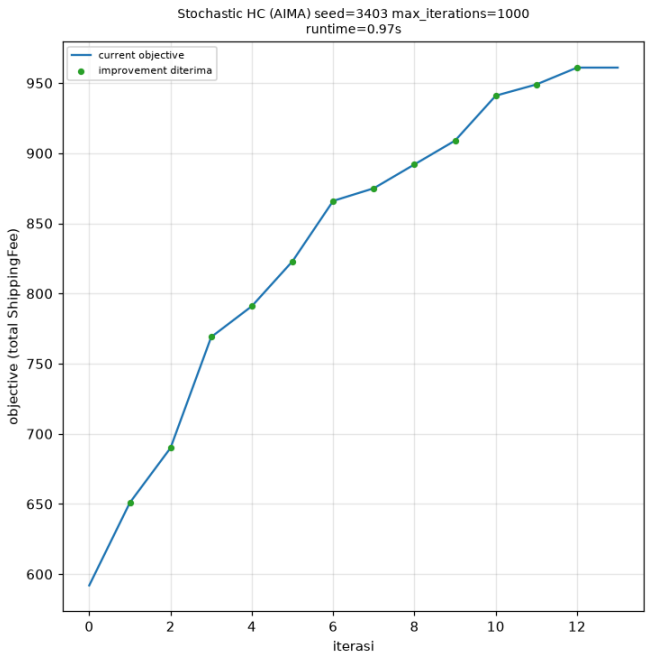
Pada eksperimen kedua, nilai objektif meningkat dari 642 menjadi 799, atau sebesar 24,45%. Algoritma berhenti pada iterasi ke-9 karena tidak ada lagi neighbor yang menaikkan nilai objektif, sehingga state akhir merupakan local optimum terhadap operator swap, relocate, dan rotate. Dari 9 iterasi, 8 langkah menaikkan nilai objektif dengan pemilihan acak yang proporsional terhadap besar kenaikan. Run ini berhenti pada nilai terendah di antara tiga run, yang menunjukkan bahwa local optimum yang dicapai bisa jauh dari kualitas solusi terbaik. Solusi akhir feasible dengan total bobot 496/500 dan 16 dari 30 kendaraan termuat.

Eksperimen 3 - Stochastic Hill-Climbing (First Approach)

***State* Awal**	
	Objective (ShippingFee) **592** Berat / MaxCapacity **493 / 500** Kendaraan dimuat **13 / 30** Utilisasi area deck **57.0%** Total potensi fee **1510** Valid **ya** **Kenapa state ini?** Initial: state awal random.
***State* Akhir**	
	Objective (ShippingFee) **961** Berat / MaxCapacity **478 / 500** Kendaraan dimuat **16 / 30** Utilisasi area deck **76.5%** Total potensi fee **1510** Valid **ya** **Kenapa state ini?** Final: current terakhir (= terbaik untuk HC).

Keterangan	**Hasil**
Seed	3403
Objective function awal	592
Best-found objective (= objective akhir)	961

Iterasi penemuan best-found	12
Peningkatan nilai objektif	369
Persentase peningkatan	62,33%
Banyak iterasi	13
Durasi searching	0,770 detik
Langkah yang menaikkan nilai (improvement)	12
Jumlah neighbor dievaluasi	3.437
Mekanisme seleksi	Proporsional terhadap besar kenaikan
Kondisi berhenti	Tidak ada neighbor yang lebih baik (local optimum)
Plot	



Pada eksperimen ketiga, nilai objektif meningkat dari 592 menjadi 961, atau sebesar 62,33%. Algoritma berhenti pada iterasi ke-13 karena tidak ada lagi neighbor yang menaikkan nilai objektif, sehingga state akhir merupakan local optimum terhadap operator swap, relocate, dan rotate. Dari 13 iterasi, 12 langkah menaikkan nilai objektif dengan pemilihan acak yang proporsional terhadap besar kenaikan. Run ini memperoleh nilai akhir tertinggi di antara tiga run

First Approach dan paling banyak melakukan perbaikan sebelum berhenti. Solusi akhir feasible dengan total bobot 478/500 dan 16 dari 30 kendaraan termuat.

3.1.2 Analisis Nilai Objective Function, Iterasi, dan Durasi Hill-Climbing

Ringkasan Tiga Kali Run Setiap Varian Hill-Climbing

Varian	Objective akhir (seed 1, 2, 3)	Rata2	Std	Rata2 iterasi	Rata2 evaluasi objective	Rata2 durasi
Stochastic Hill-Climbing (Second Approach)	898, 962, 844	901,33	59,07	2.000	2.001	0,216 detik
Hill-Climbing dengan Sideways Move	1053, 1071, 1087	1070,33	17,01	190	22.448	10,875 detik
Random Restart Hill-Climbing	1042, 1101, 1074	1072,33	29,54	928	269.939	58,199 detik
Stochastic Hill-Climbing (First Approach)	865, 799, 961	875,00	81,46	10	2.479	0,555 detik

Std adalah simpangan baku sampel (n-1). Untuk Random Restart, iterasi adalah total langkah perbaikan pada seluruh restart.

Varian Sideways dan Random Restart menghasilkan nilai objektif yang jauh lebih tinggi daripada dua varian lainnya. Rata-rata nilai akhir Sideways adalah 1070,33 dan Random Restart 1072,33, sedangkan Second Approach hanya 901,33 dan First Approach 875,00. Selisih antara Sideways dan Random Restart hanya sekitar 2 poin, sementara simpangan bakunya 17,01 dan

29,54. Dengan hanya tiga run per varian, selisih tersebut tidak cukup untuk menyatakan salah satunya lebih unggul dalam kualitas solusi. Nilai terbaik di antara seluruh eksperimen Hill-Climbing adalah 1101, diperoleh Random Restart pada seed 3302. Varian First Approach memiliki variasi hasil terbesar (simpangan baku 81,46, rentang 799 sampai 961), sedangkan Sideways paling konsisten.

Perilaku iterasi keempat varian berbeda karena kondisi berhentinya berbeda. Second Approach selalu berjalan sampai batas 2000 iterasi karena hanya mengevaluasi satu kandidat acak per iterasi dan tidak dapat memastikan state akhirnya adalah local optimum. Perbaikan terakhirnya terjadi pada iterasi ke-434, 317, dan 402, sehingga sekitar 78% sampai 84% iterasinya tidak menghasilkan perbaikan. Sideways berhenti setelah 179 sampai 200 iterasi karena 100 sideways move berturut-turut tidak menemukan perbaikan. Perbaikan terakhirnya terjadi pada iterasi ke-78, 99, dan 90. Random Restart menjalankan 100 restart, dan setiap restart berakhir di local optimum setelah sekitar 9 langkah. Rata-rata objective akhir per restart hanya sekitar 908 sampai 913, jauh di bawah nilai terbaiknya, sehingga keunggulan varian ini berasal dari memilih hasil terbaik di antara 100 local optimum. First Approach berhenti paling cepat, yaitu setelah 7, 9, dan 13 iterasi, karena tidak ada lagi neighbor yang lebih baik. Berhenti cepat di local optimum ini sejalan dengan nilai akhirnya yang paling rendah dan paling bervariasi.

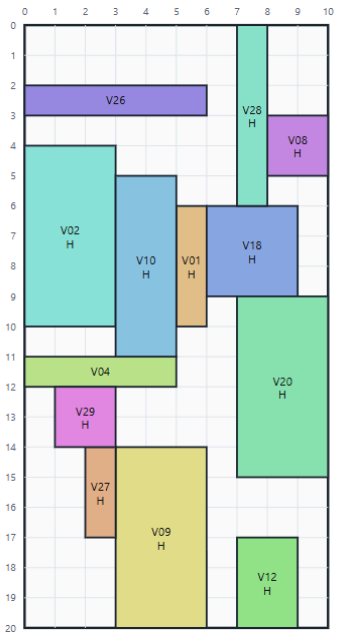
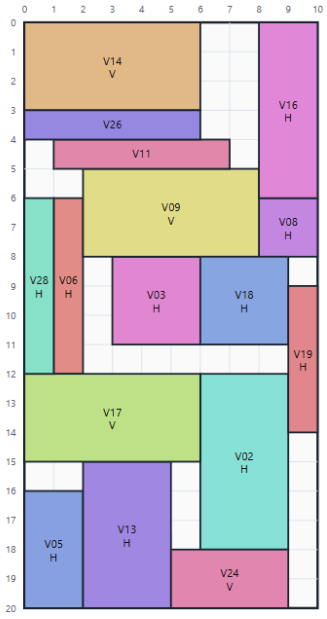
Durasi searching sangat dipengaruhi jumlah evaluasi objective yang dilakukan. Second Approach dan First Approach selesai dalam rata-rata 0,216 dan 0,555 detik. Sideways membutuhkan rata-rata 10,875 detik, sedangkan Random Restart membutuhkan 58,199 detik, atau sekitar 5,4 kali durasi Sideways dengan 12,0 kali jumlah evaluasi objective. Biaya per evaluasi tidak sama antarvarian, sehingga durasi tidak sepenuhnya sebanding dengan jumlah evaluasi. Pada kualitas solusi yang hampir sama, Sideways mencapai hasil tersebut dengan biaya komputasi jauh lebih kecil daripada Random Restart.

Hasil ini menunjukkan bahwa kemampuan melewati plateau (Sideways) dan pengulangan dari banyak titik awal (Random Restart) lebih berpengaruh daripada pemilihan neighbor secara acak (Second Approach dan First Approach) untuk instance ini. Keterbatasannya adalah setiap varian hanya diuji dengan tiga seed, sehingga perbedaan kecil antarvarian maupun antarseed perlu ditafsirkan dengan hati-hati.

3.2 Hasil Eksperimen Simulated Annealing

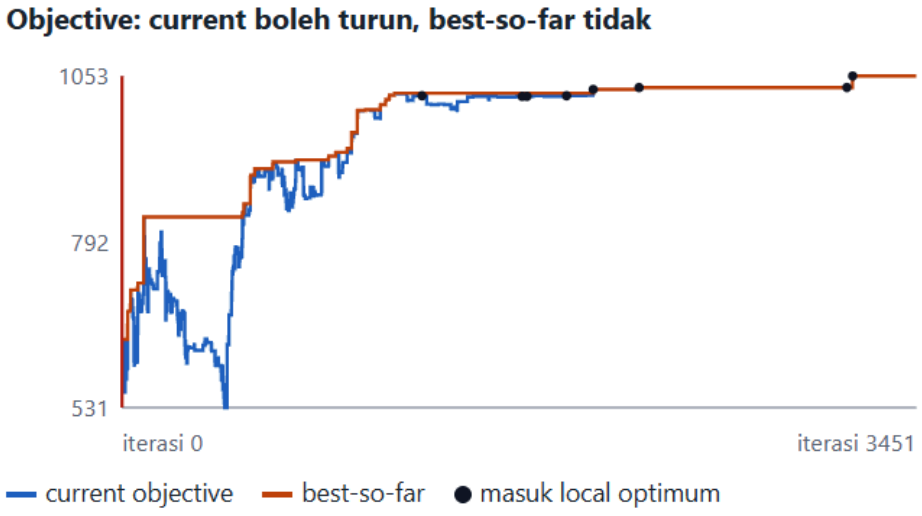
3.2.1 Hasil Tiga Kali Run Simulated Annealing

Eksperimen 1 - *Simulated Annealing*

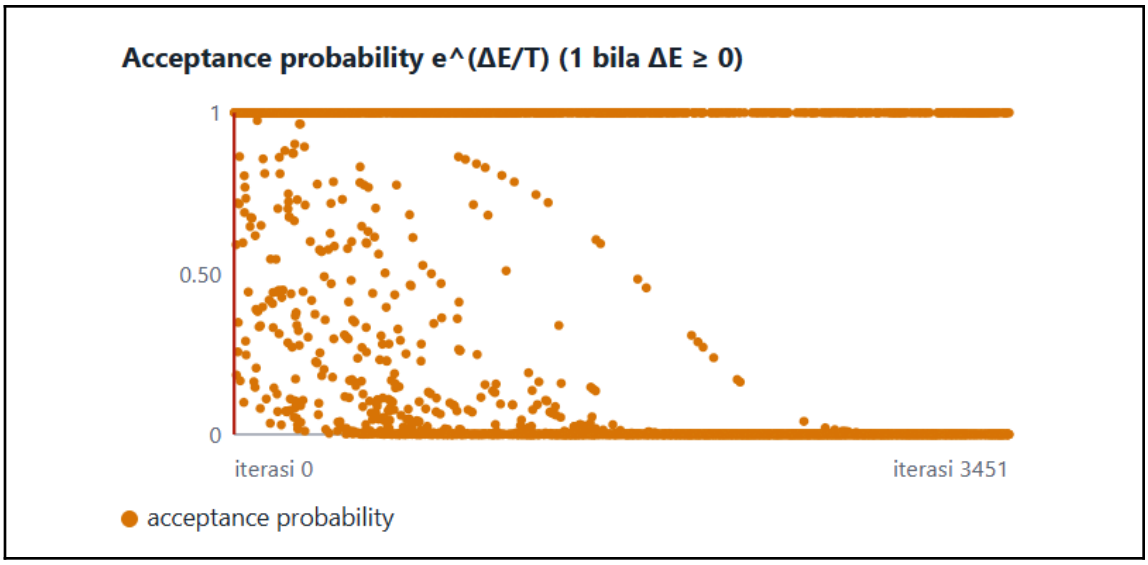
***State* Awal**	
	Objective (ShippingFee) **638** Berat / MaxCapacity **484 / 500** Kendaraan dimuat **13 / 30** Utilisasi area deck **56.5%** Total potensi fee **1510** Feasible (backend) **ya** **Kenapa state ini?** Initial: state awal random.
***State* Akhir**	
	Objective (ShippingFee) **1053** Berat / MaxCapacity **491 / 500** Kendaraan dimuat **16 / 30** Utilisasi area deck **83.0%** Total potensi fee **1510** Feasible (backend) **ya** **Kenapa state ini?** Best-found SA (iterasi 3175).

Keterangan	Hasil
Seed	4001
Objective function awal	638
Best-found objective	1053
Iterasi penemuan best-found	3175
Peningkatan nilai objektif	415
Persentase peningkatan	65,05%
Banyak iterasi	3451
Durasi searching	0,352 detik
Frekuensi stuck di local optimum	8 episode
Jumlah keluar dari local optimum	7
Kondisi berhenti	Temperatur di bawah min_temperature

Plot



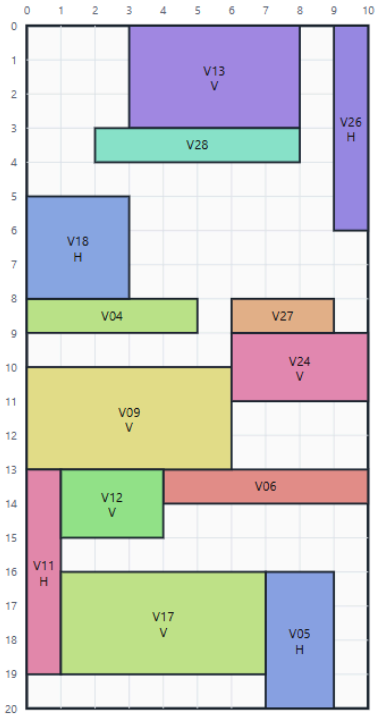
Plot $e^{\frac{\Delta E}{T}}$ **terhadap banyak iterasi yang telah dilewati**



Pada eksperimen pertama, nilai objektif meningkat dari 638 menjadi 1053, atau sebesar 65,05%. Nilai terbaik ditemukan pada iterasi ke-3175. Setelah itu, *searching* berlanjut hingga iterasi ke-3451 tanpa menemukan nilai objektif yang lebih tinggi. Selama *searching*, tercatat delapan episode *stuck* dan tujuh kali keluar dari *local optimum*. Hasil ini menunjukkan bahwa algoritma dapat meninggalkan beberapa kondisi tanpa perbaikan langsung dan melanjutkan eksplorasi. Solusi akhir tetap *feasible* dan memiliki nilai objektif yang sama dengan nilai terbaik selama *searching*.

Eksperimen 2 - *Simulated Annealing*

***State* Awal**



Objective (ShippingFee)
704

Berat / MaxCapacity
490 / 500

Kendaraan dimuat
13 / 30

Utilisasi area deck
57.0%

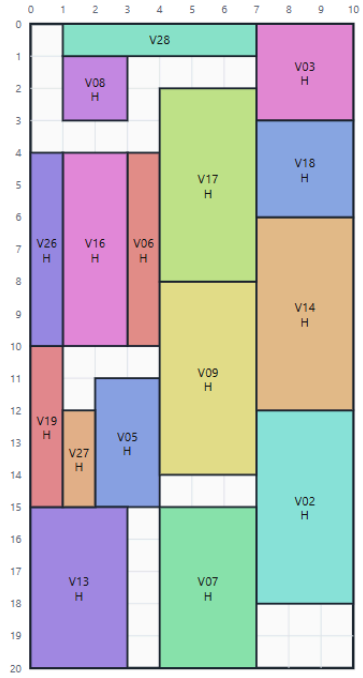
Total potensi fee
1510

Feasible (backend)
ya

Kenapa state ini?

Initial: state awal random.

***State* Akhir**



Objective (ShippingFee)
1074

Berat / MaxCapacity
488 / 500

Kendaraan dimuat
16 / 30

Utilisasi area deck
85.0%

Total potensi fee
1510

Feasible (backend)
ya

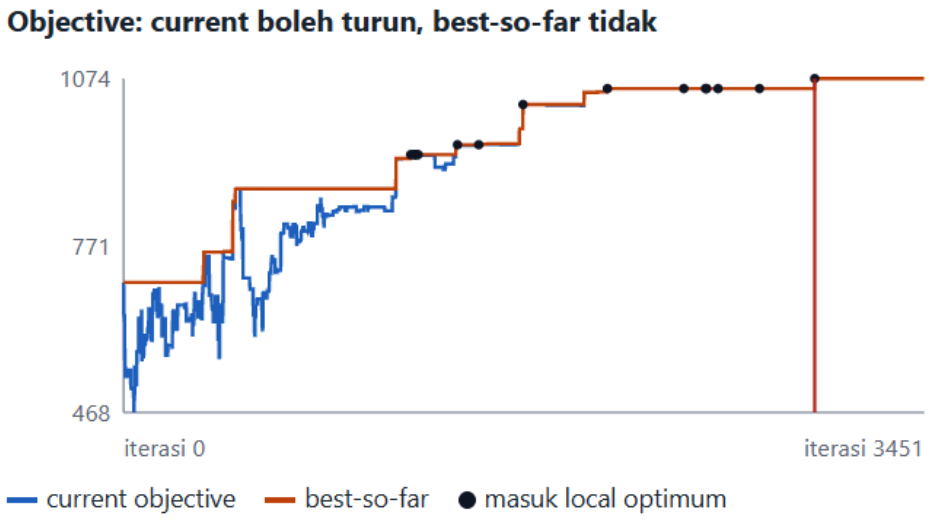
Kenapa state ini?

Best-found SA (iterasi 2980).

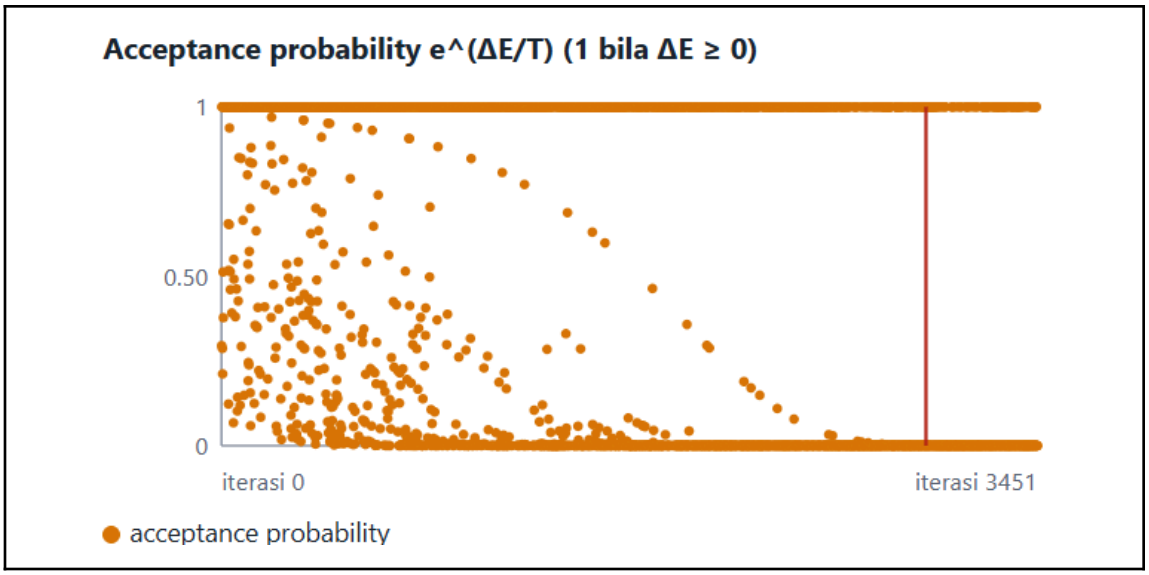
Keterangan	**Hasil**
Seed	4002

Objective function awal	704
Best-found objective	1074
Iterasi penemuan best-found	2980
Peningkatan nilai objektif	370
Persentase peningkatan	52,56%
Banyak iterasi	3451
Durasi searching	0,384 detik
Frekuensi stuck di local optimum	15 episode
Jumlah keluar dari local optimum	14
Kondisi berhenti	Temperatur di bawah min_temperature

Plot

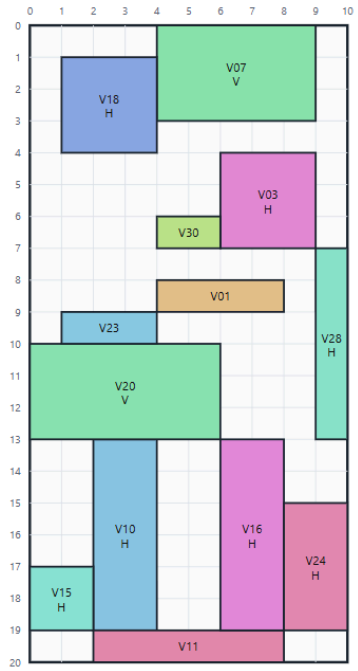
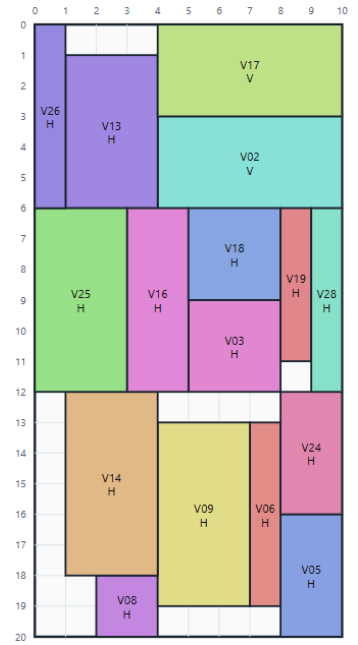


Plot $e^{\frac{\Delta E}{T}}$ **terhadap banyak iterasi yang telah dilewati**



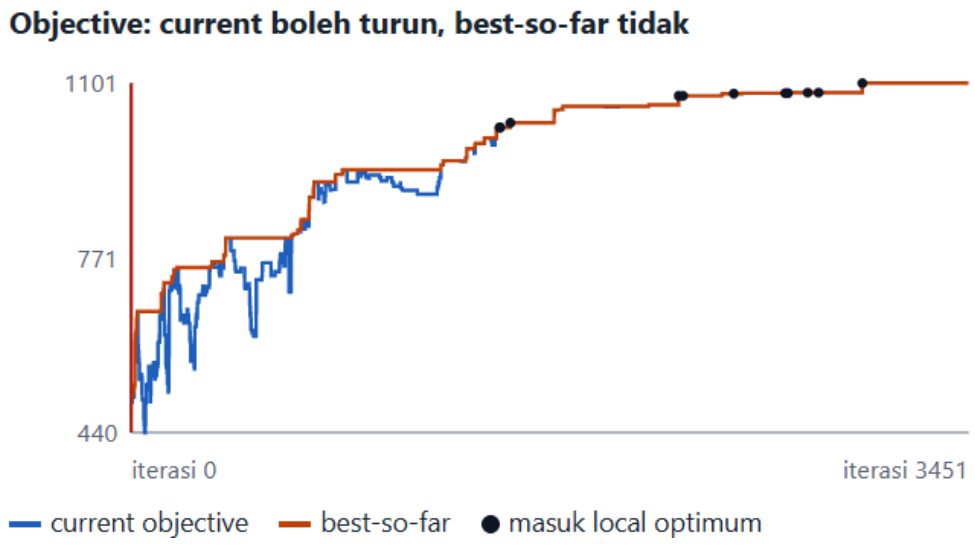
Eksperimen kedua dimulai dengan nilai objektif 704, tertinggi di antara ketiga initial state. Nilainya meningkat menjadi 1074, dengan kenaikan sebesar 370 atau 52,56%. Nilai terbaik ditemukan pada iterasi ke-2980, lebih awal dibandingkan dua eksperimen lainnya. Eksperimen ini mencatat frekuensi stuck tertinggi, yaitu 15 episode, dengan 14 kali keluar dari local optimum. Meskipun lebih sering mengalami kondisi tersebut, nilai akhirnya tetap lebih tinggi daripada eksperimen pertama. Dengan demikian, jumlah episode stuck yang lebih banyak tidak selalu menunjukkan hasil yang lebih buruk.

Eksperimen 3 - *Simulated Annealing*

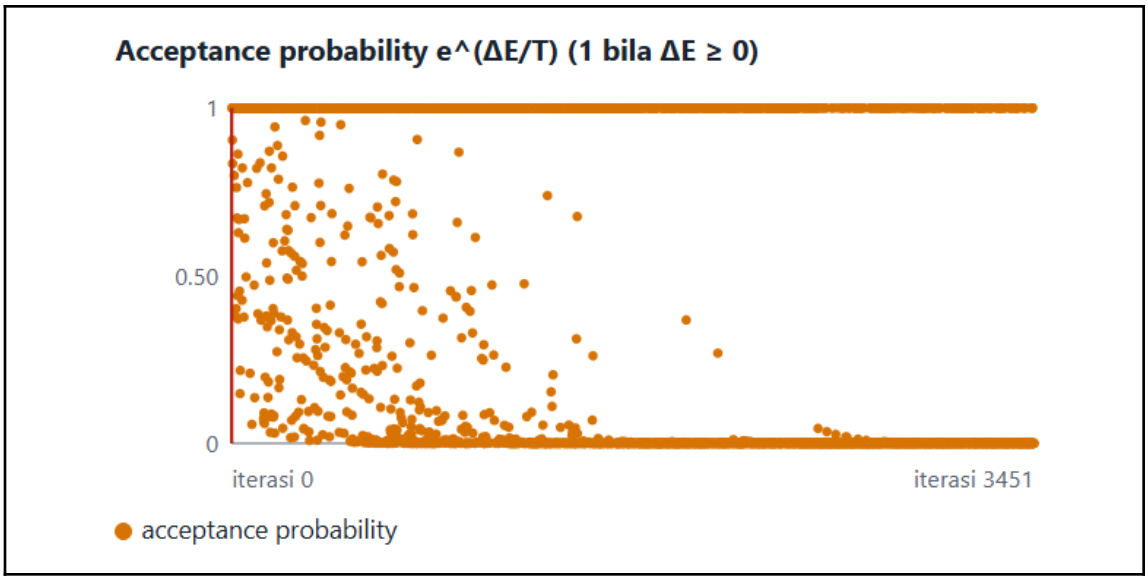
***State* Awal**	
	Objective (ShippingFee) **501** Berat / MaxCapacity **500 / 500** Kendaraan dimuat **13 / 30** Utilisasi area deck **54.0%** Total potensi fee **1510** Feasible (backend) **ya** **Kenapa state ini?** Initial: state awal random.
***State* Akhir**	
	Objective (ShippingFee) **1101** Berat / MaxCapacity **491 / 500** Kendaraan dimuat **16 / 30** Utilisasi area deck **89.0%** Total potensi fee **1510** Feasible (backend) **ya** **Kenapa state ini?** Best-found SA (iterasi 3014).

Keterangan	Hasil
Seed	4003
Objective function awal	501
Objective function akhir	1101
Best-found objective	1101
Iterasi penemuan best-found	3014
Peningkatan nilai objektif	600
Persentase peningkatan	119,76%
Banyak iterasi	3451
Durasi searching	2,543 detik
Frekuensi stuck di local optimum	11 episode
Jumlah keluar dari local optimum	10
Kondisi berhenti	Temperatur di bawah min_temperature

Plot



Plot $e^{\frac{\Delta E}{T}}$ **terhadap banyak iterasi yang telah dilewati**



Eksperimen ketiga menghasilkan nilai objektif akhir tertinggi, yaitu 1101, meskipun dimulai dari nilai awal terendah sebesar 501. Peningkatannya mencapai 600 atau 119,76%, sehingga total shipping fee akhirnya lebih dari dua kali nilai awal. Nilai terbaik ditemukan pada iterasi ke-3014. Tercatat 11 episode stuck dan 10 kali keluar dari local optimum. Solusi akhir memenuhi seluruh hard constraints dan memiliki nilai yang sama dengan best-found objective. Durasi searching sebesar 2,543 detik merupakan yang paling lama di antara ketiga eksperimen.

3.2.2 Analisis Nilai Objective Function, Iterasi, dan Durasi Simulated Annealing

Ketiga eksperimen meningkatkan total shipping fee dari nilai awal 638, 704, dan 501 menjadi 1053, 1074, dan 1101. Rata-rata nilai akhirnya mencapai 1076, dengan selisih tertinggi dan terendah sebesar 48. Eksperimen ketiga memperoleh hasil terbaik meskipun dimulai dari nilai awal terendah, dengan peningkatan sebesar 119,76%. Hasil ini menunjukkan bahwa kualitas initial state tidak langsung menentukan kualitas solusi akhir. Seluruh eksperimen berhenti setelah 3451 iterasi karena temperatur turun di bawah batas minimum 0,05. Nilai terbaik ditemukan pada iterasi ke-3175, 2980, dan 3014, sehingga perbaikan masih diperoleh pada tahap temperatur rendah. Nilai objektif akhir ketiganya sama dengan best-found objective, yang berarti setiap run berakhir dengan nilai terbaik yang pernah ditemukan selama searching.

Jumlah episode stuck masing-masing adalah 8, 15, dan 11, dengan jumlah escape sebesar 7, 14, dan 10. Eksperimen kedua paling sering mengalami stuck, tetapi hasil akhirnya tetap lebih tinggi daripada eksperimen pertama. Frekuensi stuck saja belum cukup untuk menilai performa algoritma. Perpindahan bernilai sama maupun lebih rendah memungkinkan SA meninggalkan local optimum dan melanjutkan eksplorasi, meskipun hasilnya belum menjamin optimum global. Durasi searching tercatat sebesar 0,352, 0,384, dan 2,543 detik. Eksperimen ketiga membutuhkan waktu lebih lama meskipun jumlah iterasinya sama; penyebab pastinya belum dapat ditentukan dari hasil ini. Pengukuran tersebut mencakup inisialisasi dan searching, sedangkan pemeriksaan stuck, penyimpanan hasil, dan visualisasi dilakukan di luar timer. Secara keseluruhan, ketiga run menghasilkan solusi feasible dengan nilai akhir yang relatif berdekatan, sementara hasil tertinggi diperoleh pada seed 4003.

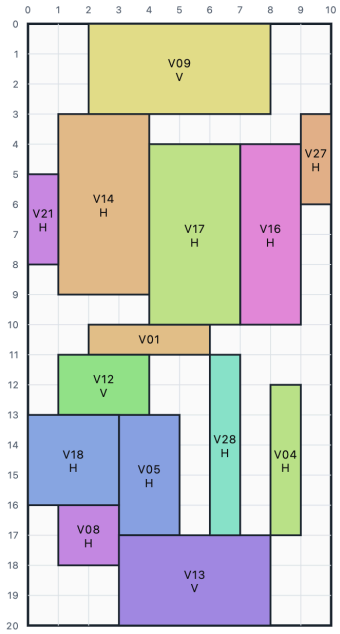
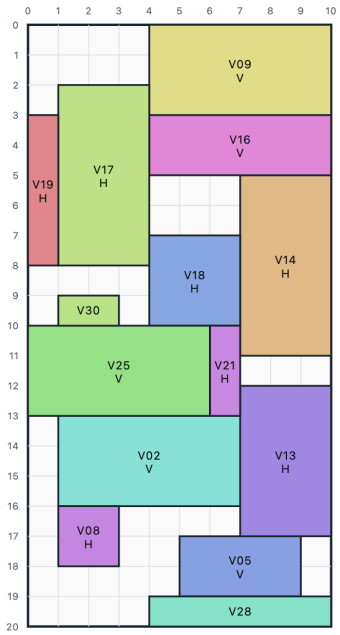
3.3 Hasil Eksperimen Genetic Algorithm

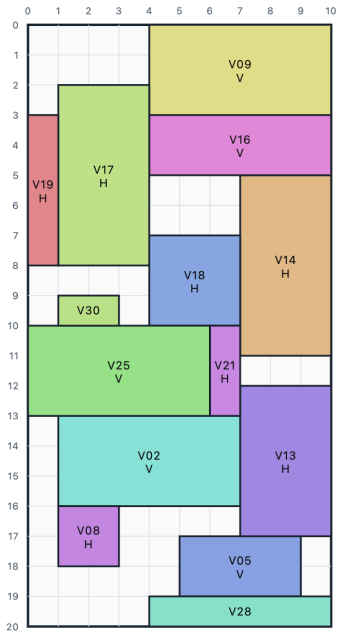
3.3.1 Variasi Banyak Iterasi

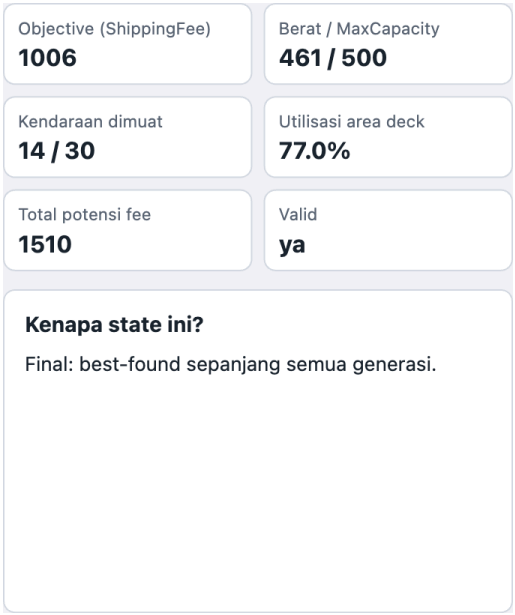
Skenario 1 menetapkan populasi 50 individu dan memvariasikan banyak iterasi menjadi 25, 100, dan 400 generasi. Seluruh 9 run menghasilkan solusi akhir yang *feasible*. Hasil tiap run diuraikan pada sembilan eksperimen berikut, kemudian dirangkum per konfigurasi.

Pada setiap gambar di bawah, panel kiri adalah individu terbaik pada populasi awal, panel tengah adalah individu terbaik hasil pencarian (best-found), dan panel kanan adalah kurva *fitness* maksimum serta rata-rata terhadap generasi. Kendaraan yang tidak dimuat dicantumkan di bawah masing-masing panel.

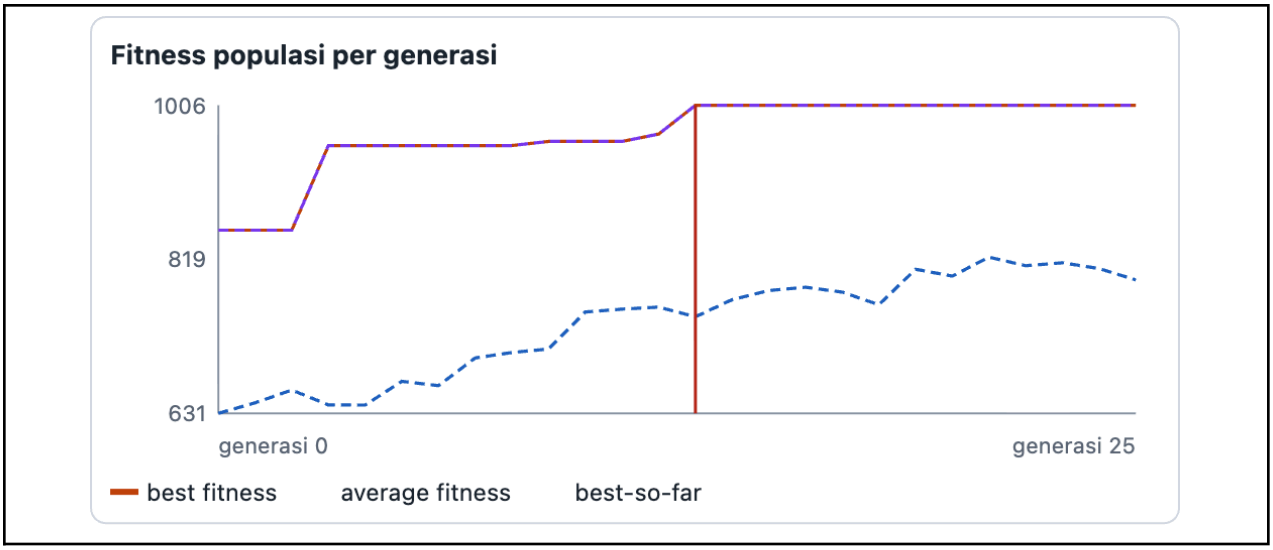
Eksperimen 1 - *Genetic Algorithm* (populasi 50, 25 generasi)

***State* Awal**	
	Objective (ShippingFee) **854** Berat / MaxCapacity **492 / 500** Kendaraan dimuat **14 / 30** Utilisasi area deck **64.5%** Total potensi fee **1510** Valid **ya** **Kenapa state ini?** Generasi 0: best fitness 854, rata-rata 631.1, terburuk 418. Best-so-far sejauh ini: 854. Individu unik: 50 dari 50. Deck menampilkan individu terbaik pada generasi ini (bukan seluruh populasi dan bukan 'current state').
***State* Best-found (generasi 13)**	
	Objective (ShippingFee) **1006** Berat / MaxCapacity **461 / 500** Kendaraan dimuat **14 / 30** Utilisasi area deck **77.0%** Total potensi fee **1510** Valid **ya** **Kenapa state ini?** Generasi 13: best fitness 1006, rata-rata 748.6, terburuk 510. Best-so-far sejauh ini: 1006. Individu unik: 50 dari 50. Deck menampilkan individu terbaik pada generasi ini (bukan seluruh populasi dan bukan 'current state').
***State* Akhir**	



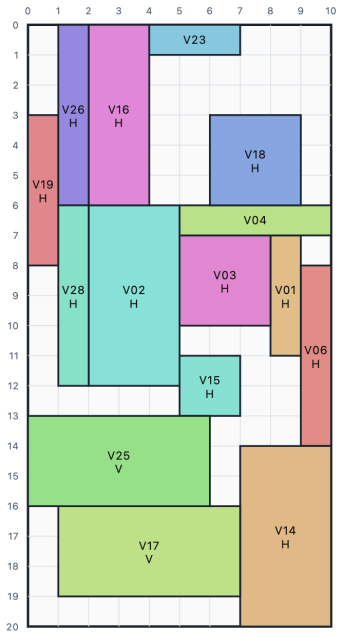
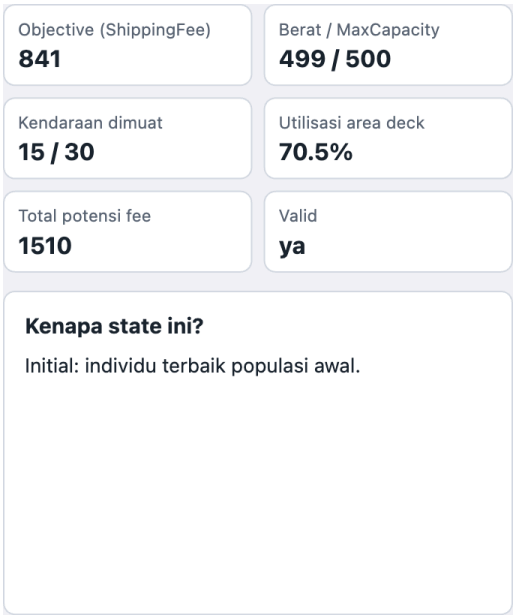


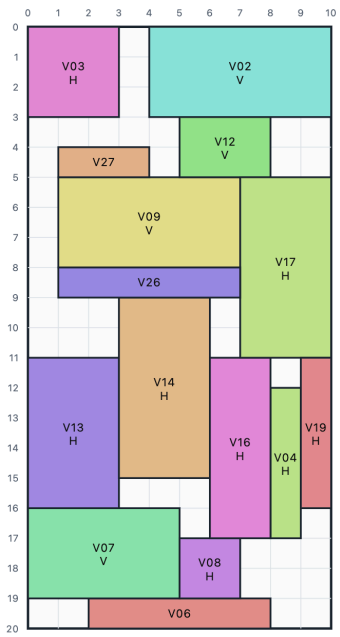
Keterangan	Hasil
Seed	5001
Objective function awal	854
Best-found objective	1006
Generasi penemuan best-found	13
Peningkatan nilai objektif	152 (17,80%)
Fitness rata-rata generasi akhir	793,5
Individu unik pada generasi akhir	48 dari 50
Child yang diperbaiki (*repair*)	849 dari 1.225 (69%)
Total berat dan kendaraan dimuat	461 dari 500, 14 dari 30 kendaraan
Durasi *searching*	0,322 detik
Kondisi berhenti	Banyak generasi tercapai
Plot	



Nilai objektif naik dari 854 menjadi 1006. Nilai terbaik ditemukan pada generasi 13, lalu tidak ada perbaikan sampai generasi 25. Dari 1.225 *child*, 849 harus diperbaiki, dan rata-rata *fitness* populasi akhir sebesar 793,5 masih jauh di bawah *fitness* maksimum.

Eksperimen 2 - *Genetic Algorithm* (populasi 50, 25 generasi)

***State* Awal**	
	
***State* Best-found (generasi 4)**	



Objective (ShippingFee) **996**

Berat / MaxCapacity **471 / 500**

Kendaraan dimuat **15 / 30**

Utilisasi area deck **79.0%**

Total potensi fee **1510**

Valid **ya**

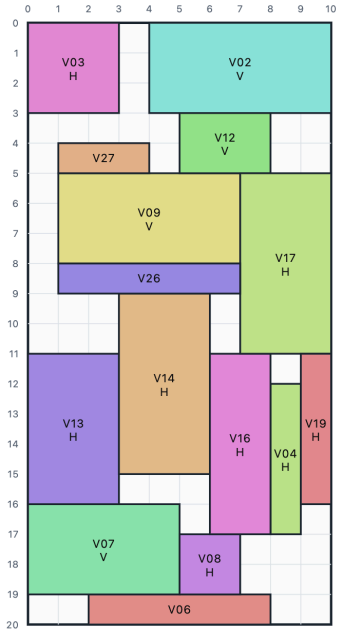
Kenapa state ini?

Generasi 4: best fitness 996, rata-rata 648.7, terburuk 413.

Best-so-far sejauh ini: 996. Individu unik: 50 dari 50.

Deck menampilkan individu terbaik pada generasi ini (bukan seluruh populasi dan bukan 'current state').

***State* Akhir**



Objective (ShippingFee) **996**

Berat / MaxCapacity **471 / 500**

Kendaraan dimuat **15 / 30**

Utilisasi area deck **79.0%**

Total potensi fee **1510**

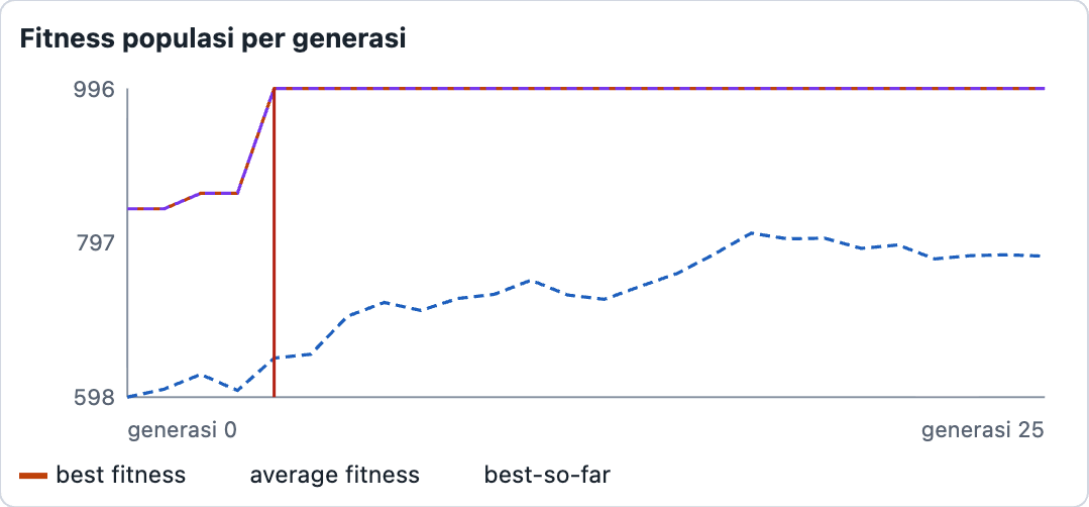
Valid **ya**

Kenapa state ini?

Final: best-found sepanjang semua generasi.

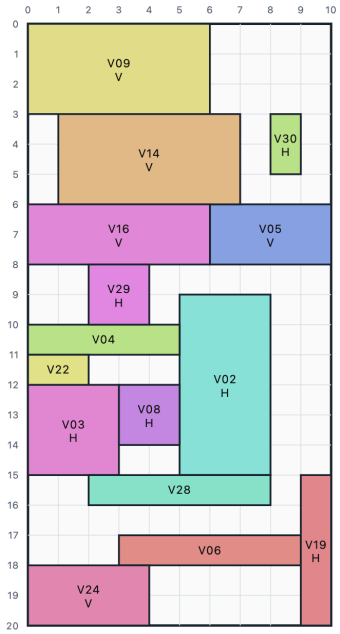
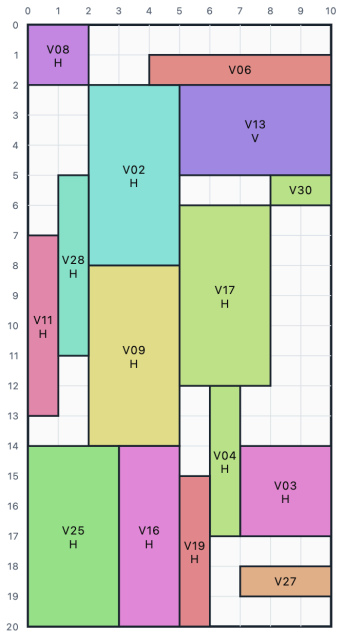
Keterangan	Hasil
Seed	5002
Objective function awal	841
Best-found objective	996
Generasi penemuan best-found	4
Peningkatan nilai objektif	155 (18,43%)

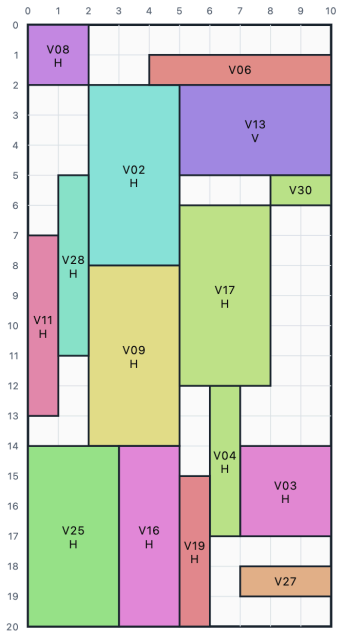
Fitness rata-rata generasi akhir	779,9
Individu unik pada generasi akhir	48 dari 50
Child yang diperbaiki (*repair*)	929 dari 1.225 (76%)
Total berat dan kendaraan dimuat	471 dari 500, 15 dari 30 kendaraan
Durasi *searching*	0,333 detik
Kondisi berhenti	Banyak generasi tercapai
Plot	

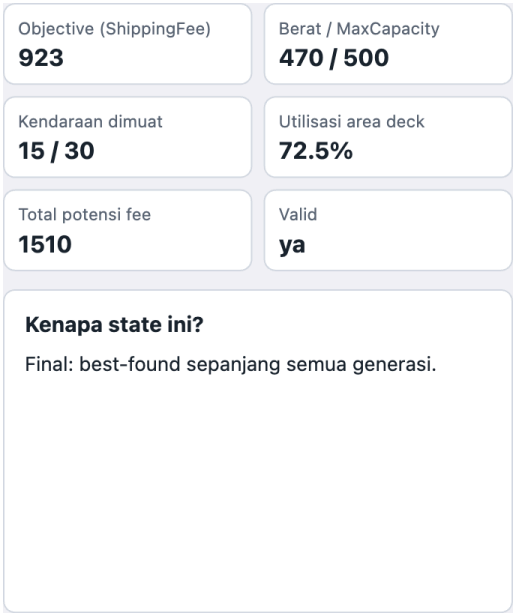


Run ini menemukan nilai terbaiknya paling awal di antara seluruh run, yaitu pada generasi 4, dan tidak membaik selama 21 generasi sisanya. Nilai akhirnya naik 155 dari nilai awal. Karena pencarian dihentikan pada generasi 25, data ini tidak menunjukkan apakah perbaikan akan muncul bila generasi ditambah.

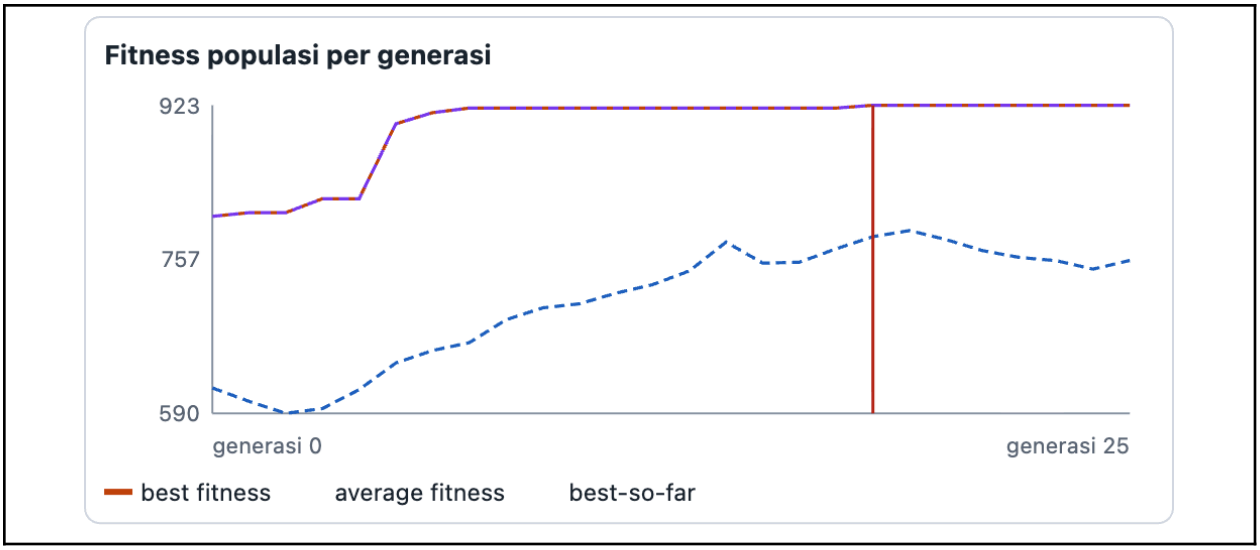
Eksperimen 3 - *Genetic Algorithm* (populasi 50, 25 generasi)

***State* Awal**	
	Objective (ShippingFee) **803** Berat / MaxCapacity **482 / 500** Kendaraan dimuat **15 / 30** Utilisasi area deck **62.5%** Total potensi fee **1510** Valid **ya** **Kenapa state ini?** Initial: individu terbaik populasi awal.
***State* Best-found (generasi 18)**	
	Objective (ShippingFee) **923** Berat / MaxCapacity **470 / 500** Kendaraan dimuat **15 / 30** Utilisasi area deck **72.5%** Total potensi fee **1510** Valid **ya** **Kenapa state ini?** Generasi 18: best fitness 923, rata-rata 781.1, terburuk 550. Best-so-far sejauh ini: 923. Individu unik: 48 dari 50. Deck menampilkan individu terbaik pada generasi ini (bukan seluruh populasi dan bukan 'current state').
***State* Akhir**	



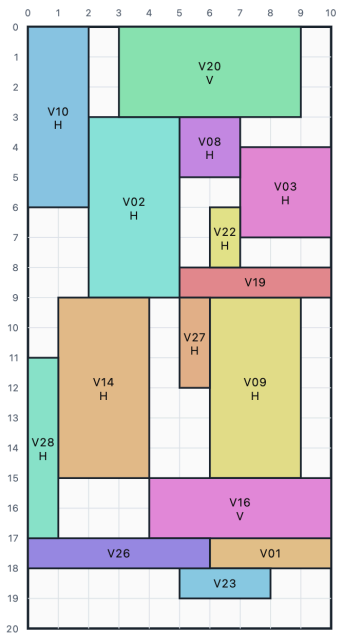
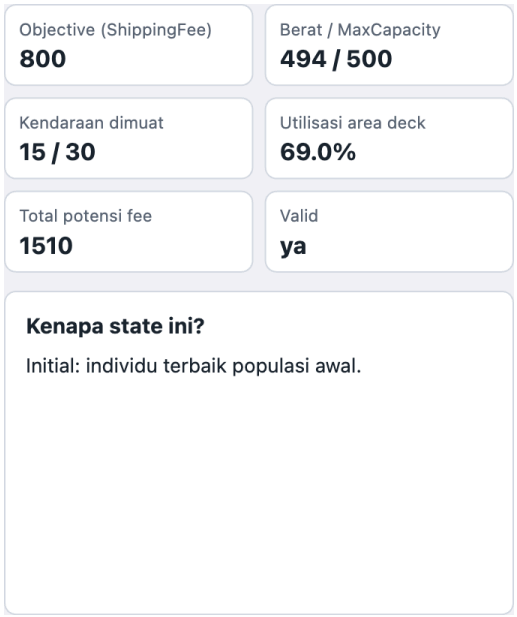


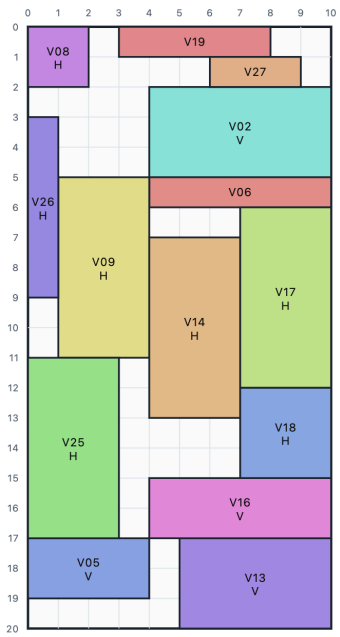
Keterangan	Hasil
Seed	5003
Objective function awal	803
Best-found objective	923
Generasi penemuan best-found	18
Peningkatan nilai objektif	120 (14,94%)
Fitness rata-rata generasi akhir	755,4
Individu unik pada generasi akhir	49 dari 50
Child yang diperbaiki (*repair*)	942 dari 1.225 (77%)
Total berat dan kendaraan dimuat	470 dari 500, 15 dari 30 kendaraan
Durasi *searching*	0,334 detik
Kondisi berhenti	Banyak generasi tercapai
Plot	



Eksperimen ketiga menghasilkan nilai akhir terendah dari seluruh 18 run, yaitu 923, atau 52 di bawah rata-rata konfigurasinya (975,0). Peningkatannya juga terkecil pada skenario ini, yaitu 14,94%. Rata-rata *fitness* populasi akhir sebesar 755,4 paling rendah di antara tiga run pada konfigurasi ini.

Eksperimen 4 - *Genetic Algorithm* (populasi 50, 100 generasi)

***State* Awal**	
	
***State* Best-found (generasi 97)**	



Objective (ShippingFee)	Berat / MaxCapacity
1025	**469 / 500**
Kendaraan dimuat	Utilisasi area deck
14 / 30	**79.0%**
Total potensi fee	Valid
1510	**ya**

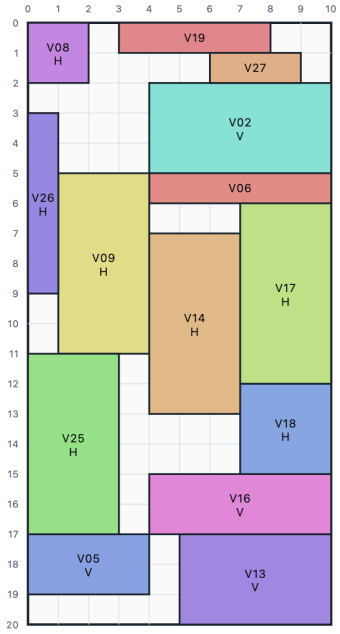
Kenapa state ini?

Generasi 97: best fitness 1025, rata-rata 897.4, terburuk 729.

Best-so-far sejauh ini: 1025. Individu unik: 48 dari 50.

Deck menampilkan individu terbaik pada generasi ini (bukan seluruh populasi dan bukan 'current state').

***State* Akhir**



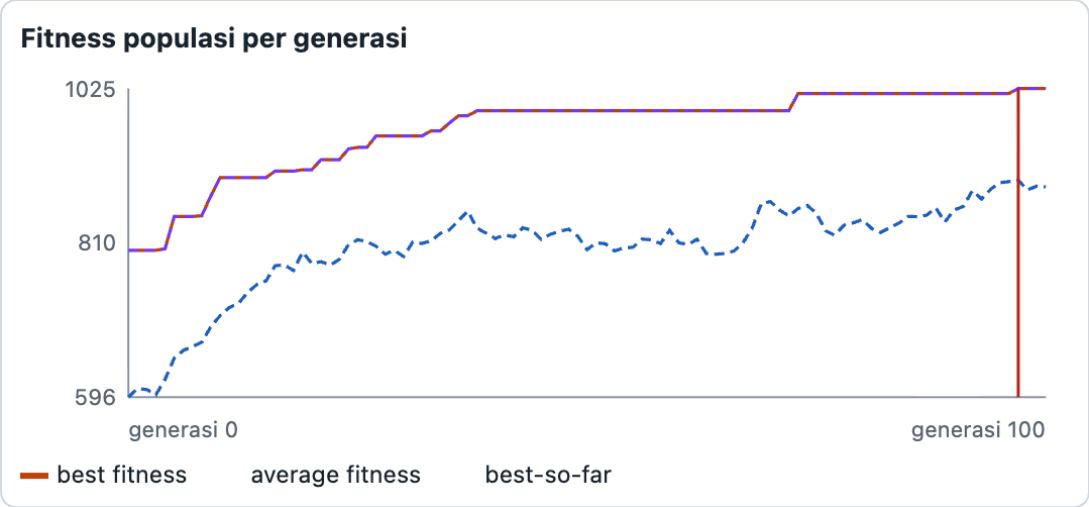
Objective (ShippingFee)	Berat / MaxCapacity
1025	**469 / 500**
Kendaraan dimuat	Utilisasi area deck
14 / 30	**79.0%**
Total potensi fee	Valid
1510	**ya**

Kenapa state ini?

Final: best-found sepanjang semua generasi.

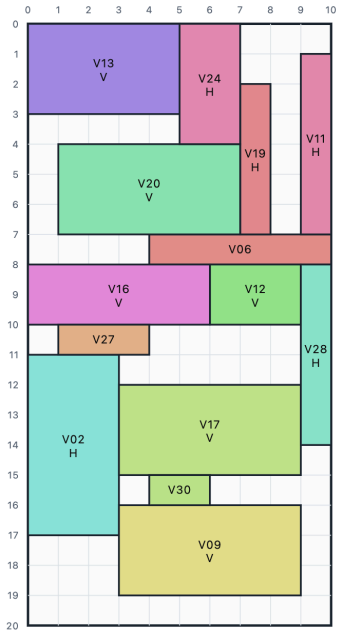
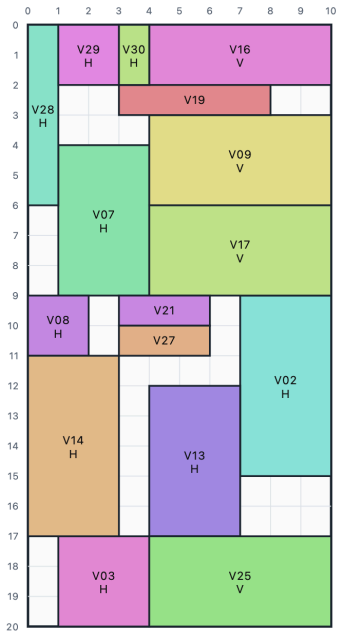
Keterangan	**Hasil**
Seed	5004
Objective function awal	800
Best-found objective	1025
Generasi penemuan best-found	97
Peningkatan nilai objektif	225 (28,13%)

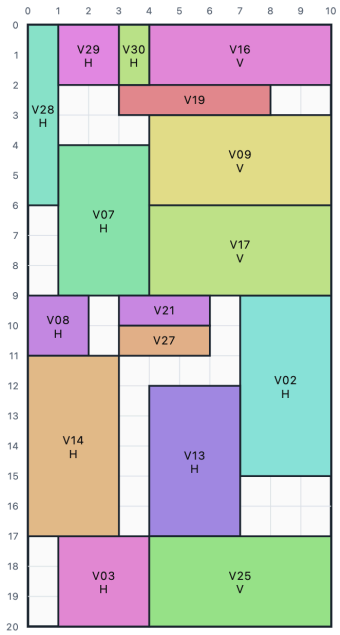
Fitness rata-rata generasi akhir	888,1
Individu unik pada generasi akhir	49 dari 50
Child yang diperbaiki (*repair*)	3.406 dari 4.900 (70%)
Total berat dan kendaraan dimuat	469 dari 500, 14 dari 30 kendaraan
Durasi *searching*	1,234 detik
Kondisi berhenti	Banyak generasi tercapai
Plot	

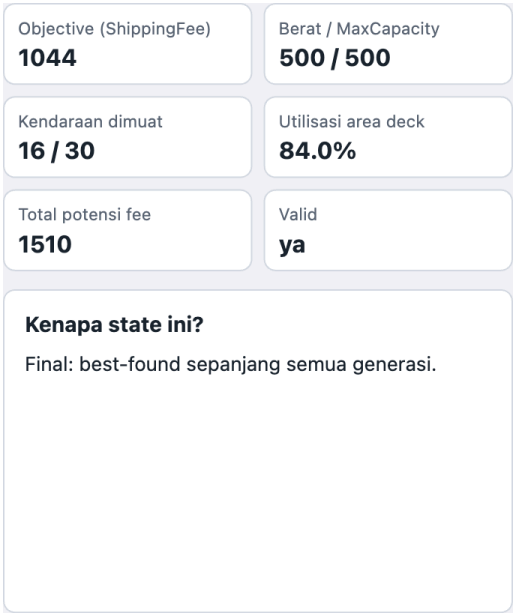


Nilai terbaik pada run ini baru muncul pada generasi 97 dari 100, sehingga pencarian masih menemukan perbaikan menjelang akhir dan belum jenuh ketika dihentikan. Rata-rata *fitness* populasi akhir sebesar 888,1 merupakan yang tertinggi pada konfigurasi ini.

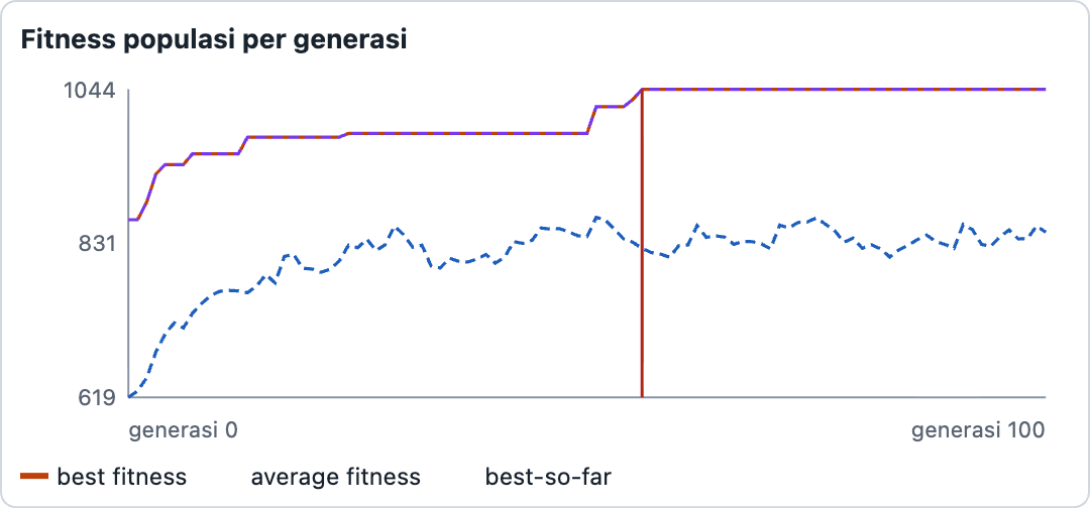
Eksperimen 5 - *Genetic Algorithm* (populasi 50, 100 generasi)

***State* Awal**	
	Objective (ShippingFee) **864** Berat / MaxCapacity **486 / 500** Kendaraan dimuat **14 / 30** Utilisasi area deck **70.5%** Total potensi fee **1510** Valid **ya** **Kenapa state ini?** Initial: individu terbaik populasi awal.
***State* Best-found (generasi 56)**	
	Objective (ShippingFee) **1044** Berat / MaxCapacity **500 / 500** Kendaraan dimuat **16 / 30** Utilisasi area deck **84.0%** Total potensi fee **1510** Valid **ya** **Kenapa state ini?** Generasi 56: best fitness 1044, rata-rata 824.3, terburuk 653. Best-so-far sejauh ini: 1044. Individu unik: 50 dari 50. Deck menampilkan individu terbaik pada generasi ini (bukan seluruh populasi dan bukan 'current state').
***State* Akhir**	



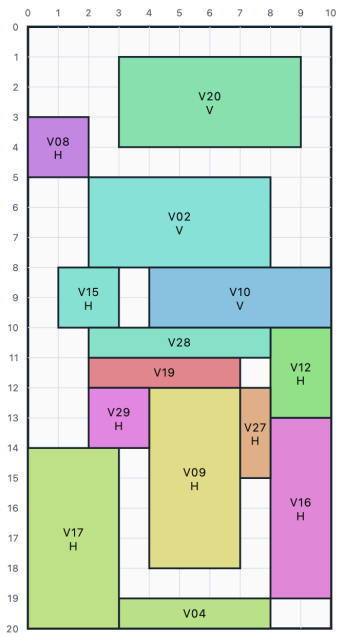
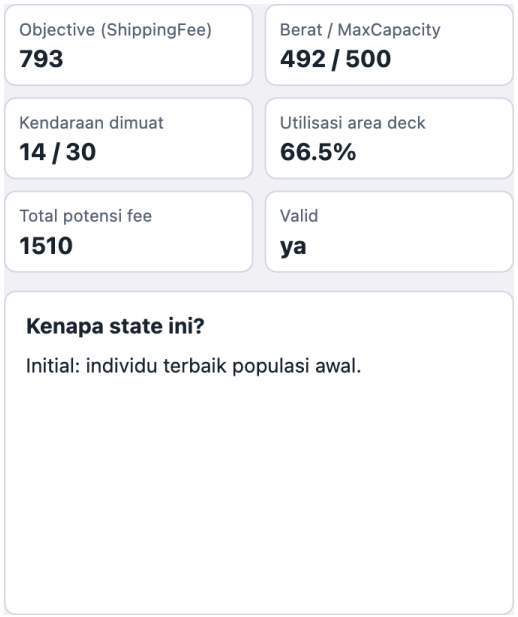


Keterangan	Hasil
Seed	5005
Objective function awal	864
Best-found objective	1044
Generasi penemuan best-found	56
Peningkatan nilai objektif	180 (20,83%)
Fitness rata-rata generasi akhir	846,7
Individu unik pada generasi akhir	49 dari 50
Child yang diperbaiki (*repair*)	3.791 dari 4.900 (77%)
Total berat dan kendaraan dimuat	500 dari 500, 16 dari 30 kendaraan
Durasi *searching*	1,377 detik
Kondisi berhenti	Banyak generasi tercapai
Plot	



Eksperimen kelima menghasilkan nilai tertinggi pada konfigurasi ini, yaitu 1044, dengan peningkatan 20,83% karena nilai awalnya sudah paling tinggi (864). Total berat solusi tepat 500, sehingga kapasitas berat kapal terpakai penuh. Nilai terbaik ditemukan pada generasi 56 dan tidak membaik selama 44 generasi berikutnya.

Eksperimen 6 - *Genetic Algorithm* (populasi 50, 100 generasi)

***State* Awal**	
	
***State* Best-found (generasi 43)**	



Objective (ShippingFee) **1035**	Berat / MaxCapacity **479 / 500**
Kendaraan dimuat **16 / 30**	Utilisasi area deck **85.0%**
Total potensi fee **1510**	Valid **ya**

Kenapa state ini?

Generasi 43: best fitness 1035, rata-rata 859.0, terburuk 729.

Best-so-far sejauh ini: 1035. Individu unik: 50 dari 50.

Deck menampilkan individu terbaik pada generasi ini (bukan seluruh populasi dan bukan 'current state').

***State* Akhir**



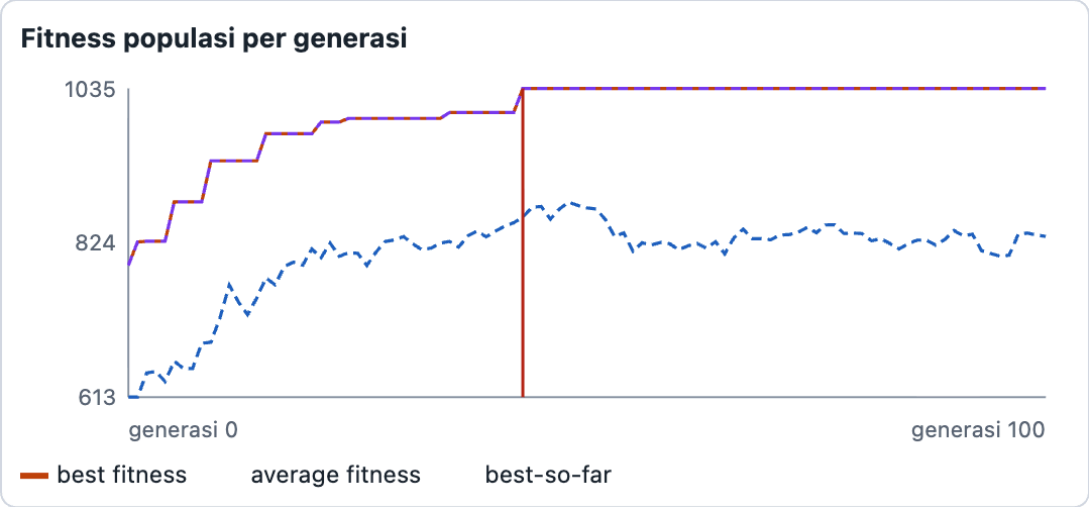
Objective (ShippingFee) **1035**	Berat / MaxCapacity **479 / 500**
Kendaraan dimuat **16 / 30**	Utilisasi area deck **85.0%**
Total potensi fee **1510**	Valid **ya**

Kenapa state ini?

Final: best-found sepanjang semua generasi.

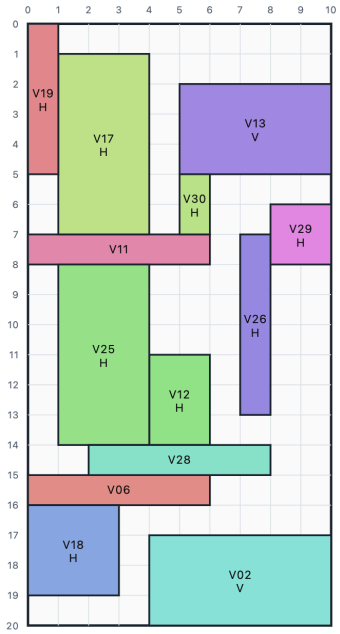
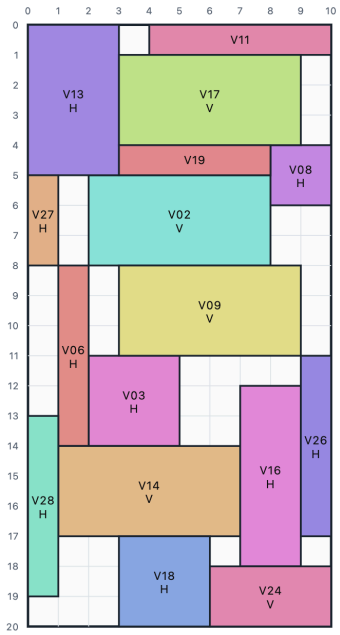
Keterangan	**Hasil**
Seed	5006
Objective function awal	793
Best-found objective	1035
Generasi penemuan best-found	43
Peningkatan nilai objektif	242 (30,52%)

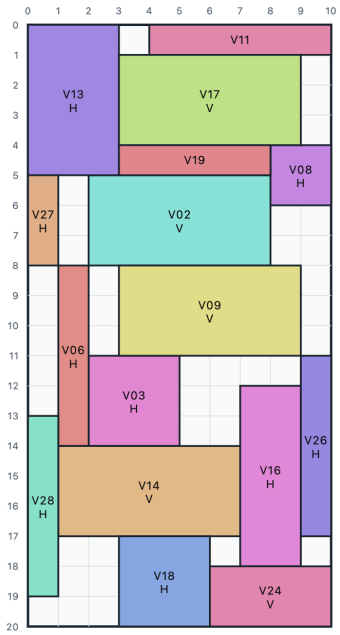
Fitness rata-rata generasi akhir	832,7
Individu unik pada generasi akhir	49 dari 50
Child yang diperbaiki (*repair*)	3.427 dari 4.900 (70%)
Total berat dan kendaraan dimuat	479 dari 500, 16 dari 30 kendaraan
Durasi *searching*	1,238 detik
Kondisi berhenti	Banyak generasi tercapai
Plot	

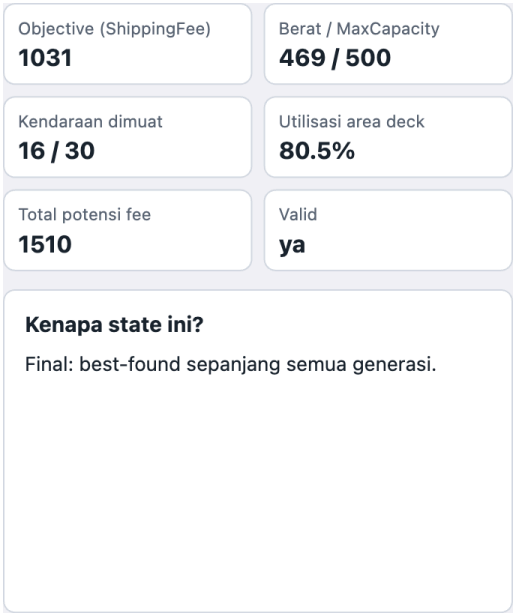


Nilai awal terendah pada konfigurasi ini (793) naik 242 menjadi 1035, atau 30,52%, yang merupakan peningkatan persentase terbesar pada konfigurasi tersebut. Nilai terbaik ditemukan pada generasi 43, dan 57 generasi sesudahnya tidak menghasilkan nilai yang lebih tinggi.

Eksperimen 7 - *Genetic Algorithm* (populasi 50, 400 generasi)

***State* Awal**	
	Objective (ShippingFee) **732** Berat / MaxCapacity **489 / 500** Kendaraan dimuat **13 / 30** Utilisasi area deck **59.5%** Total potensi fee **1510** Valid **ya** **Kenapa state ini?** Initial: individu terbaik populasi awal.
***State* Best-found (generasi 306)**	
	Objective (ShippingFee) **1031** Berat / MaxCapacity **469 / 500** Kendaraan dimuat **16 / 30** Utilisasi area deck **80.5%** Total potensi fee **1510** Valid **ya** **Kenapa state ini?** Generasi 306: best fitness 1031, rata-rata 806.8, terburuk 633. Best-so-far sejauh ini: 1031. Individu unik: 49 dari 50. Deck menampilkan individu terbaik pada generasi ini (bukan seluruh populasi dan bukan 'current state').
***State* Akhir**	



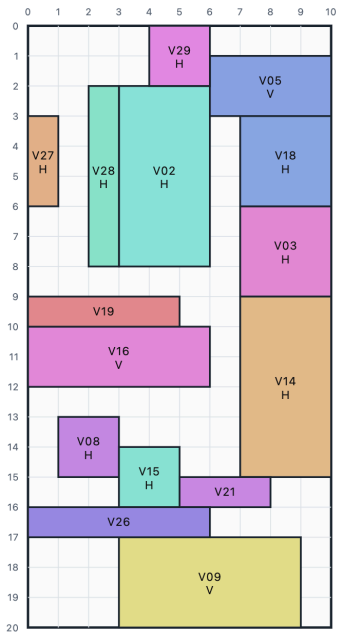


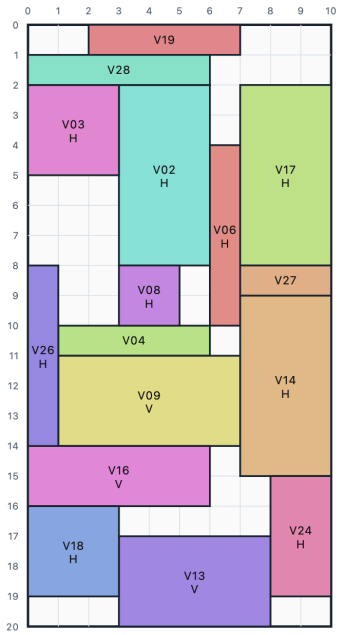
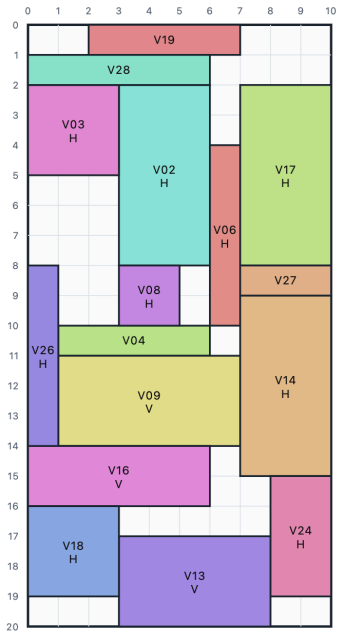
Keterangan	Hasil
Seed	5007
Objective function awal	732
Best-found objective	1031
Generasi penemuan best-found	306
Peningkatan nilai objektif	299 (40,85%)
Fitness rata-rata generasi akhir	799,0
Individu unik pada generasi akhir	48 dari 50
Child yang diperbaiki (*repair*)	12.877 dari 19.600 (66%)
Total berat dan kendaraan dimuat	469 dari 500, 16 dari 30 kendaraan
Durasi *searching*	4,563 detik
Kondisi berhenti	Banyak generasi tercapai
Plot	



Nilai awal run ini (732) adalah yang terendah di skenario 1, sehingga peningkatannya mencapai 299 atau 40,85%, tertinggi pada skenario tersebut. Nilai terbaik ditemukan pada generasi 306, jauh di atas generasi penemuan terbaik pada konfigurasi 100 generasi, yang berarti penambahan generasi masih memberi sedikit perbaikan pada bagian akhir.

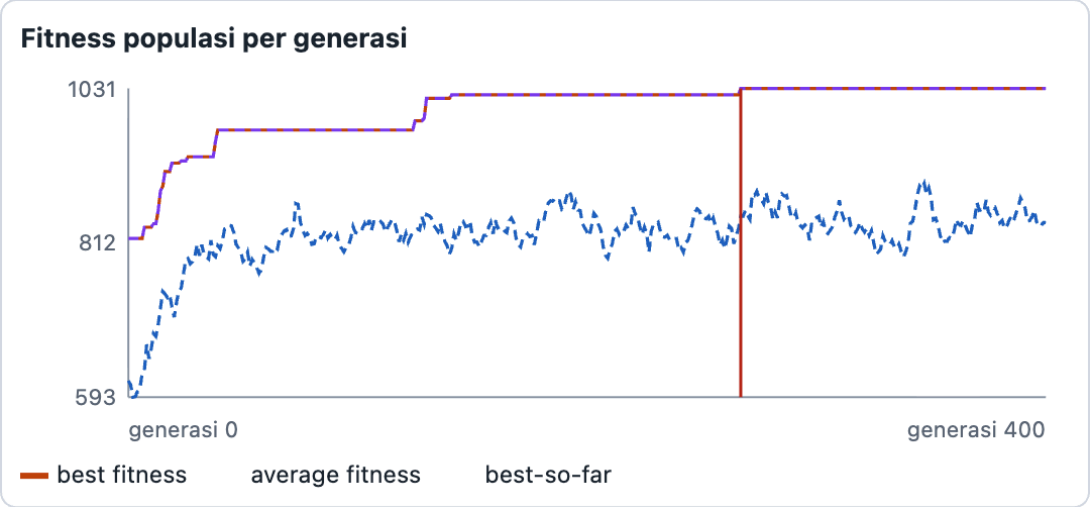
Eksperimen 8 - *Genetic Algorithm* (populasi 50, 400 generasi)

***State* Awal**	
	Objective (ShippingFee) **818** Berat / MaxCapacity **474 / 500** Kendaraan dimuat **15 / 30** Utilisasi area deck **63.5%** Total potensi fee **1510** Valid **ya** **Kenapa state ini?** Initial: individu terbaik populasi awal.

State Best-found (generasi 267)	
	Objective (ShippingFee) **1031** Berat / MaxCapacity **464 / 500** Kendaraan dimuat **16 / 30** Utilisasi area deck **80.0%** Total potensi fee **1510** Valid **ya** **Kenapa state ini?** Generasi 267: best fitness 1031, rata-rata 849.5, terburuk 592. Best-so-far sejauh ini: 1031. Individu unik: 48 dari 50. Deck menampilkan individu terbaik pada generasi ini (bukan seluruh populasi dan bukan 'current state').
***State* Akhir**	
	Objective (ShippingFee) **1031** Berat / MaxCapacity **464 / 500** Kendaraan dimuat **16 / 30** Utilisasi area deck **80.0%** Total potensi fee **1510** Valid **ya** **Kenapa state ini?** Final: best-found sepanjang semua generasi.

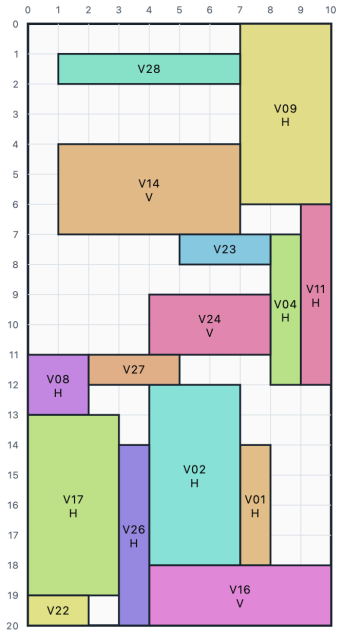
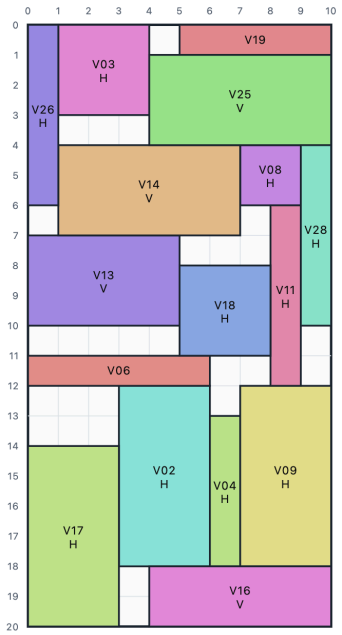
Keterangan	Hasil
Seed	5008
Objective function awal	818
Best-found objective	1031
Generasi penemuan best-found	267

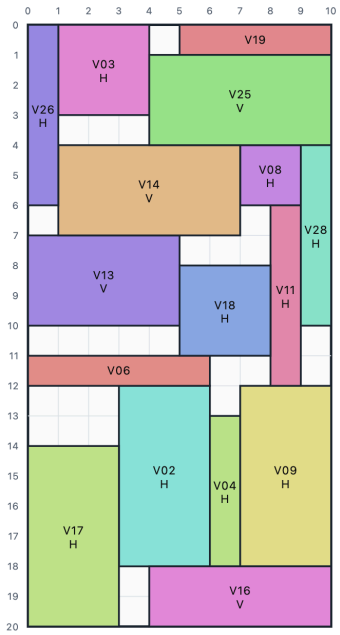
Peningkatan nilai objektif	213 (26,04%)
Fitness rata-rata generasi akhir	840,5
Individu unik pada generasi akhir	50 dari 50
Child yang diperbaiki (*repair*)	13.834 dari 19.600 (71%)
Total berat dan kendaraan dimuat	464 dari 500, 16 dari 30 kendaraan
Durasi *searching*	4,966 detik
Kondisi berhenti	Banyak generasi tercapai
Plot	

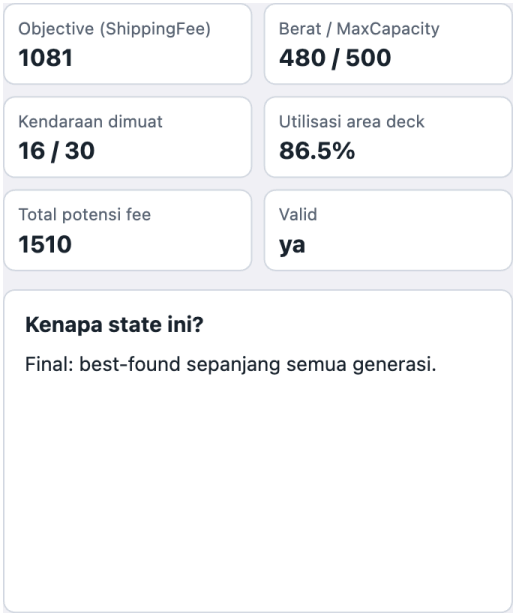


Nilai akhir run ini sama dengan Eksperimen 7, yaitu 1031, tetapi susunannya berbeda (total berat 464 berbanding 469) dan ditemukan lebih awal, pada generasi 267. Dua *seed* yang berbeda mencapai nilai yang sama dengan susunan berbeda. Dengan hanya tiga run, hal ini belum cukup untuk menyimpulkan bahwa 1031 adalah batas yang dapat dijangkau konfigurasi ini.

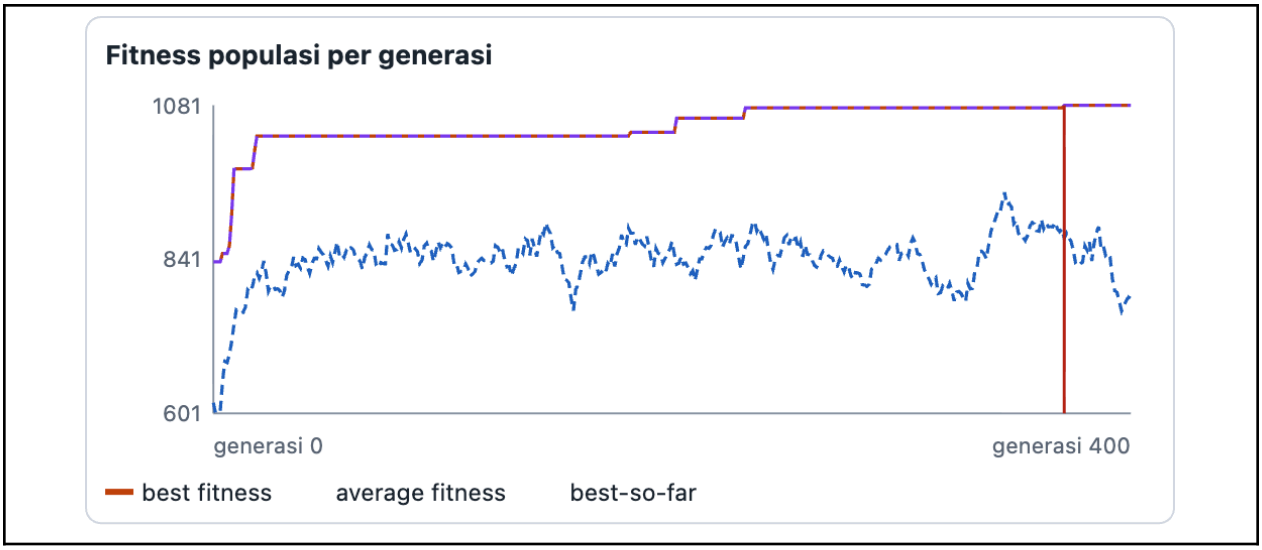
Eksperimen 9 - *Genetic Algorithm* (populasi 50, 400 generasi)

***State* Awal**	
	Objective (ShippingFee) **837** Berat / MaxCapacity **499 / 500** Kendaraan dimuat **15 / 30** Utilisasi area deck **65.5%** Total potensi fee **1510** Valid **ya** **Kenapa state ini?** Initial: individu terbaik populasi awal.
***State* Best-found (generasi 371)**	
	Objective (ShippingFee) **1081** Berat / MaxCapacity **480 / 500** Kendaraan dimuat **16 / 30** Utilisasi area deck **86.5%** Total potensi fee **1510** Valid **ya** **Kenapa state ini?** Generasi 371: best fitness 1081, rata-rata 873.5, terburuk 701. Best-so-far sejauh ini: 1081. Individu unik: 50 dari 50. Deck menampilkan individu terbaik pada generasi ini (bukan seluruh populasi dan bukan 'current state').
***State* Akhir**	





Keterangan	Hasil
Seed	5009
Objective function awal	837
Best-found objective	1081
Generasi penemuan best-found	371
Peningkatan nilai objektif	244 (29,15%)
Fitness rata-rata generasi akhir	779,2
Individu unik pada generasi akhir	47 dari 50
Child yang diperbaiki (*repair*)	13.666 dari 19.600 (70%)
Total berat dan kendaraan dimuat	480 dari 500, 16 dari 30 kendaraan
Durasi *searching*	4,823 detik
Kondisi berhenti	Banyak generasi tercapai
Plot	

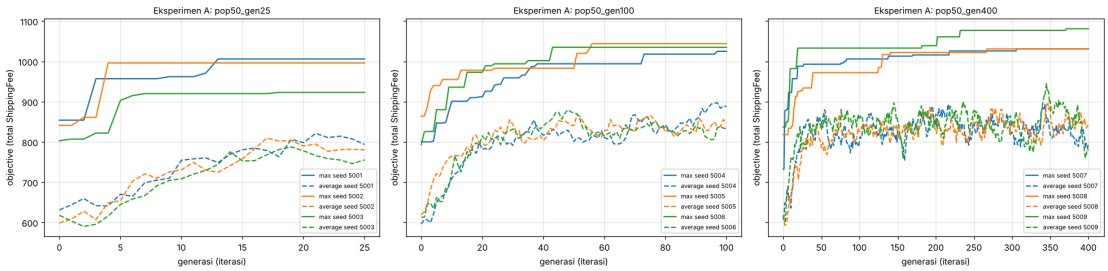


Eksperimen kesembilan menghasilkan nilai tertinggi pada skenario 1 dan tertinggi pada seluruh 18 run, yaitu 1081. Nilai ini ditemukan pada generasi 371 dan melampaui dua run lain pada konfigurasi yang sama sebesar 50. Rata-rata *fitness* populasi akhir pada run ini justru yang terendah pada konfigurasinya (779,2), sehingga nilai akhir yang tinggi tidak disertai populasi yang lebih baik secara keseluruhan.

Ringkasan per konfigurasi dihitung dari tiga run masing-masing.

Tabel Ringkasan Variasi Banyak Iterasi

Generasi	Rata-rata objektif akhir	Simpangan baku	Terendah sampai tertinggi	Rata-rata generasi penemuan terbaik	Rata-rata durasi (detik)
25	975,0	45,3	923 sampai 1006	11,7	0,330
100	1034,7	9,5	1025 sampai 1044	65,3	1,283
400	1047,7	28,9	1031 sampai 1081	314,7	4,784



Gambar. Fitness maksimum (garis tebal) dan rata-rata (garis putus-putus) terhadap generasi pada skenario 1.

Rata-rata objektif akhir naik dari 975,0 pada 25 generasi menjadi 1034,7 pada 100 generasi, atau bertambah 59,7. Menaikkan generasi dari 100 menjadi 400 hanya menambah 13,0, padahal evaluasi *fitness* bertambah empat kali lipat dan durasi naik sekitar 3,7 kali. Manfaat tambahan generasi karena itu menurun dengan cepat. Pada 25 generasi, ketiga run sudah menemukan nilai terbaiknya pada generasi 4 sampai 18, lalu tidak menemukan perbaikan lagi sampai batas 25 generasi tercapai. Pada 400 generasi, nilai terbaik justru baru ditemukan pada generasi 267 sampai 371, yang berarti pencarian masih menemukan perbaikan kecil pada bagian akhir dan belum benar-benar jenuh.

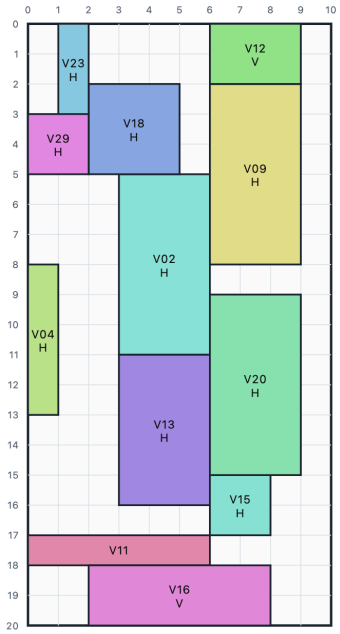
Kurva pada gambar memperlihatkan pola yang khas. *Fitness* maksimum naik cepat pada 20 sampai 30 generasi pertama, lalu naik dalam lompatan kecil yang jarang. *Fitness* rata-rata naik dari sekitar 600 menjadi 800 sampai 850 dalam 30 generasi pertama, kemudian bergerak naik turun pada kisaran itu sampai generasi terakhir. Nilai rata-rata akhir pada ketiga konfigurasi adalah 776, 856, dan 806, jadi menambah generasi tidak menaikkan kualitas populasi secara keseluruhan. Hasil terbaik pada skenario ini adalah 1081 pada *seed* 5009 dengan 400 generasi (Eksperimen 9), yang memuat 16 kendaraan dengan total berat 480 dari 500.

3.3.2 Variasi Jumlah Populasi

Skenario 2 menetapkan banyak iterasi sebesar 100 generasi dan memvariasikan jumlah populasi menjadi 10, 50, dan 200 individu. Seluruh 9 run menghasilkan solusi akhir yang *feasible*. Hasil tiap run diuraikan pada sembilan eksperimen berikut, kemudian dirangkum per konfigurasi. Panel pada tiap gambar sama seperti pada skenario 1, yaitu individu terbaik populasi awal, individu terbaik hasil pencarian, dan kurva *fitness*.

Eksperimen 1 - *Genetic Algorithm* (populasi 10, 100 generasi)

State Awal

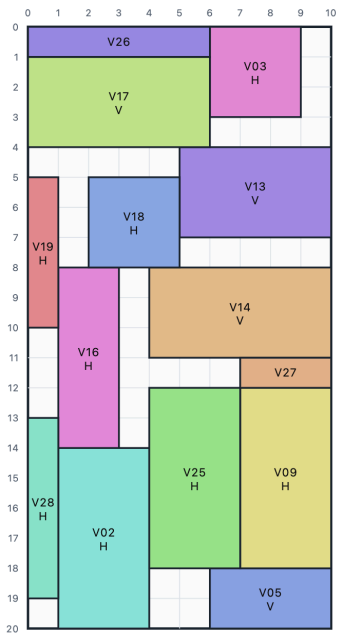


Objective (ShippingFee) **680**	Berat / MaxCapacity **496 / 500**
Kendaraan dimuat **12 / 30**	Utilisasi area deck **59.0%**
Total potensi fee **1510**	Valid **ya**

Kenapa state ini?

Initial: individu terbaik populasi awal.

State Best-found (generasi 74)



Objective (ShippingFee) **1019**	Berat / MaxCapacity **439 / 500**
Kendaraan dimuat **14 / 30**	Utilisasi area deck **81.5%**
Total potensi fee **1510**	Valid **ya**

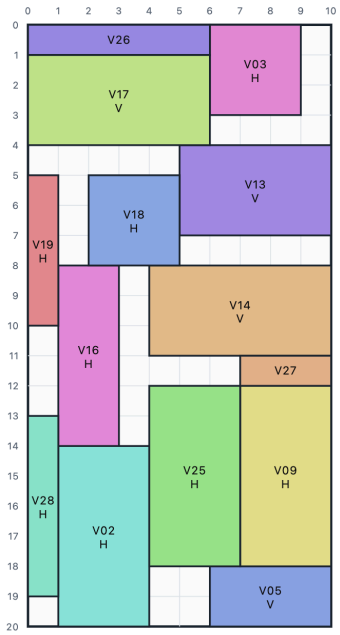
Kenapa state ini?

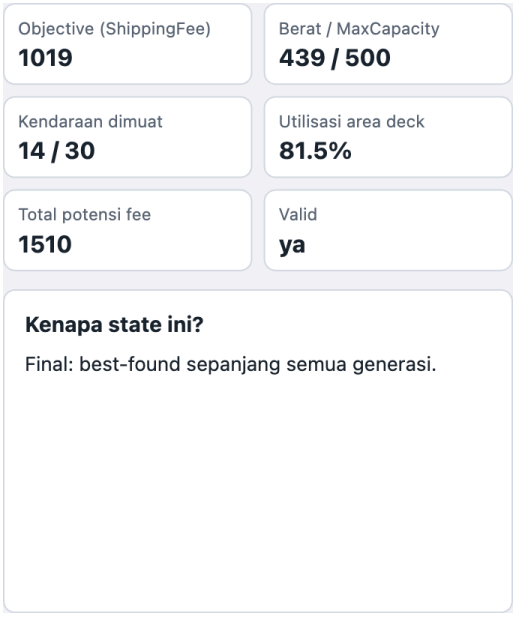
Generasi 74: best fitness 1019, rata-rata 936.2, terburuk 813.

Best-so-far sejauh ini: 1019. Individu unik: 8 dari 10.

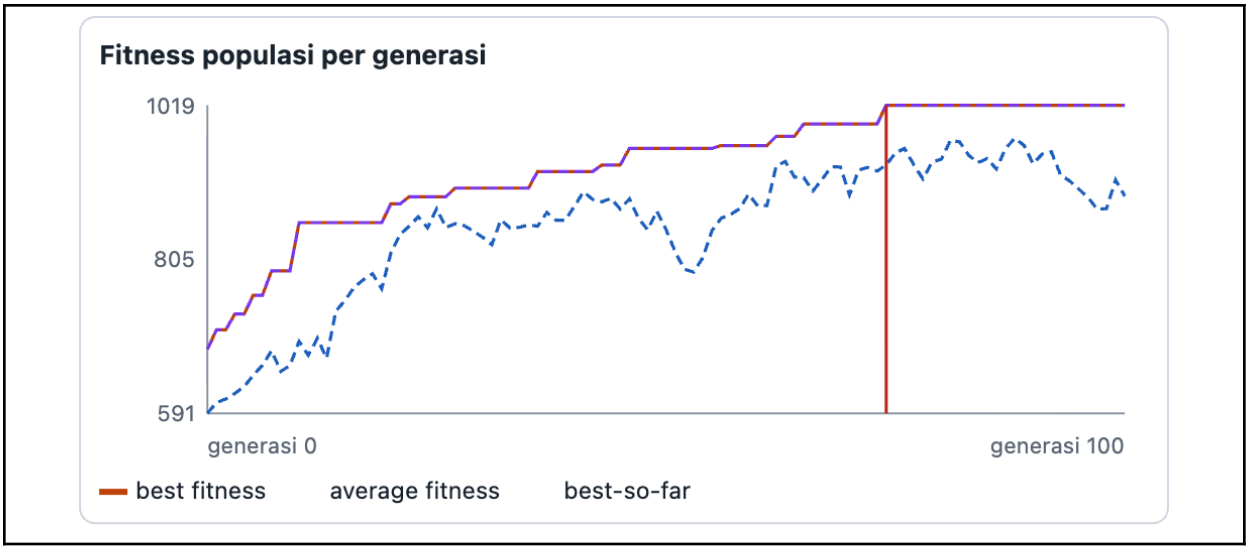
Deck menampilkan individu terbaik pada generasi ini (bukan seluruh populasi dan bukan 'current state').

State Akhir



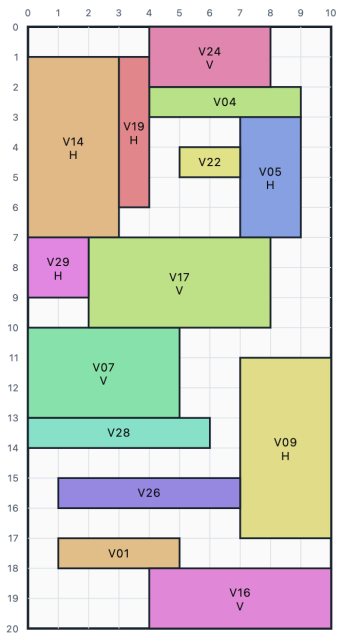


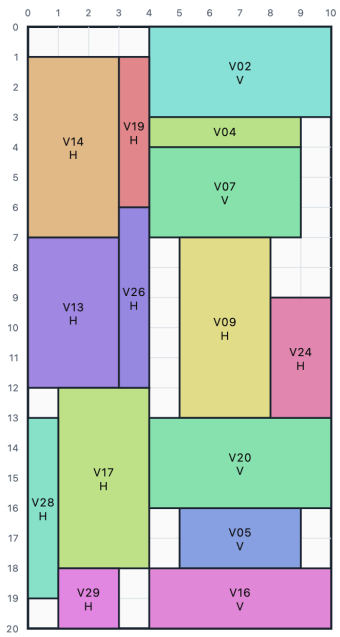
Keterangan	Hasil
Seed	5101
Objective function awal	680
Best-found objective	1019
Generasi penemuan best-found	74
Peningkatan nilai objektif	339 (49,85%)
Fitness rata-rata generasi akhir	892,8
Individu unik pada generasi akhir	7 dari 10
Child yang diperbaiki (*repair*)	396 dari 900 (44%)
Total berat dan kendaraan dimuat	439 dari 500, 14 dari 30 kendaraan
Durasi *searching*	0,148 detik
Kondisi berhenti	Banyak generasi tercapai
Plot	



Peningkatan sebesar 49,85% adalah yang terbesar dari seluruh 18 run, karena nilai awal 680 juga yang terendah. Pada generasi akhir hanya 7 dari 10 individu yang berbeda, dan rata-rata *fitness* sebesar 892,8 sudah dekat dengan nilai maksimum, sehingga populasi telah menyerupai satu susunan. Seluruh pencarian selesai dalam 0,148 detik.

Eksperimen 2 - *Genetic Algorithm* (populasi 10, 100 generasi)

***State* Awal**	
	Objective (ShippingFee) **775** Berat / MaxCapacity **486 / 500** Kendaraan dimuat **14 / 30** Utilisasi area deck **64.5%** Total potensi fee **1510** Valid **ya** **Kenapa state ini?** Initial: individu terbaik populasi awal.
***State* Best-found (generasi 56)**	



Objective (ShippingFee) **1025**	Berat / MaxCapacity **500 / 500**
Kendaraan dimuat **15 / 30**	Utilisasi area deck **87.0%**
Total potensi fee **1510**	Valid **ya**

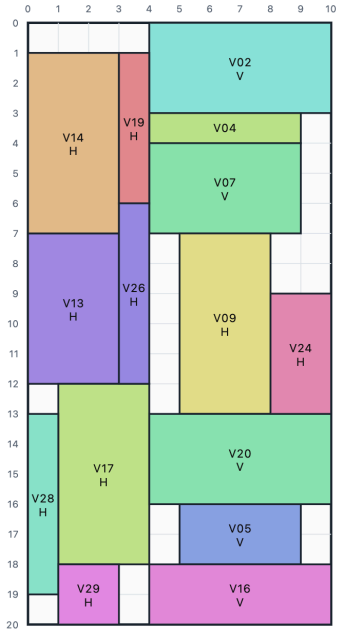
Kenapa state ini?

Generasi 56: best fitness 1025, rata-rata 934.1, terburuk 803.

Best-so-far sejauh ini: 1025. Individu unik: 8 dari 10.

Deck menampilkan individu terbaik pada generasi ini (bukan seluruh populasi dan bukan 'current state').

***State* Akhir**



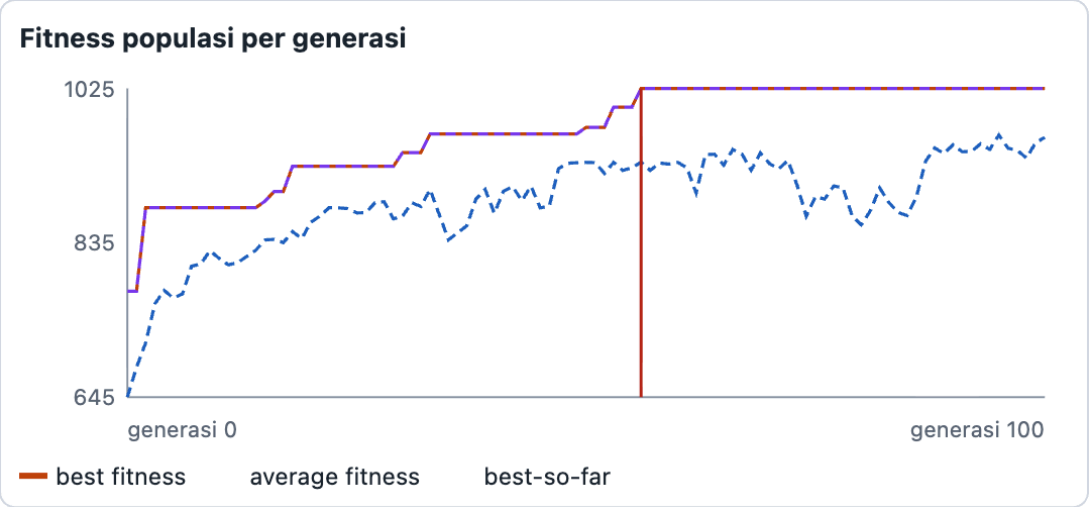
Objective (ShippingFee) **1025**	Berat / MaxCapacity **500 / 500**
Kendaraan dimuat **15 / 30**	Utilisasi area deck **87.0%**
Total potensi fee **1510**	Valid **ya**

Kenapa state ini?

Final: best-found sepanjang semua generasi.

Keterangan	**Hasil**
Seed	5102
Objective function awal	775
Best-found objective	1025
Generasi penemuan best-found	56
Peningkatan nilai objektif	250 (32,26%)

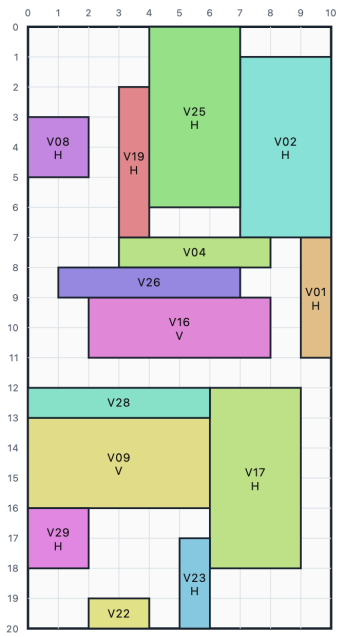
Fitness rata-rata generasi akhir	964,7
Individu unik pada generasi akhir	9 dari 10
Child yang diperbaiki (*repair*)	498 dari 900 (55%)
Total berat dan kendaraan dimuat	500 dari 500, 15 dari 30 kendaraan
Durasi *searching*	0,189 detik
Kondisi berhenti	Banyak generasi tercapai
Plot	



Run ini mencapai nilai tertinggi pada konfigurasi populasi 10 (1025). Rata-rata *fitness* akhir sebesar 964,7 hanya 60 di bawah nilai maksimum, jarak terkecil di antara seluruh run, dan total beratnya tepat 500. Populasi yang rapat seperti ini menandakan pencarian telah mengumpul pada satu area solusi.

Eksperimen 3 - *Genetic Algorithm* (populasi 10, 100 generasi)

***State* Awal**

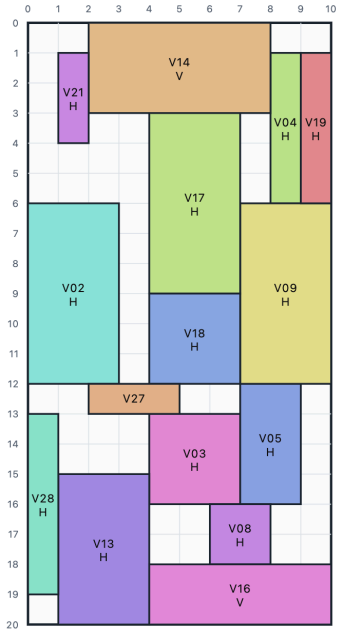


Objective (ShippingFee) **766**	Berat / MaxCapacity **485 / 500**
Kendaraan dimuat **14 / 30**	Utilisasi area deck **61.5%**
Total potensi fee **1510**	Valid **ya**

Kenapa state ini?

Initial: individu terbaik populasi awal.

***State* Best-found (generasi 98)**



Objective (ShippingFee) **1000**	Berat / MaxCapacity **453 / 500**
Kendaraan dimuat **15 / 30**	Utilisasi area deck **75.5%**
Total potensi fee **1510**	Valid **ya**

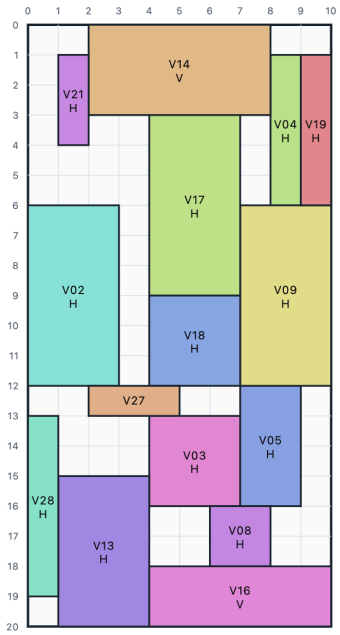
Kenapa state ini?

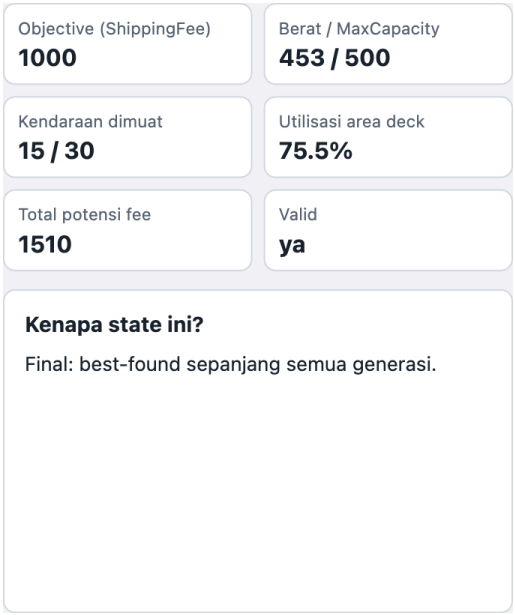
Generasi 98: best fitness 1000, rata-rata 926.3, terburuk 785.

Best-so-far sejauh ini: 1000. Individu unik: 10 dari 10.

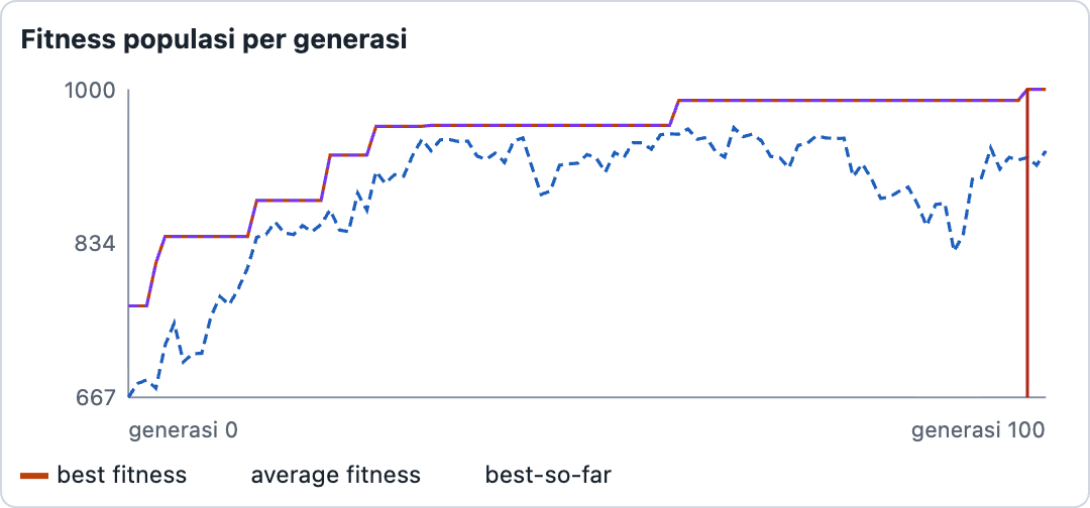
Deck menampilkan individu terbaik pada generasi ini (bukan seluruh populasi dan bukan 'current state').

***State* Akhir**



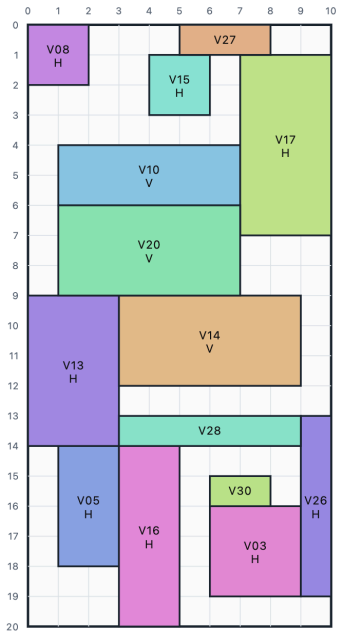


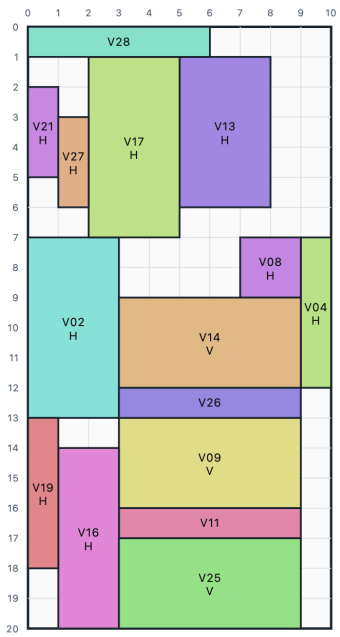
Keterangan	Hasil
Seed	5103
Objective function awal	766
Best-found objective	1000
Generasi penemuan best-found	98
Peningkatan nilai objektif	234 (30,55%)
Fitness rata-rata generasi akhir	933,5
Individu unik pada generasi akhir	9 dari 10
Child yang diperbaiki (*repair*)	435 dari 900 (48%)
Total berat dan kendaraan dimuat	453 dari 500, 15 dari 30 kendaraan
Durasi *searching*	0,188 detik
Kondisi berhenti	Banyak generasi tercapai
Plot	



Nilai akhir 1000 adalah yang terendah pada konfigurasi ini, dan baru ditemukan pada generasi 98 dari 100. Meskipun populasinya hanya 10 individu, hasilnya masih berada dalam selisih 25 dari run terbaik pada konfigurasi yang sama.

Eksperimen 4 - *Genetic Algorithm* (populasi 50, 100 generasi)

***State* Awal**	
	Objective (ShippingFee) **790** Berat / MaxCapacity **499 / 500** Kendaraan dimuat **14 / 30** Utilisasi area deck **67.5%** Total potensi fee **1510** Valid **ya** **Kenapa state ini?** Initial: individu terbaik populasi awal.
***State* Best-found (generasi 100)**	



Objective (ShippingFee)	Berat / MaxCapacity
1001	**458 / 500**
Kendaraan dimuat	Utilisasi area deck
15 / 30	**77.5%**
Total potensi fee	Valid
1510	**ya**

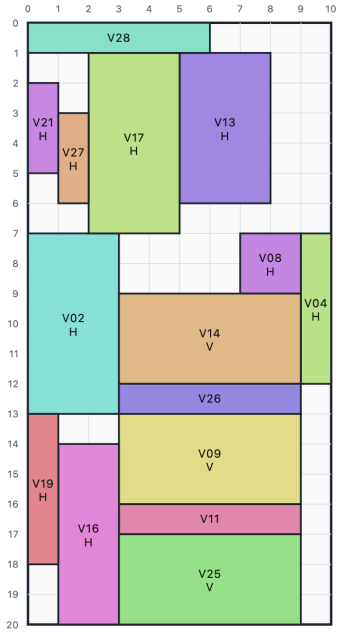
Kenapa state ini?

Generasi 100: best fitness 1001, rata-rata 825.8, terburuk 675.

Best-so-far sejauh ini: 1001. Individu unik: 49 dari 50.

Deck menampilkan individu terbaik pada generasi ini (bukan seluruh populasi dan bukan 'current state').

***State* Akhir**



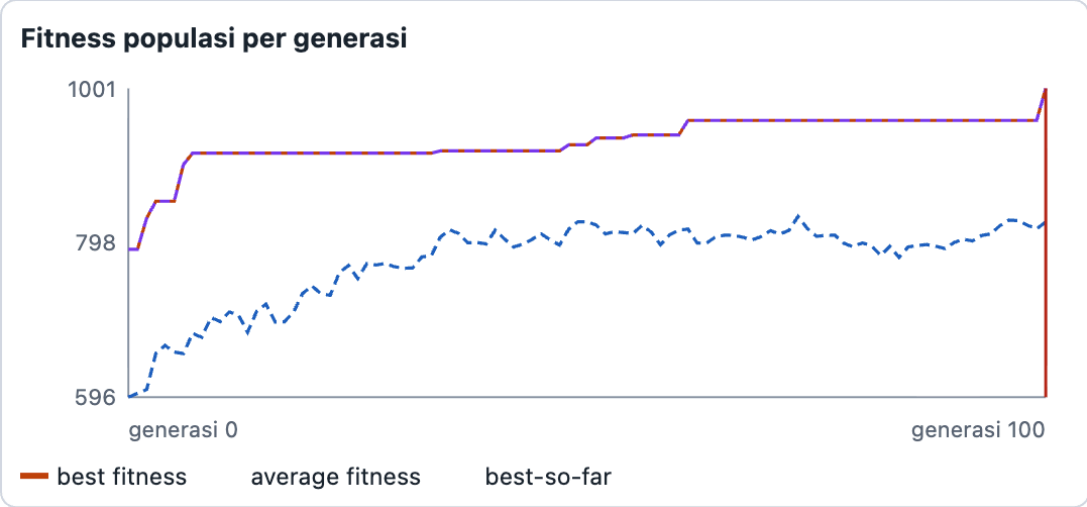
Objective (ShippingFee)	Berat / MaxCapacity
1001	**458 / 500**
Kendaraan dimuat	Utilisasi area deck
15 / 30	**77.5%**
Total potensi fee	Valid
1510	**ya**

Kenapa state ini?

Final: best-found sepanjang semua generasi.

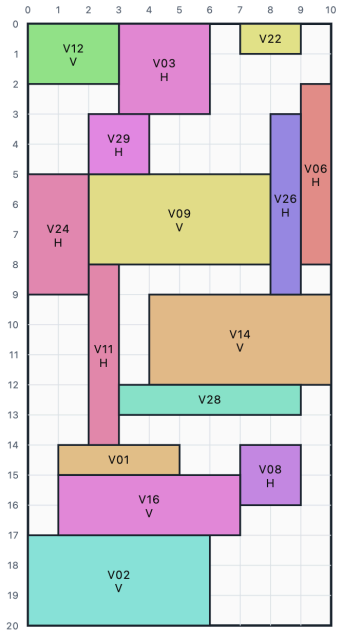
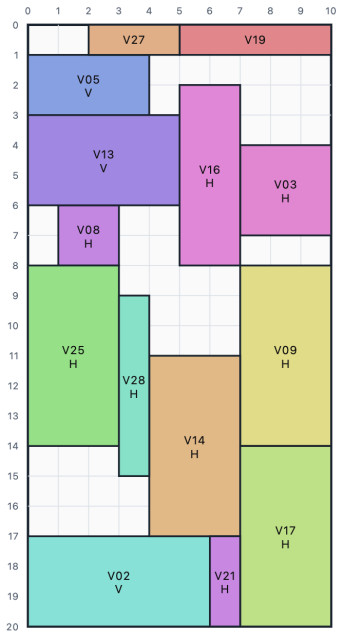
Keterangan	Hasil
Seed	5104
Objective function awal	790
Best-found objective	1001
Generasi penemuan best-found	100
Peningkatan nilai objektif	211 (26,71%)

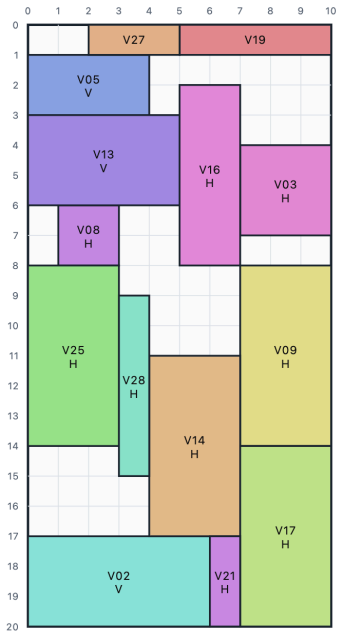
Fitness rata-rata generasi akhir	825,8
Individu unik pada generasi akhir	49 dari 50
Child yang diperbaiki (*repair*)	3.338 dari 4.900 (68%)
Total berat dan kendaraan dimuat	458 dari 500, 15 dari 30 kendaraan
Durasi *searching*	1,192 detik
Kondisi berhenti	Banyak generasi tercapai
Plot	

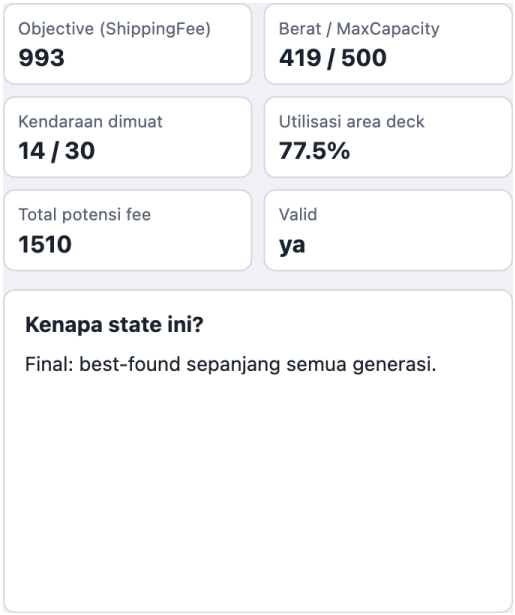


Nilai terbaik pada run ini ditemukan tepat pada generasi 100, yaitu generasi terakhir, sehingga pencarian masih membaik ketika dihentikan. Nilai akhir 1001 adalah yang terendah kedua pada konfigurasi populasi 50 di skenario ini.

Eksperimen 5 - *Genetic Algorithm* (populasi 50, 100 generasi)

***State* Awal**	
	Objective (ShippingFee) **775** Berat / MaxCapacity **494 / 500** Kendaraan dimuat **15 / 30** Utilisasi area deck **63.5%** Total potensi fee **1510** Valid **ya** **Kenapa state ini?** Initial: individu terbaik populasi awal.
***State* Best-found (generasi 52)**	
	Objective (ShippingFee) **993** Berat / MaxCapacity **419 / 500** Kendaraan dimuat **14 / 30** Utilisasi area deck **77.5%** Total potensi fee **1510** Valid **ya** **Kenapa state ini?** Generasi 52: best fitness 993, rata-rata 831.7, terburuk 682. Best-so-far sejauh ini: 993. Individu unik: 49 dari 50. Deck menampilkan individu terbaik pada generasi ini (bukan seluruh populasi dan bukan 'current state').
***State* Akhir**	





Keterangan	Hasil
Seed	5105
Objective function awal	775
Best-found objective	993
Generasi penemuan best-found	52
Peningkatan nilai objektif	218 (28,13%)
Fitness rata-rata generasi akhir	784,6
Individu unik pada generasi akhir	50 dari 50
Child yang diperbaiki (*repair*)	3.602 dari 4.900 (74%)
Total berat dan kendaraan dimuat	419 dari 500, 14 dari 30 kendaraan
Durasi *searching*	1,235 detik
Kondisi berhenti	Banyak generasi tercapai
Plot	



Eksperimen kelima menghasilkan nilai terendah pada konfigurasi ini (993). Total berat solusinya hanya 419 dari 500, jadi 81 satuan kapasitas tidak terpakai, yang mengindikasikan bahwa pembatas pada susunan ini kemungkinan adalah luas geladak, bukan kapasitas berat. Nilai terbaik muncul pada generasi 52 dan tidak membaik sesudahnya.

Eksperimen 6 - *Genetic Algorithm* (populasi 50, 100 generasi)

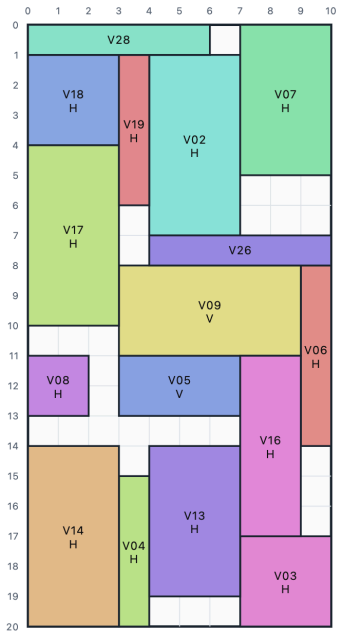
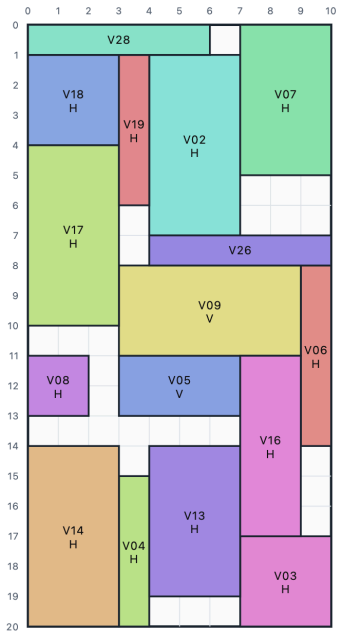
***State* Awal**



Objective (ShippingFee)	Berat / MaxCapacity
794	**497 / 500**
Kendaraan dimuat	Utilisasi area deck
15 / 30	**66.5%**
Total potensi fee	Valid
1510	**ya**

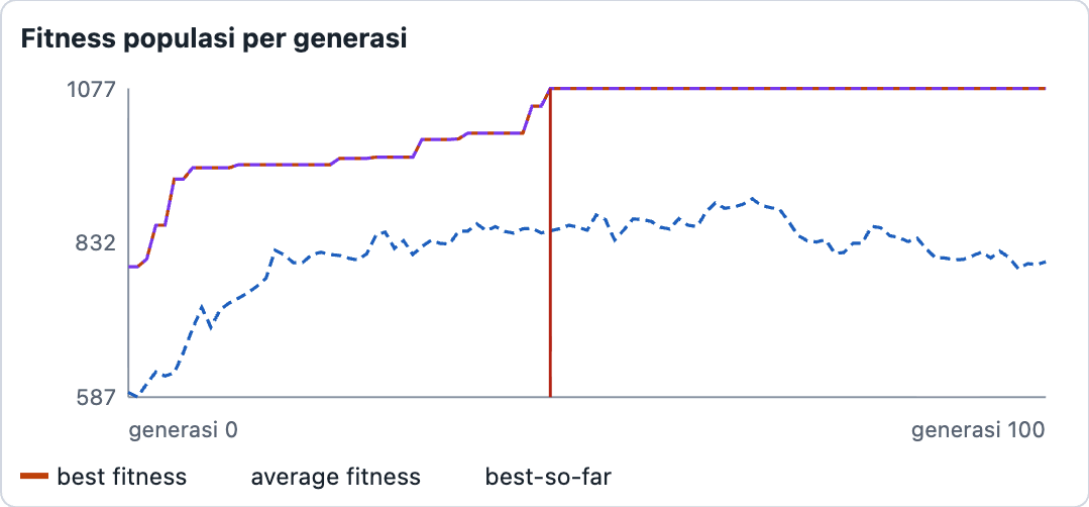
Kenapa state ini?

Initial: individu terbaik populasi awal.

State Best-found (generasi 46)	
	Objective (ShippingFee) **1077** Berat / MaxCapacity **489 / 500** Kendaraan dimuat **16 / 30** Utilisasi area deck **86.0%** Total potensi fee **1510** Valid **ya** **Kenapa state ini?** Generasi 46: best fitness 1077, rata-rata 851.1, terburuk 710. Best-so-far sejauh ini: 1077. Individu unik: 46 dari 50. Deck menampilkan individu terbaik pada generasi ini (bukan seluruh populasi dan bukan 'current state').
***State* Akhir**	
	Objective (ShippingFee) **1077** Berat / MaxCapacity **489 / 500** Kendaraan dimuat **16 / 30** Utilisasi area deck **86.0%** Total potensi fee **1510** Valid **ya** **Kenapa state ini?** Final: best-found sepanjang semua generasi.

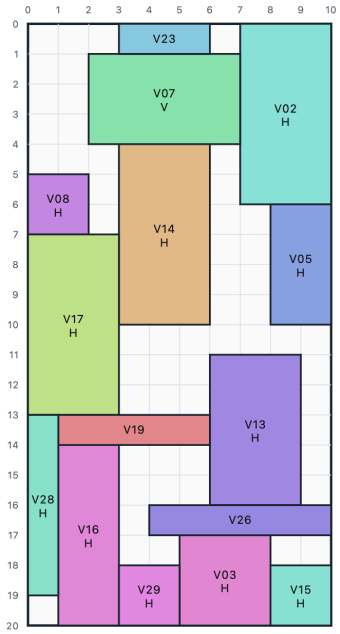
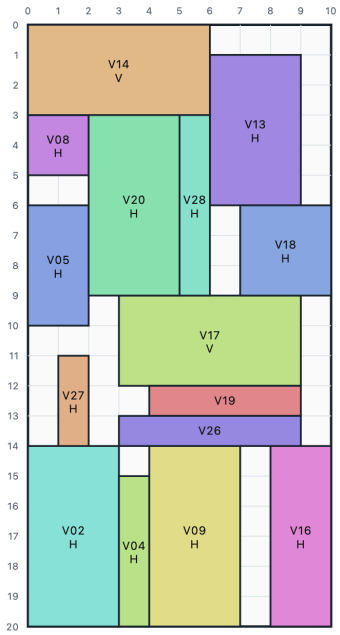
Keterangan	Hasil
Seed	5106
Objective function awal	794
Best-found objective	1077
Generasi penemuan best-found	46

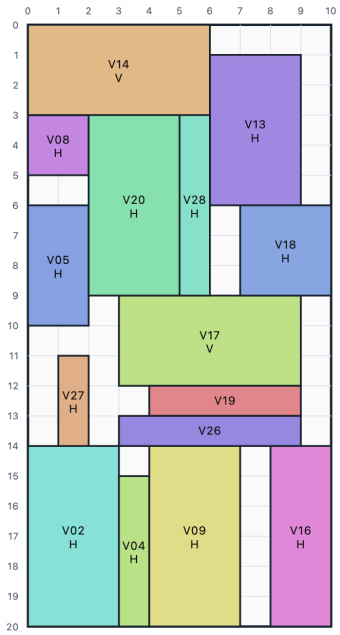
Peningkatan nilai objektif	283 (35,64%)
Fitness rata-rata generasi akhir	801,7
Individu unik pada generasi akhir	49 dari 50
Child yang diperbaiki (*repair*)	3.288 dari 4.900 (67%)
Total berat dan kendaraan dimuat	489 dari 500, 16 dari 30 kendaraan
Durasi *searching*	1,180 detik
Kondisi berhenti	Banyak generasi tercapai
Plot	

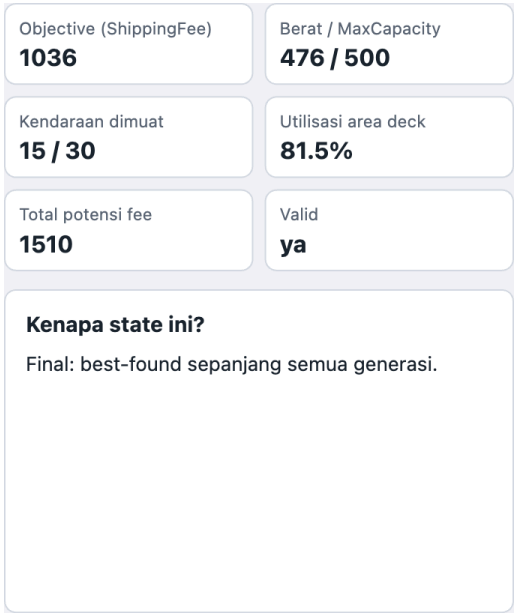


Nilai 1077 adalah tertinggi pada skenario 2 dan tertinggi kedua pada seluruh 18 run. Nilai ini 84 lebih tinggi daripada Eksperimen 5 yang memakai konfigurasi sama (populasi 50, 100 generasi), dan 33 lebih tinggi daripada hasil tertinggi konfigurasi yang sama pada skenario 1 (1044). Selisih sebesar ini menunjukkan bahwa variasi antarseed pada GA cukup besar.

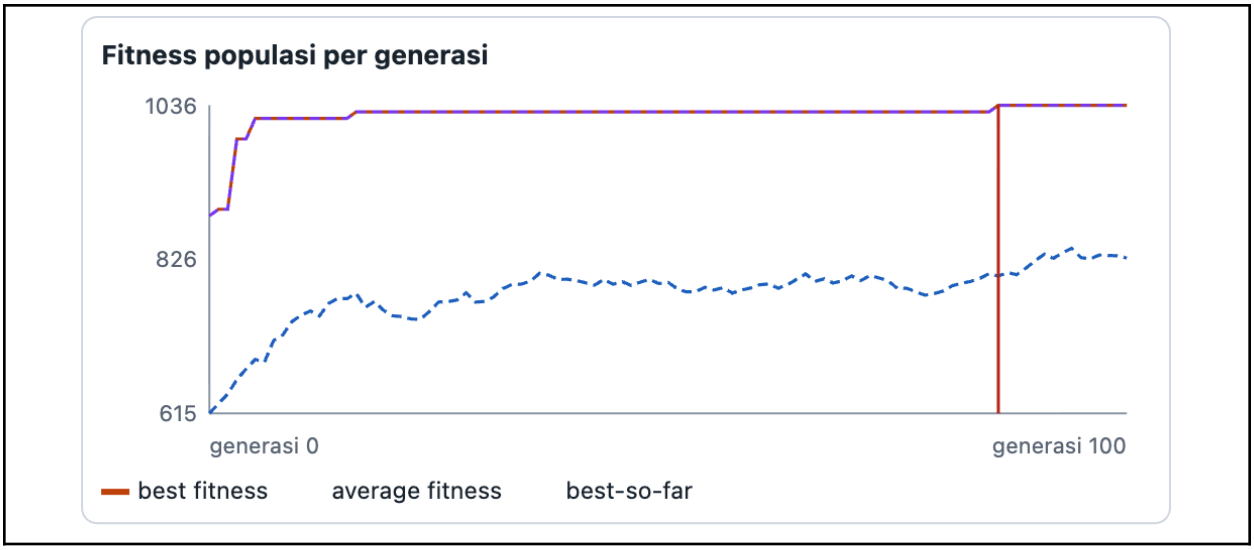
Eksperimen 7 - *Genetic Algorithm* (populasi 200, 100 generasi)

***State* Awal**	
	Objective (ShippingFee) **885** Berat / MaxCapacity **500 / 500** Kendaraan dimuat **15 / 30** Utilisasi area deck **72.5%** Total potensi fee **1510** Valid **ya** **Kenapa state ini?** Initial: individu terbaik populasi awal.
***State* Best-found (generasi 86)**	
	Objective (ShippingFee) **1036** Berat / MaxCapacity **476 / 500** Kendaraan dimuat **15 / 30** Utilisasi area deck **81.5%** Total potensi fee **1510** Valid **ya** **Kenapa state ini?** Generasi 86: best fitness 1036, rata-rata 802.8, terburuk 582. Best-so-far sejauh ini: 1036. Individu unik: 200 dari 200. Deck menampilkan individu terbaik pada generasi ini (bukan seluruh populasi dan bukan 'current state').
***State* Akhir**	



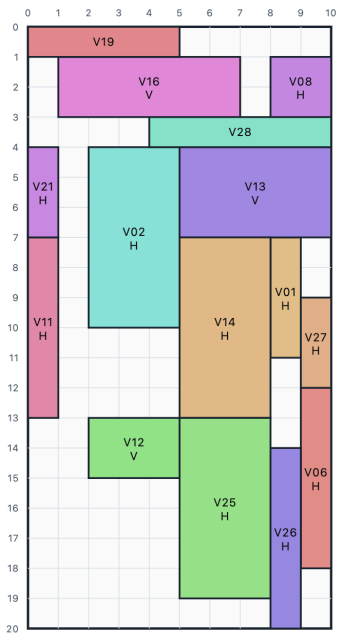


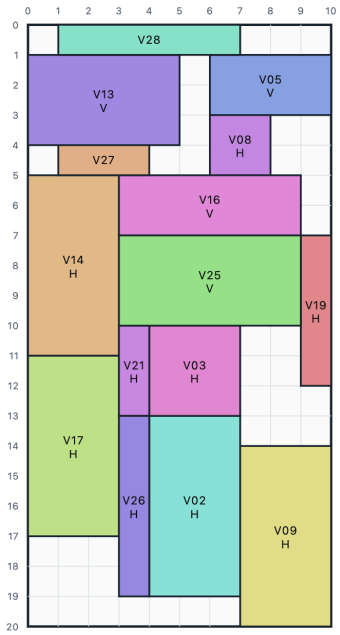
Keterangan	Hasil
Seed	5107
Objective function awal	885
Best-found objective	1036
Generasi penemuan best-found	86
Peningkatan nilai objektif	151 (17,06%)
Fitness rata-rata generasi akhir	827,5
Individu unik pada generasi akhir	196 dari 200
Child yang diperbaiki (*repair*)	15.801 dari 19.900 (79%)
Total berat dan kendaraan dimuat	476 dari 500, 15 dari 30 kendaraan
Durasi *searching*	5,483 detik
Kondisi berhenti	Banyak generasi tercapai
Plot	



Nilai awal 885 adalah yang tertinggi dari seluruh 18 run, karena individu terbaik dipilih dari 200 susunan acak. Akibatnya peningkatannya hanya 17,06% meskipun nilai akhirnya 1036 yang tertinggi pada konfigurasi ini. Durasi 5,483 detik adalah yang terpanjang di skenario 2.

Eksperimen 8 - *Genetic Algorithm* (populasi 200, 100 generasi)

***State* Awal**	
	Objective (ShippingFee) **821** Berat / MaxCapacity **490 / 500** Kendaraan dimuat **15 / 30** Utilisasi area deck **65.0%** Total potensi fee **1510** Valid **ya** **Kenapa state ini?** Initial: individu terbaik populasi awal.
***State* Best-found (generasi 85)**	



Objective (ShippingFee)	Berat / MaxCapacity
1025	**455 / 500**
Kendaraan dimuat	Utilisasi area deck
15 / 30	**80.5%**
Total potensi fee	Valid
1510	**ya**

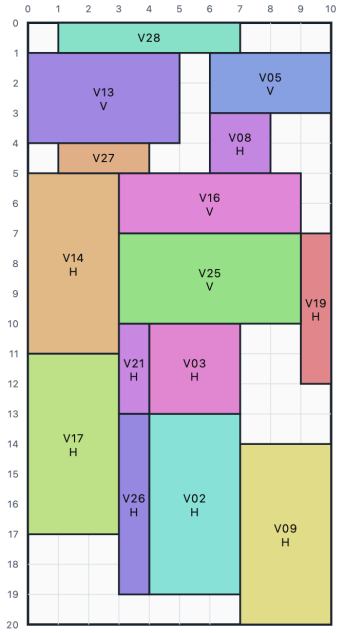
Kenapa state ini?

Generasi 85: best fitness 1025, rata-rata 777.9, terburuk 558.

Best-so-far sejauh ini: 1025. Individu unik: 198 dari 200.

Deck menampilkan individu terbaik pada generasi ini (bukan seluruh populasi dan bukan 'current state').

***State* Akhir**



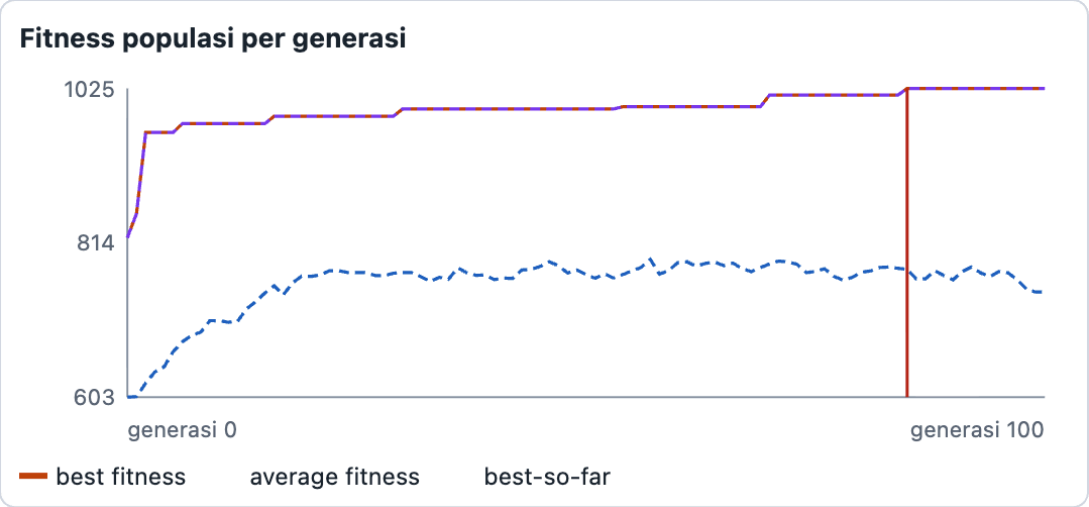
Objective (ShippingFee)	Berat / MaxCapacity
1025	**455 / 500**
Kendaraan dimuat	Utilisasi area deck
15 / 30	**80.5%**
Total potensi fee	Valid
1510	**ya**

Kenapa state ini?

Final: best-found sepanjang semua generasi.

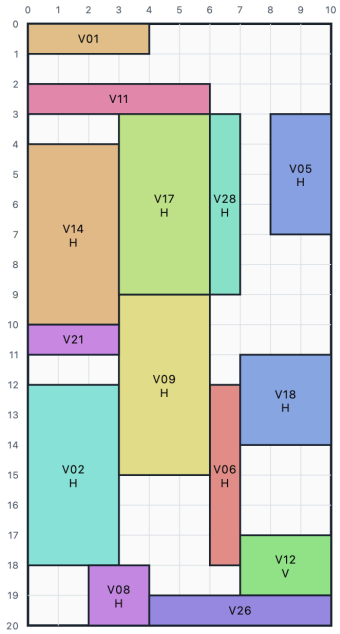
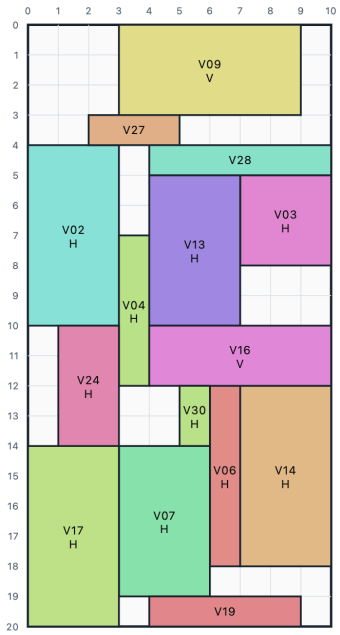
Keterangan	Hasil
Seed	5108
Objective function awal	821
Best-found objective	1025
Generasi penemuan best-found	85
Peningkatan nilai objektif	204 (24,85%)

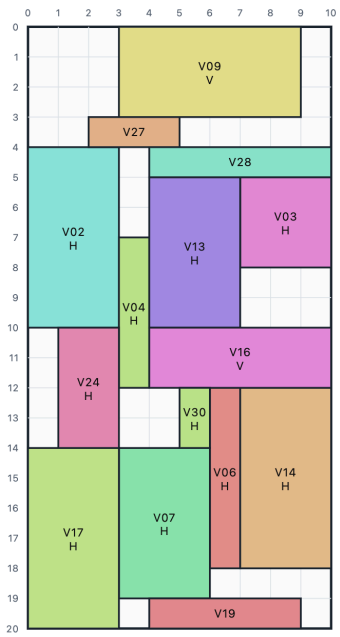
Fitness rata-rata generasi akhir	746,9
Individu unik pada generasi akhir	200 dari 200
Child yang diperbaiki (*repair*)	16.050 dari 19.900 (81%)
Total berat dan kendaraan dimuat	455 dari 500, 15 dari 30 kendaraan
Durasi *searching*	5,455 detik
Kondisi berhenti	Banyak generasi tercapai
Plot	

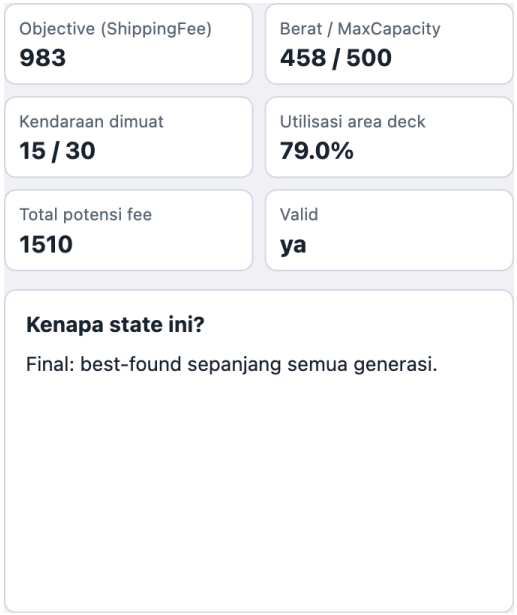


Seluruh 200 individu pada generasi akhir berbeda, dan 81% *child* harus diperbaiki, angka tertinggi dari seluruh run. Rata-rata *fitness* akhir sebesar 746,9 berada 278 di bawah nilai maksimum. Hal ini menunjukkan bahwa pada populasi besar, keragaman terjaga tetapi kualitas populasi secara keseluruhan tertinggal jauh dari individu terbaiknya.

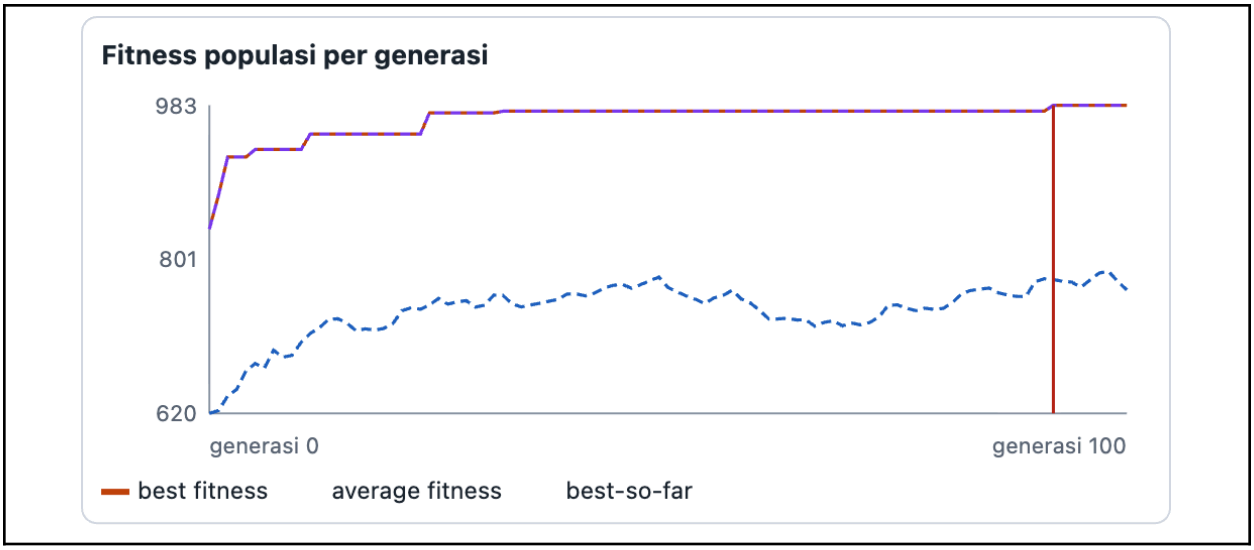
Eksperimen 9 - *Genetic Algorithm* (populasi 200, 100 generasi)

***State* Awal**	
	Objective (ShippingFee) **837** Berat / MaxCapacity **496 / 500** Kendaraan dimuat **14 / 30** Utilisasi area deck **65.0%** Total potensi fee **1510** Valid **ya** **Kenapa state ini?** Initial: individu terbaik populasi awal.
***State* Best-found (generasi 92)**	
	Objective (ShippingFee) **983** Berat / MaxCapacity **458 / 500** Kendaraan dimuat **15 / 30** Utilisasi area deck **79.0%** Total potensi fee **1510** Valid **ya** **Kenapa state ini?** Generasi 92: best fitness 983, rata-rata 777.6, terburuk 488. Best-so-far sejauh ini: 983. Individu unik: 198 dari 200. Deck menampilkan individu terbaik pada generasi ini (bukan seluruh populasi dan bukan 'current state').
***State* Akhir**	





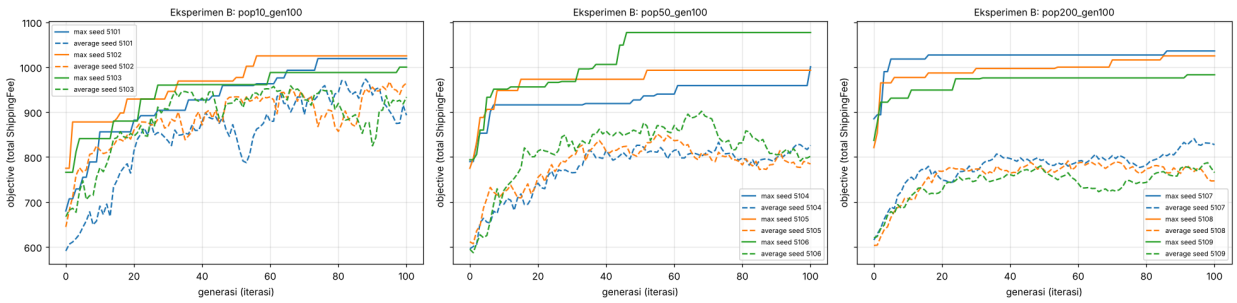
Keterangan	Hasil
Seed	5109
Objective function awal	837
Best-found objective	983
Generasi penemuan best-found	92
Peningkatan nilai objektif	146 (17,44%)
Fitness rata-rata generasi akhir	765,4
Individu unik pada generasi akhir	198 dari 200
Child yang diperbaiki (*repair*)	15.980 dari 19.900 (80%)
Total berat dan kendaraan dimuat	458 dari 500, 15 dari 30 kendaraan
Durasi *searching*	5,322 detik
Kondisi berhenti	Banyak generasi tercapai
Plot	



Eksperimen kesembilan menghasilkan nilai terendah pada skenario 2 dan terendah kedua pada seluruh 18 run, yaitu 983, meskipun memakai populasi terbesar dan durasi lebih dari lima detik. Hasil ini menunjukkan bahwa populasi besar tidak menjamin nilai akhir yang lebih tinggi pada 100 generasi. Ringkasan per konfigurasi dihitung dari tiga run masing-masing. Kolom individu unik menunjukkan banyaknya individu berbeda pada generasi terakhir, dirata-ratakan dari tiga run.

Tabel Ringkasan Variasi Jumlah Populasi

Populasi	Rata2\ objektif akhir	Simpangan baku	Terendah sampai tertinggi	Rata2 *fitness* populasi akhir	Individu unik di generasi akhir	Rata-rata durasi (detik)
10	1014,7	13,1	1000 sampai 1025	930,3	8,3 dari 10	0,175
50	1023,7	46,4	993 sampai 1077	804,1	49,3 dari 50	1,203
200	1014,7	28,0	983 sampai 1036	779,9	198 dari 200	5,420



Gambar. Fitness maksimum (garis tebal) dan rata-rata (garis putus-putus) terhadap generasi pada skenario 2.

Rata-rata objektif akhir pada ketiga ukuran populasi berdekatan, yaitu 1014,7, 1023,7, dan 1014,7. Selisih tertinggi antarkonfigurasi sebesar 9,0 jauh lebih kecil daripada simpangan baku pada populasi 50 yang mencapai 46,4, sehingga dengan tiga run per konfigurasi tidak ada ukuran populasi yang dapat dinyatakan lebih baik. Sebaliknya, durasi sangat dipengaruhi ukuran populasi. Populasi 200 membutuhkan rata-rata 5,420 detik, sekitar 31 kali lebih lama daripada populasi 10 yang hanya 0,175 detik, dan jumlah evaluasi fitnessnya 20 kali lebih banyak (20.200 berbanding 1.010) untuk hasil yang sama.

Ukuran populasi terlihat jelas pada keragaman populasi. Pada populasi 10, rata-rata *fitness* akhir sebesar 930,3 sudah mendekati *fitness* maksimum, dan hanya 8,3 dari 10 individu yang berbeda, sehingga populasi telah menyerupai satu susunan saja. Pada populasi 200, rata-rata *fitness* akhir sebesar 779,9 masih jauh di bawah maksimum dan 198 dari 200 individu berbeda. Populasi besar mempertahankan keragaman, tetapi 100 generasi tidak cukup untuk memanfaatkannya. Nilai objektif awal pada populasi 200 juga lebih tinggi (rata-rata 847,7 berbanding 740,3 pada populasi 10) karena individu terbaik dipilih dari sampel acak yang lebih besar, sehingga persentase peningkatannya lebih kecil. Hasil terbaik pada skenario ini adalah 1077 pada *seed* 5106 dengan populasi 50 (Eksperimen 6).

3.3.3 Analisis Fitness Maksimum, Rata-Rata, dan Durasi

Keenam konfigurasi dirangkum pada tabel berikut. Setiap baris adalah rata-rata dari tiga run.

Tabel Perbandingan Enam Konfigurasi

Konfigurasi	Evaluasi *fitness*	*Fitness* maksimum akhir	*Fitness* rata-rata populasi akhir	*Child* yang perlu *repair*	Durasi (detik)	Durasi per 1000 evaluasi (detik)
Populasi 50, 25 generasi	1.300	975,0	776,3	74%	0,330	0,254
Populasi 50, 100 generasi (skenario 1)	5.050	1034,7	855,9	72%	1,283	0,254
Populasi 50, 400 generasi	20.050	1047,7	806,2	69%	4,784	0,239
Populasi 10, 100 generasi	1.010	1014,7	930,3	49%	0,175	0,173
Populasi 50, 100 generasi (skenario 2)	5.050	1023,7	804,1	70%	1,203	0,238
Populasi 200, 100 generasi	20.200	1014,7	779,9	80%	5,420	0,268

***Fitness* maksimum.** *Fitness* maksimum tidak pernah turun pada seluruh 18 run karena *elitism* menyalin individu terbaik ke generasi berikutnya. Kurvanya berbentuk tangga, yaitu naik cepat pada fase awal lalu bertambah dalam lompatan kecil yang jarang. Peningkatan terbesar terjadi pada 20 sampai 30 generasi pertama. Setelah itu, perbaikan datang dari kombinasi yang kebetulan lebih baik, bukan dari kenaikan stabil. Nilai tertinggi yang dicapai GA adalah 1081 (*seed* 5009), sedangkan rata-rata nilai akhir dari 18 run adalah 1018,4.

***Fitness* rata-rata.** Rata-rata populasi naik dari sekitar 600 menjadi 780 sampai 930, lalu berhenti naik dan bergerak naik turun. Jaraknya terhadap *fitness* maksimum

tetap besar, yaitu sekitar 180 sampai 240 pada populasi 50 dan 200. Ada dua penyebab yang dapat kami lihat dari data. Pertama, *roulette wheel* memberi peluang terpilih yang hanya sedikit lebih besar bagi individu terbaik. Pada populasi 50 dengan *fitness* maksimum sekitar 1035 dan rata-rata sekitar 856, peluang individu terbaik hanya sekitar 1,2 kali peluang uniform karena seluruh *fitness* berada pada orde yang sama. Tekanan seleksi yang lemah ini membuat populasi tidak terdorong merapat ke individu terbaik. Kedua, *crossover* satu titik menggabungkan penempatan dari dua susunan yang berbeda, sehingga 69% sampai 80% *child* pada populasi 50 dan 200 melanggar constraint dan harus diperbaiki dengan mengeluarkan sebagian kendaraan. *Child* hasil perbaikan cenderung lebih buruk daripada *parent*-nya, yang menahan kenaikan rata-rata. Pada populasi 10, *parent* saling mirip sehingga hanya 49% *child* yang perlu diperbaiki dan rata-rata populasi akhir mencapai 930,3.

Durasi. Durasi *searching* bertambah hampir lurus terhadap jumlah evaluasi *fitness*, yaitu sekitar 0,17 sampai 0,27 detik per 1000 evaluasi. Dengan demikian durasi ditentukan oleh hasil kali populasi dan generasi, bukan oleh salah satunya. Profil eksekusi pada populasi 50 dengan 50 generasi menunjukkan bahwa fungsi *repair* menghabiskan sekitar 96% dari waktu total, sehingga biaya terbesar GA ini bukan pada penghitungan *fitness*, melainkan pada perbaikan *child*. Hal ini juga menjelaskan mengapa durasi per evaluasi lebih kecil pada populasi 10 (0,173 detik), yang *child*-nya lebih jarang membutuhkan *repair*, dan lebih besar pada populasi 200 (0,268 detik), yang *child*-nya paling sering membutuhkan *repair*.

Bagaimana pengaruh banyak generasi dan jumlah populasi pada GA? Pada skenario dengan populasi tetap, rata-rata nilai akhir meningkat dari 975,00 pada 25 generasi menjadi 1034,67 pada 100 generasi dan 1047,67 pada 400 generasi. Kenaikan dari 100 ke 400 generasi hanya 13 poin, sementara durasi yang tercatat naik dari 1,283 menjadi 4,784 detik. Pada skenario jumlah populasi, rata-ratanya adalah 1014,67 untuk populasi 10, 1023,67 untuk populasi 50, dan 1014,67 untuk populasi 200. Jadi, menambah generasi membantu pada konfigurasi yang diuji, meskipun kenaikannya makin kecil. Memperbesar populasi belum menunjukkan peningkatan nilai akhir yang konsisten, sementara pekerjaan komputasinya bertambah. Hasil skenario ini tetap perlu dibaca dalam batas tiga *run* untuk setiap konfigurasi.

Secara keseluruhan, tambahan generasi dan tambahan populasi sama-sama menaikkan biaya secara proporsional, tetapi manfaatnya cepat menurun. Konfigurasi populasi 10 dengan 100 generasi mencapai rata-rata 1014,7 dalam 0,175 detik, nilai yang sama dengan populasi 200 yang membutuhkan 5,420 detik. Jika yang dikejar adalah efisiensi, populasi 10 dengan 100 generasi paling unggul. Jika yang dikejar adalah nilai rata-rata tertinggi, populasi 50 dengan 400 generasi memberi 1047,7 dengan biaya 4,784 detik.

3.4 Perbandingan Hasil Antar Algoritma

3.4.1 Perbandingan Nilai Objective Function

Perbandingan menggunakan dataset synthetic-2026, dengan tiga run untuk setiap konfigurasi. Kualitas solusi dinilai dari total shipping fee kendaraan yang dimuat. Semakin tinggi nilai objektif, semakin baik solusi yang diperoleh.

Tabel Perbandingan Nilai *Objective Function* Antaralgoritma

Algoritma	Rata-rata	Terendah	Tertinggi
Simulated Annealing	1076	1053	1101
HC random restart	1072,33	1042	1101
HC sideways move	1070,33	1053	1087
GA, populasi 50 dan 400 generasi	1047,67	1031	1081
GA, populasi 50 dan 100 generasi	1034,67	1025	1044
HC stochastic, pendekatan ppt kuliah	901,33	844	962
HC stochastic, pendekatan AIMA	875	799	961

SA menghasilkan rata-rata tertinggi, tetapi selisihnya terhadap HC *random restart* dan *sideways move* hanya 3,67 dan 5,67. HC *random restart* juga mencapai nilai terbaik 1101. Dengan demikian, hasil pengujian menunjukkan bahwa ketiga metode tersebut menghasilkan kualitas yang berdekatan, sementara GA dan kedua pendekatan HC *stochastic* memperoleh rata-rata lebih rendah. Perbedaan ini berkaitan dengan cara algoritma melanjutkan *searching*. SA dapat menerima penurunan nilai, *sideways move* menerima perpindahan bernilai sama, dan *random restart* mencoba kembali dari solusi awal yang berbeda. Ketiganya memberi kesempatan untuk menemukan perbaikan setelah satu lintasan tidak lagi menghasilkan peningkatan. Pada GA, penggabungan penempatan melalui *crossover* dapat menghasilkan individu tidak valid yang perlu diperbaiki, sehingga sebagian kendaraan mungkin dikeluarkan selama proses *repair*.

Seberapa dekat hasilnya dengan *global optimum*? Nilai optimum sebenarnya belum diketahui. Solusi *feasible* terbaik yang ditemukan bernilai 1101, dicapai oleh SA dan HC *random restart*. Nilai ini menjadi batas bawah. Perhitungan *knapsack* berdasarkan kapasitas berat menghasilkan batas atas 1155, tetapi mengabaikan apakah kendaraan dapat ditempatkan di geladak tanpa tumpang tindih. Jadi, jika perhitungan batas atas tersebut benar, berlaku:

$$1101 \leq f^{*} \leq 1155$$

dengan f^{*} sebagai nilai *global optimum*. Hasil terbaik yang ditemukan mencapai 95,3% dari batas atas dan berjarak paling banyak 54 poin dari optimum. Angka 1101 belum dapat disebut optimum global karena masih mungkin ada penataan *feasible* dengan nilai lebih tinggi.

3.4.2 Perbandingan Durasi dan Konsistensi Hasil

Bagaimana perbandingan durasinya? Durasi SA pada tiga *run* yang dilaporkan adalah 0,352, 0,384, dan 2,543 detik. Namun, angka durasi HC pada tabel perbandingan sebelumnya berasal dari kelompok hasil yang berbeda dengan nilai HC terbaru. Karena itu, kami belum menggunakan angka tersebut untuk menyatakan algoritma mana yang paling cepat. Perbandingan durasi yang adil perlu memakai *run* dengan konfigurasi,

cakupan pengukuran, dan kondisi komputer yang sebanding. Dari sisi work per step, SA dan HC *stochastic* mengambil kandidat acak, sedangkan HC *sideways move* memeriksa lebih banyak *neighbor*, HC *random restart* mengulang *searching* dari beberapa titik awal, dan GA mengevaluasi banyak individu dalam setiap generasi. Karena pekerjaannya berbeda, jumlah iterasi atau generasi saja tidak cukup untuk membandingkan kecepatan.

Tabel Perbandingan Durasi dan Konsistensi Hasil

Algoritma	**Rata2 durasi (detik)**	**Durasi terhadap SA**	**Simpangan baku objektif**	**Koefisien variasi**	**Selisih tertinggi & terendah**
Simulated Annealing	0,184	1,0 kali	24,1	2,2%	48
Hill-Climbing stochastic	0,287	1,6 kali	59,1	6,6%	118
Hill-Climbing stochastic (buku teks)	0,326	1,8 kali	20,3	2,3%	40
Genetic Algorithm (populasi 50, 100 generasi)	1,283	7,0 kali	9,5	0,9%	19
Hill-Climbing random restart	1,647	9,0 kali	42,3	4,2%	59
Hill-Climbing sideways move	3,215	17,5 kali	41,5	4,0%	81
Genetic Algorithm (populasi 50, 400 generasi)	4,784	26,0 kali	28,9	2,8%	50

Seberapa konsisten hasil tiap algoritma? Pada tiga *run* yang ditampilkan, GA dengan 100 generasi memiliki selisih nilai terkecil, yaitu 19. Akan tetapi, hasilnya juga lebih rendah daripada SA, HC *random restart*, dan HC *sideways move*. Di antara tiga metode dengan rata-rata tertinggi, HC *sideways move* memiliki sebaran paling sempit, disusul SA dan HC *random restart*. Kedua pendekatan HC *stochastic* memiliki sebaran paling lebar sekaligus rata-rata terendah. Jadi, hasil yang konsisten perlu dibaca bersama kualitas solusinya: nilai yang selalu berdekatan belum tentu merupakan nilai yang baik. Tiga *run* per konfigurasi juga masih terlalu sedikit untuk memastikan pola konsistensi tersebut berlaku secara umum.

3.4.3 Analisis Kelebihan dan Keterbatasan Algoritma

Tabel Kelebihan dan Keterbatasan Tiap Algoritma

Algoritma	Kelebihan pada pengujian ini	Keterbatasan pada pengujian ini
HC stochastic	Implementasinya sederhana, algoritma mengambil kandidat dan hanya berpindah jika nilainya meningkat.	Dua pendekatan yang diuji menghasilkan rata-rata 901,33 dan 875,00, paling rendah di antara metode yang dibandingkan. Selisih nilai tertinggi dan terendahnya juga besar, yaitu 118 dan 162.
HC sideways move	Perpindahan bernilai sama memberi kesempatan untuk mengubah penataan tanpa kehilangan shipping fee. Rata-ratanya mencapai 1070,33, hanya 5,67 poin di bawah SA, dengan selisih antar-run sebesar 34.	Tidak setiap perubahan bernilai sama membuka jalan menuju solusi yang lebih baik. Jumlah langkah sideways juga dibatasi, sehingga searching dapat berhenti sebelum menemukan perbaikan berikutnya.
HC random restart	Beberapa titik awal memberi kesempatan mencoba penataan yang berbeda. Rata-ratanya 1072,33 dan salah satu run mencapai 1101, menyamai hasil terbaik SA.	Setiap restart mengulang proses searching. Hasil tiga run masih berkisar dari 1042 sampai 1101, sehingga pilihan titik awal tetap berpengaruh.

Simulated Annealing	Rata-rata nilai akhirnya paling tinggi, yaitu 1076,00. Penerimaan kandidat yang lebih buruk memungkinkan algoritma meninggalkan penataan yang sulit diperbaiki melalui langkah yang selalu menaikkan nilai.	Hasilnya bergantung pada temperatur awal dan laju pendinginan yang dipilih. Karena kandidat yang lebih buruk dapat diterima, final state tidak selalu menjadi state terbaik yang ditemukan selama searching.
Genetic Algorithm	Populasi memungkinkan beberapa penataan diproses dalam satu generasi, sementara elitism menjaga individu terbaik. Pada populasi 50, penambahan generasi dari 100 menjadi 400 menaikkan rata-rata hasil dari 1034,67 menjadi 1047,67.	Penambahan generasi tersebut hanya memberi kenaikan 13 poin pada rata-rata hasil. Individu hasil crossover juga dapat melanggar constraints dan memerlukan repair. Pada konfigurasi 400 generasi, rata-ratanya masih di bawah SA serta HC random restart dan sideways move.

Algoritma mana yang paling sesuai untuk kasus ini? Jika ukurannya rata-rata nilai objektif, SA unggul tipis. Jika ukurannya nilai terbaik yang pernah dicapai, SA berbagi posisi teratas dengan HC *random restart*. HC *sideways move* juga layak dipertimbangkan karena rata-ratanya hampir sama dengan keduanya dan sebaran hasilnya lebih kecil. Kami belum dapat memilih algoritma tercepat dari data durasi yang saat ini sebanding. Dengan demikian, pada dataset ini SA merupakan pilihan kuat dari sisi kualitas rata-rata, tetapi belum ada dasar untuk menyebutnya paling baik dalam setiap ukuran.

Apa yang masih bisa diperbaiki? Untuk seluruh algoritma, pengujian pada lebih banyak *seed* dan dataset akan memperjelas apakah perbedaan yang terlihat memang bertahan. Pada GA, pengaruh metode seleksi, *crossover*, dan *repair* dapat diuji satu per satu untuk melihat bagian mana yang paling membatasi hasil. Pada SA, beberapa jadwal pendinginan dapat dibandingkan; pada HC, jumlah *restart* dan batas langkah *sideways* dapat divariasikan. Usulan tersebut masih berupa arah eksperimen berikutnya, belum merupakan hasil yang dibuktikan oleh pengujian ini.

BAB IV
KESIMPULAN & SARAN

4.1 Hasil Eksperimen Hill-Climbing

Hasil eksperimen menunjukkan bahwa pemilihan vehicle dan penataannya sama-sama penting untuk memperoleh total shipping fee yang tinggi. Memenuhi kapasitas berat saja belum tentu menghasilkan solusi yang baik, karena setiap vehicle memiliki ukuran, berat, dan shipping fee yang berbeda. Seluruh solusi akhir yang dilaporkan tetap memenuhi batas area penempatan, tidak saling tumpang tindih, dan tidak melebihi kapasitas kapal.

Dari algoritma yang diuji, Simulated Annealing memperoleh rata-rata nilai objektif tertinggi, yaitu 1076,00. Meski begitu, hasil HC random restart dan HC sideways move tidak jauh berbeda, dengan rata-rata 1072,33 dan 1070,33. HC random restart bahkan mencapai nilai terbaik 1101, sama seperti Simulated Annealing. Sementara itu, kedua pendekatan HC stochastic menghasilkan nilai rata-rata yang lebih rendah dan hasil antar-run yang lebih bervariasi. Jadi, pada kasus ini, kemampuan melanjutkan searching dari penataan yang sulit diperbaiki tampak berpengaruh terhadap kualitas hasil.

Pada Genetic Algorithm, penambahan jumlah generasi meningkatkan rata-rata nilai akhir, tetapi kenaikannya semakin kecil. Dengan populasi 50, rata-rata hasil naik dari 975,00 pada 25 generasi menjadi 1034,67 pada 100 generasi, lalu 1047,67 pada 400 generasi. Sebaliknya, memperbesar populasi pada skenario yang diuji belum memberi peningkatan nilai yang konsisten. Hasil GA masih berada di bawah tiga metode dengan rata-rata tertinggi, meskipun algoritma ini dapat mengolah beberapa kandidat solusi dalam setiap generasi.

Solusi terbaik yang ditemukan bernilai 1101. Nilai tersebut belum dapat dinyatakan sebagai global optimum, sebab perhitungan berdasarkan kapasitas berat saja masih memberikan batas atas 1155. Dengan demikian, nilai optimum untuk dataset ini berada di antara 1101 dan 1155. Secara keseluruhan, Simulated Annealing dan Hill Climbing random restart merupakan algoritma yang unggul.

4.2 Saran

1. Eksperimen dapat diulang dengan lebih banyak *seed* dan dataset yang berbeda. Dengan begitu, kita dapat melihat apakah hasil yang berdekatan antara *Simulated Annealing*, HC *random restart*, dan HC *sideways move* tetap terjadi ketika jumlah atau karakteristik *vehicle* berubah.
2. Durasi seluruh algoritma perlu diukur ulang pada kondisi komputer dan cakupan pengukuran yang sama. Hasil tersebut akan membuat perbandingan waktu lebih jelas, terutama karena setiap algoritma melakukan pekerjaan yang berbeda dalam satu iterasi atau generasi.
3. Pengujian parameter dapat diperluas secara bertahap, misalnya jadwal penurunan temperatur pada *Simulated Annealing*, jumlah *restart* pada HC, serta jumlah generasi dan populasi pada GA. Setiap parameter sebaiknya diubah secara terpisah agar pengaruhnya terhadap kualitas solusi dan durasi *searching* lebih mudah dilihat.
4. Untuk memastikan nilai *global optimum*, persoalan ini dapat dicoba dengan metode eksak yang memperhitungkan berat sekaligus posisi dan orientasi setiap *vehicle*. Nilai optimal tersebut nantinya dapat menjadi pembanding yang lebih pasti bagi hasil ketiga algoritma.

BAB V
PEMBAGIAN TUGAS

Berikut merupakan pembagian tugas dalam pengerjaan tugas besar IF3070 Dasar Intelegensi Artifisial.

Tabel 4.1 Pembagian Tugas

NIM	Nama	Pembagian Tugas
18224009	Keisha Daffa Aryani	Mengerjakan Implementasi *Simulated Annealing*
		Mengerjakan visualisasi dari *Simulated Annealing*
		Menyediakan template dan mengerjakan laporan Bab 1, Bab 2 yang berkaitan dengan Gambaran Umum Solusi, Objective Function & State, dan Simulated Annealing, Bab 3 yang berkaitan dengan Simulated Annealing, Bab 4, dan Referensi
18224025	Muhammad Faris Daffa	Mengerjakan *Genetic Algorithm*
		Mengerjakan visualisasi dari *Genetic Algorithm*
		Mengerjakan Front-End, Core, dan Baseline awal
18224027	Nafhan Shafy Aulia	Mengerjakan semua varian *Hill Climbing*

		Setup *boilerplate* untuk struktur kode dan juga *Github Repo*
		Merapikan Front End
		Mencari referensi implementasi kode *python* yang berkaitan dengan algoritma yang digunakan
		Mengerjakan Bab 2 dan juga Bab 3 yang berkaitan dengan *Hill Climbing*

REFERENSI

Brownlee, J. (2021). *Stochastic hill climbing in Python from scratch.* Machine Learning Mastery. https://machinelearningmastery.com/stochastic-hill-climbing-in-python-from-scratch/

Hajek, B. (1988). Cooling schedules for optimal annealing. *Mathematics of Operations Research, 13*(2), 311–329. https://doi.org/10.1287/moor.13.2.311

Levine, J., & Ducatelle, F. (2004). Ant colony optimisation and local search for bin packing and cutting stock problems. *Journal of the Operational Research Society, 55*(7), 705–716. https://doi.org/10.1057/palgrave.jors.2601723

Nourani, Y., & Andresen, B. (1998). A comparison of simulated annealing cooling strategies. *Journal of Physics A: Mathematical and General, 31*(41), 8373–8385. https://doi.org/10.1088/0305-4470/31/41/003

Rahnamayan, S., Tizhoosh, H. R., & Salama, M. M. A. (2007). A novel population initialization method for accelerating evolutionary algorithms. *Computers & Mathematics with Applications, 53*(10), 1605–1614. https://doi.org/10.1016/j.camwa.2006.08.039

Russell, S., & Norvig, P. (2009). *Artificial intelligence: A modern approach* (3rd ed.). Prentice Hall.

Russell, S., Norvig, P., & contributors. (n.d.). *aima-python* [Source code]. GitHub. https://github.com/aimacode/aima-python (diakses 22 Oktober 2026)

Perry, M. (n.d.). *simanneal* [Source code]. GitHub. https://github.com/perrygeo/simanneal (diakses 22 Oktober 2026)

De Rainville, F.-M., Fortin, F.-A., Gardner, M.-A., Parizeau, M., & Gagné, C. (n.d.). *DEAP* [Source code]. GitHub. https://github.com/DEAP/deap (diakses 22 Oktober 2026)

Gad, A. F. (n.d.). *PyGAD / GeneticAlgorithmPython* [Source code]. GitHub. https://github.com/ahmedfgad/GeneticAlgorithmPython (diakses 22 Oktober 2026)

Hayes, G. (n.d.). *mlrose* [Source code]. GitHub. https://github.com/gkhayes/mlrose (diakses 22 Oktober 2026)

secnot. (n.d.). *rectpack* [Source code]. GitHub. https://github.com/secnot/rectpack (diakses 22 Oktober 2026)

Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by simulated annealing. Science, 220(4598). https://www2.stat.duke.edu/~scs/Courses/Stat376/Papers/TemperAnneal/KirkpatrickAnnealScience1983.pdf

LAMPIRAN

Lampiran 1. Tautan GitHub

https://github.com/nafhansa/IF3070_G08_MachineUnlearning

SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:

Nama/NIM	:	Keisha Daffa Aryani/18224009, Muhammad Faris Daffa/18224025, Nafhan Shafy Aulia/18224027
Fakultas/Sekolah	:	Sekolah Teknik Elektro dan Informatika
Kode Mata Kuliah	:	IF3070
Nama Mata Kuliah	:	Dasar Artificial Intelligence
Judul Tugas/Proposal	:	Optimasi Penempatan Kendaraan Pada Kapal Menggunakan *Local Search*

Menyatakan bahwa kami menggunakan AI dalam penyusunan luaran tugas pada mata kuliah yang tertulis di atas dengan BATASAN:

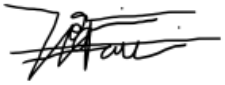
Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	-
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya	Level 3, AI Assisted Workflow. Terbatas pada pemberian saran struktur dan penyingkatan. Substansi, hasil eksperimen, analisis, dan kesimpulan disusun fundamentally oleh kelompok.
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	Level 3, AI Assisted Workflow. Digunakan sebagai sarana diskusi untuk mengklarifikasi konsep yang telah diberikan selama kuliah, brainstorming ide, konsep pengerjaan, serta membantu memahami pesan error pada kode. Pemahaman, analisis, dan keputusan akhir tetap ditentukan oleh kelompok.
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	-

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 22 Oktober 2026



Muhammad Faris Daffa
18224025



(Nafhan Shafy Aulia)
18224027



(Keisha Daffa Aryani)
18224009